GAOL v5

NOT Just Another
Interval Arithmetic Library

Edition 5.0
Last updated 4 November 2026

Jordan Ninin
Frédéric Goualard
ENSTA, France — LS2N, Nantes Université, France

Copyright © 2001 Swiss Federal Institute of Technology, Switzerland
Copyright © 2002-2016 LINA UMR CNRS 6241, France
Copyright © 2017-2026 LS2N UMR CNRS 6004, France
Copyright © 2026 ENSTA, France
All Trademarks, Copyrights and Trade Names are the property of their respective owners even if they are not specified below.
Part of the work was done while Frédéric Goualard was a postdoctorate at the *Swiss Federal Institute of Technology, Lausanne, Switzerland* supported by the *European Research Consortium for Informatics and Mathematics* fellowship programme.
This is edition 5.0 of the documentation of GAOL v5, which follows the manual of GAOL 4 by Frédéric Goualard. It is consistent with version 5.0.0 of the `gaol` library.
Permission is granted to make and distribute verbatim copies of this manual provided the copyright notice and this permission notice are preserved on all copies.

GAOL IS PROVIDED "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THIS SOFTWARE IS WITH YOU. SHOULD THIS SOFTWARE PROVE DEFECTIVE, YOU ASSUME THE COST OF ALL NECESSARY SERVICING, REPAIR OR CORRECTION.

Recipriversexcluson. n. *A number whose existence can only be defined as being anything other than itself.*

Douglas Adams, 1982
Life, the Universe and Everything

Contents

Copyright **ix**

1 Introduction **1**
1.1 GAOL v5 . . . 1
1.2 About this manual . . . 2

2 Installation **3**
2.1 Supported platforms . . . 3
2.2 Getting the software . . . 3
2.3 Prerequisites . . . 4
2.3.1 Mandatory tools . . . 4
2.3.2 Compilers and options refused . . . 4
2.3.3 Tools for maintainers . . . 5
2.3.4 Optional tools . . . 6
2.4 Building and installing with CMake . . . 6
2.4.1 Options . . . 6
2.4.2 The archive of the sources and the packages . . . 8
2.5 Building and installing with autotools . . . 9
2.6 Building and installing with meson . . . 10
2.7 Tests . . . 11
2.8 What is installed . . . 12

3 An overview of GAOL **13**
3.1 Compiling a program using GAOL . . . 14
3.1.1 From CMake . . . 15
3.1.2 From pkg-config . . . 16
3.2 The rounding direction . . . 17
3.2.1 What the upward rounding does to the program . . . 18
3.2.2 The NEON intervals . . . 21
3.2.3 Flush-to-zero and denormals-are-zero . . . 21
3.3 The floating-point exceptions . . . 24
3.4 The namespaces . . . 27
3.5 The powers in the two namespaces . . . 28
3.6 Common errors . . . 29
3.6.1 Floating-point arithmetic and rounding . . . 29
3.6.2 Functions of numbers . . . 30
3.6.3 Midpoint and radius . . . 30
3.6.4 The doubles of the program . . . 31
3.6.5 Zero . . . 31
3.6.6 The sign of a zero bound . . . 31
3.6.7 The empty set and the domain of a function . . . 31
3.6.8 Relations, sorting and containers . . . 32

3.6.9 Both namespaces . . . 32
3.6.10 Bisection . . . 33
3.6.11 Newton's method: the division and the relational division 33
3.6.12 Reading text . . . 33
3.6.13 Threads and the global settings . . . 34
3.6.14 The flags of interval arithmetic . . . 34

4 Initialization and cleanup 35

5 Interval creation and assignment 39
5.1 Constructors . . . 39
5.2 Straight assignment . . . 42
5.3 Assignment combined with an operation . . . 43

6 Interval constants 45

7 Interval relations 47
7.1 Set relations . . . 47
7.2 Certainly relations . . . 50
7.3 The order of IEEE 1788-2015 . . . 51
7.4 Relational Symbols . . . 52
7.5 Intervals in containers and algorithms . . . 53
7.6 Interval-specific relations . . . 54

8 Interval Arithmetic 57
8.1 Functional Arithmetic . . . 58
8.1.1 Basic operations . . . 58
8.1.2 Powers and roots . . . 59
8.1.3 Exponentials and logarithms . . . 61
8.1.4 Trigonometric functions . . . 62
8.1.5 Hyperbolic functions . . . 64
8.1.6 Other functions . . . 65
8.2 Relational Arithmetic . . . 66
8.2.1 $(n+1)$-ary relational functions . . . 67
8.3 Cancellative addition and subtraction . . . 72

9 Interval functions 73
9.1 Integer functions . . . 77
9.2 Splitting methods . . . 78
9.3 Union and intersection . . . 79

10 Input/output 81
10.1 Reading intervals . . . 81
10.1.1 Input format . . . 82
10.2 Writing intervals . . . 86
10.2.1 Converting intervals to strings . . . 86
10.2.2 Output format . . . 86
10.2.3 Choosing the number of digits to display . . . 89
10.2.4 Example . . . 90

11 The names of IEEE 1788-2015 93
11.1 Where the standard and GAOL differ . . . 94
11.2 The operations . . . 95
11.3 Reading with the names of the standard . . . 96
11.4 What is not provided . . . 97
11.5 Coming from another library . . . 97

12 Accuracy of the operations **101**
12.1 Accuracy modes . . . 101
12.2 Tightness of each operation . . . 102

13 Floating-point numbers **105**
13.1 Floating-point constants . . . 105
13.2 Floating-point functions . . . 105

14 Manipulating the FPU **107**
14.1 Rounding functions . . . 107

15 Version information **109**

16 Additional functions **111**

17 Error handling **113**
17.1 Exceptions . . . 114
17.1.1 The `std::invalid_argument` exception . . . 115
17.1.2 The `std::domain_error` exception . . . 115
17.2 Warnings . . . 115

18 Debugging facilities **117**

19 Additional Documentation **119**
19.1 Documentation on GAOL . . . 119
19.2 References . . . 119

20 Reporting bugs **121**

21 Contributors **123**

22 Moving a program from GAOL 4 to GAOL v5 **125**
22.1 Compiling and linking . . . 125
22.2 Results that change . . . 126

Library Copying **129**

Copyright

GAOL is distributed under the GNU Library General Public License, version 2 (see Section 22.2, page 129). The copyright for the initial version—named `cell`—(year 2001) is owned by the *Swiss Federal Institute of Technology*, Lausanne, Switzerland. The copyright for the following versions (from 2002 onward) is owned by the *Laboratoire d'Informatique de Nantes-Atlantique*, then by the *Laboratoire des Sciences du Numérique de Nantes*, France.

The files GAOL v5 adds carry the copyright of ENSTA, France, and are distributed under the same license as GAOL; the files of GAOL that GAOL v5 changes keep the copyright of their authors.

GAOL v5 computes its elementary functions with CORE-MATH [13], whose sources are distributed with GAOL (in `3rd/math-core`) and compiled into its library. CORE-MATH is distributed under the MIT license, which is reproduced with the license of GAOL (see Section 22.2, page 129).

[13] Alexei Sibidanov, Paul Zimmermann, and Stéphane Glondu. The CORE-MATH project. In *2022 IEEE 29th Symposium on Computer Arithmetic (ARITH)*, pages 26–34, 2022.

1 Introduction

GAOL[†] is a C++ library to perform arithmetic with floating-point intervals. The development of GAOL was initiated at the *Swiss Federal Institute of Technology*, Lausanne, Switzerland, while F. Goualard was a post-doctorate fellow supported by the *Swiss National Science Foundation*. It started as a limited version of *Jail* (now *halloween*), a templated C++ interval library developed during Goualard's PhD.

To our knowledge, a unique feature of GAOL among the C++ interval arithmetic libraries available when it was written is the implementation of *relational arithmetic operators* required by interval constraint arithmetic software (see Section 8.2, page 66). Hence, the game of the name: GAOL is *not* JAIL (*Just Another Interval Library*). The standard for interval arithmetic, IEEE 1788-2015 [9], has since required such operators, as its *reverse functions*.

[†] For those readers who are not native English speakers, "gaol" should be pronounced jāl, like the word "jail", of which it is a, chiefly British, variant.

[9] IEEE. *IEEE Standard for Interval Arithmetic*. Institute of Electrical and Electronics Engineers, 2015. IEEE Std 1788-2015, doi:10.1109/IEEESTD.2015.7140721.

1.1 GAOL v5

GAOL v5, written by Jordan Ninin at ENSTA, continues GAOL as Frédéric Goualard developed it, from its version 4.2.2. It was written for Codac, whose intervals are built upon GAOL, and brings the changes GAOL needs to compile and compute right on every system it can: Linux, macOS and Windows, on x86, x86-64, ARM, 64-bit ARM and the other processors of Debian. What it brings, and this manual describes:

- **The tightest bounds for the elementary functions.** Every elementary function is bounded with CORE-MATH [13], which is correctly rounded in the rounding direction in effect: the value GAOL computes at a bound, rounding upward, *is* the upper bound, and the double below it the lower one. The bounds are the same on every architecture and with every compiler, and there is no mathematical library to install, to find or to link along with GAOL: CORE-MATH is compiled into it.
- **IEEE 1788-2015.** GAOL v5 follows the standard for interval arithmetic in the operations GAOL provides: the empty set, the infinite bounds, the domains of the functions, the comparisons and the interval literals. The few cases where it differs, mostly the `pow` of the namespace `gaol`, are stated where the operations are described, and the tightness of each operation is documented, as the standard requires (Chapter 12).

[13] Alexei Sibidanov, Paul Zimmermann, and Stéphane Glondu. The CORE-MATH project. In *2022 IEEE 29th Symposium on Computer Arithmetic (ARITH)*, pages 26–34, 2022.

- **New operations**, which the standard requires or recommends: `exp2`, `exp10`, `log2`, `log10`, `fma`, the integer functions `sign`, `trunc`, `round_ties_to_even` and `round_ties_to_away`, sixteen recommended functions (`expm1`, `log1p`, `hypot`, `compoundm1`, `sinpi`...), the cancellative subtraction and addition, the radius, and the comparisons `less`, `strictly_less`, `is_entire` and `is_common_interval`; `atan2`, which GAOL declared, is implemented.
- **The names of the standard.** The namespace `gaol_ieee1788` gives each operation of the standard that GAOL provides the name and the argument order the standard gives it (Chapter 11).
- **The rounding direction is the business of GAOL.** Each operation that computes a bound sets the rounding direction upward when it is not, whichever direction the calling code left: the user no longer has to keep it upward (see Section 3.2, page 17).
- **Exact input and output.** Numbers are read exactly, and intervals written in decimal are rounded outward, whatever the C library; written in hexadecimal, an interval is read back as the same doubles.
- **Three builds that agree.** A CMake build, which is the one to use, and the autotools and meson builds of GAOL, which configure GAOL the same way; tests of the bounds GAOL computes against their exact values; a continuous integration on Linux, macOS and Windows, with the compilers and on the processors listed in Section 2.1.

A program written for GAOL 4 needs a few changes, which Chapter 22 lists: first of all, `gaol/gaol.h` no longer opens the namespace `gaol`.

1.2 About this manual

This document is both a manual and a reference to use GAOL. It assumes a prior knowledge of interval arithmetic. Refer to the books and papers by Goldberg, Neumaier, and others [2, 8, 10, 1, 11] for a basic presentation of floating-point arithmetic, interval arithmetic and the use thereof.

The main entry point for interval arithmetic on the Web is the *Interval Computation site* of Vladik Kreinovich, `http://www.cs.utep.edu/interval-comp/`.

The pages of the directory `doc` of the sources give the details of each change GAOL v5 brings, with its measures (see Section 19, page 119). The mark [GAOL v5] follows what behaves otherwise than in GAOL 4.

The examples of the reference show what they print. An example that writes a `bool` sets `boolalpha` first: without it, `cout` writes 1 and 0.

Classes, methods, functions, macros, constants and variables available in the library but not described in this document are likely to change or to be removed. Consequently, they should be used with caution, if at all. Among them are the intervals of floats, `gaol::intervalf` and `gaol::interval2f`, which are unfinished and which none of the builds compiles: only a developer of GAOL does, defining `GAOL_FLOAT_INTERVALS` in `gaol/gaol_config.h`.

[2] David Marc Goldberg. What every computer scientist should know about floating-point arithmetic. *ACM Computing Surveys*, 23(1):5–48, March 1991.

[8] IEEE. IEEE standard for binary floating-point arithmetic. Technical Report IEEE Std 754-1985, Institute of Electrical and Electronics Engineers, 1985. Reaffirmed 1990.

[10] Ramon Edgar Moore. *Interval Analysis*. Prentice-Hall, Englewood Cliffs, N. J., 1966.

[1] Götz Alefeld and Jürgen Herzberger. *Introduction to Interval Computations*. Academic Press Inc., New York, USA, 1983. Traduit par Jon Rokne de l'original Allemand *'Einführung In Die Intervallrechnung'*.

[11] Arnold Neumaier. *Interval methods for systems of equations*, volume 37 of *Encyclopedia of Mathematics and its Applications*. Cambridge University Press, 1990.

2 Installation

GAOL v5 is built with CMake, which is the build to use. The autotools and meson builds of GAOL are kept for those who use them: on a given machine with a given compiler, GAOL behaves the same whichever build configured it, the three giving it the same macros and the same flags.

2.1 Supported platforms

The continuous integration of GAOL v5 builds it and runs its tests on:

- Linux: Ubuntu 22.04, 24.04 and 26.04 on x86-64 and 64-bit ARM, with GCC and Clang; Debian 12 and 13 on x86-64, 64-bit ARM and 32-bit ARM (armhf), and Debian 12 on 32-bit x86; manylinux_2_28 on x86-64 and 64-bit ARM; Alpine (musl) on x86-64 and 64-bit ARM; Debian 13 on s390x, ppc64le and riscv64;
- macOS 14, 15 and 26, on arm64 and x86-64, with AppleClang, the Clang of LLVM and GCC;
- Windows: Visual Studio 2022 and 2026 on x86, x64 and arm64; MinGW-w64 GCC 14 and 15 on x64, 12 to 15 on x86; MSYS2 (UCRT64 with GCC, CLANG64 and CLANGARM64 with Clang); Cygwin (GCC, x64).

The bounds GAOL computes are the same on all of them. On 64-bit ARM processors, GAOL computes its intervals with NEON instructions, whatever the compiler: GCC, Clang, AppleClang, clang-cl, Visual C++ and the Clang of MinGW-w64 (see Section 3.2.2, page 21). [GAOL v5]

2.2 Getting the software

GAOL v5 is available at `https://github.com/Jordan08/GAOL`. The GAOL of Frédéric Goualard, which GAOL v5 continues, is available at `https://github.com/goualard-f/GAOL`, and presented in the GAOL section of `https://frederic.goualard.net`.

GAOL comes as source code, to be compiled for the computer it is used on. The sources of CORE-MATH are part of it (`3rd/math-core`): there is nothing else to download.

2.3 Prerequisites

2.3.1 Mandatory tools

- A C++11 and C99 compiler: GCC, Clang, AppleClang, Visual C++ (Visual Studio 2022 or later), clang-cl (the Clang of Visual Studio, Clang 16 or later), or GCC and Clang for Windows (MinGW-w64, MSYS2, Cygwin).
- CMake 3.14 or later; or, for the autotools build, a shell and `make`; or, for the meson build, meson and ninja.

No mathematical library is needed: CORE-MATH is compiled into GAOL.

2.3.2 Compilers and options refused

Each of the following gave bounds not enclosing the exact results or was not tested. The three builds refuse the compilers when configuring, the C compiler, which builds CORE-MATH, as well as the C++ compiler, with a message naming what to use instead, and keep the options away by giving GAOL the flags of interval arithmetic (see Section 3.1, page 14); `gaol/gaol_config.h` refuses them again when compiling, for the code including the headers of GAOL too, all but the second one, which no macro of the compiler shows:

- Clang for 32-bit ARM processors, which does not honour the rounding direction there (GCC does);
- a compiler saying of `-frounding-math` "overriding currently unsupported rounding mode on this target", as Clang 14 for 64-bit ARM (Clang 18 honours the rounding direction there);
- a MinGW-w64 whose `fma()` or `round()` is wrong. On x64, before mingw-w64 12, and linked with `msvcrt.dll` rather than the UCRT (MSYS2 MINGW64, the cross compilers of Debian and Ubuntu), they are those of the math library of mingw-w64, computed in doubles: that `fma()` is not correctly rounded, and CORE-MATH computes with it where the compiler has no `-mfma` (`GAOL_FMA OFF`; GCC for Windows on x64 and on 32-bit x86 takes it by default, and these toolchains stay refused, their `round()` being wrong too); that `round()` depends on the rounding direction. On 32-bit x86, before mingw-w64 11, as the mingw-w64 9 of WinLibs, whose `fma()` rounds each of its partial sums to a double. On ARM, as on x64, before mingw-w64 12 or linked with `msvcrt.dll`, not tested. A toolchain of mingw-w64 12 or later linking the UCRT (MinGW-Builds GCC 14 and 15, MSYS2 UCRT64 and CLANG64) takes those of `ucrtbase.dll`, which pass the tests;
- Cygwin without the fused multiply-add instructions (`GAOL_FMA OFF`, or a processor without them): its `fma()` is not correctly rounded (`fma(1 + 2^-30, 1 - 2^-30, -1)` gives 0), and `std::fma` is a call of it without optimization; GAOL calls the instruction instead (`-mfma`), and Cygwin is built and tested with it;
- `-ffast-math`, `-Ofast` and `/fp:fast`, with which the compiler rounds to nearest and drops the checks of NaN and infinities;
- `-ffinite-math-only`, which `-ffast-math` and `-Ofast` turn on: the compiler takes NaN and infinities never to occur, in the inline functions of GAOL too, and the empty interval, whose bounds are NaN, is no longer told empty;

- Visual C++ without `/fp:strict`, which then assumes rounding to nearest, and clang-cl from Clang 16 without it, which then compiles with `-fno-rounding-math -ffp-contract=on`; [GAOL v5]
- doubles computed on the x87 unit of 32-bit x86 processors, in extended precision (without `-msse2 -mfpmath=sse`, `/arch:SSE2` with Visual C++, or `-msse2` with clang-cl, which computes them on the x87 unit with `/arch:SSE2`): CORE-MATH assumes every operation on doubles rounded to a double.

`-fno-fast-math`, one of the flags of interval arithmetic, turns `-ffast-math` and `-ffinite-math-only` off when it comes after them on the command line, and does nothing when `-ffast-math` or `-ffinite-math-only` comes after `-fno-fast-math`, where the compilation stops. With Clang, `-ffast-math` and `-Ofast` turn `-frounding-math` off too, which `-fno-fast-math` does not turn on again: the remedy is all the flags of interval arithmetic after the option, not `-fno-fast-math` alone. With GCC, `-fno-fast-math` turns off all the fast-math optimizations of `-Ofast` but `-fcx-limited-range`, which changes the arithmetic of complex numbers alone (GCC 9.4), wherever it stands on the command line, before `-Ofast` too: `-frounding-math` stays, and `-Ofast` is not refused there. No macro of the compiler shows the following, which `gaol/gaol_config.h` cannot refuse and which give wrong results all the same: `-funsafe-math-optimizations` and `-ffast-math -fno-finite-math-only` with GCC, which rewrites the addition by which GAOL sees the rounding direction, so that an operation does not set it upward again after the code using GAOL left it elsewhere; `-fno-signed-zeros`, with which GCC and Clang drop the addition of 0 by which it sees the flush-to-zero mode (see Section 3.2.3, page 21), and which the `-funsafe-math-optimizations` of Clang implies; `-fno-honor-nans` alone with Clang, which does to the empty interval what `-ffinite-math-only` does; and `-ffast-math` or `-Ofast` followed by `-fno-fast-math` with Clang, which leaves `-frounding-math` off. The code using GAOL is not to be compiled with them.
The link of the program is not refused, and with the options `-Ofast`, `-ffast-math` and `-funsafe-math-optimizations` it makes the bounds wrong, whatever the flags the code using GAOL was compiled with: the modes it sets flush the subnormal results and operands to zero (see Section 3.2.3, page 21). [GAOL v5]

2.3.3 Tools for maintainers

GAOL uses code produced by GNU Flex and GNU Bison for parsing the expressions used to initialize an interval. The three builds compile the lexer and the parser committed with the sources of `gaol/parser`, generated by flex 2.6.4 and bison 3.8.2 from `gaol/parser/gaol_interval_lexer.lpp` and `gaol/parser/gaol_interval_parser.ypp`:

```
% gaol/parser/gaol_interval_lexer.cpp
% gaol/parser/gaol_interval_parser.cpp
% gaol/parser/gaol_interval_parser.h
```

No build generates them again by itself, and none needs flex nor bison. After changing one of the two files, the target `parser` of the three builds (`make parser`, `ninja parser`) generates the lexer and the parser again in `gaol/parser`, with the flex and the bison of the system, and they are committed; `gaol/parser/regenerate_parser.sh` runs the same commands without a build:

```
% sh gaol/parser/regenerate_parser.sh
```

2.3.4 Optional tools

- `flex` and `bison`, to generate the lexer and the parser again (`make parser`);
- bash, and Python 3 with numpy, for `make perf` (see Section 2.7, page 11);
- `pdflatex`, `bibtex` and `makeindex`, to build this manual.

2.4 Building and installing with CMake

In the root directory of the sources:

```
% cmake -S . -B build -DCMAKE_INSTALL_PREFIX=prefix \
        -DWITH_TESTS=ON
% cmake --build build --config Release
% cmake --build build --target test
% cmake --install build --config Release
```

The build compiles the thirty-eight sources of CORE-MATH GAOL uses into `libgaol`, which is a static library unless `BUILD_SHARED_LIBS` is `ON`: there is nothing else to build and nothing else to install. The build type is `Release` unless another is given; brought in by a project that gives none (`add_subdirectory`, `FetchContent`), GAOL and CORE-MATH are compiled as in `Release` all the same: `-O3` (`/O2` with Visual C++), `NDEBUG` and the optimizations of `configure`. With a generator of several configurations, GAOL built alone puts `Release` first among them, which Ninja Multi-Config builds by default; `cmake -build` with Visual Studio builds `Debug` all the same without `--config`. [GAOL v5]

2.4.1 Options

The options are given to the first command, as `-D`*`option`*`=`*`value`*:

- `CMAKE_BUILD_TYPE`. `Release` by default; `Debug` builds GAOL without optimization, with debugging information, and enables the debugging facilities (see Section 18, page 117);
- `CMAKE_INSTALL_PREFIX`. Where `cmake -install` puts GAOL;
- `BUILD_SHARED_LIBS`. `OFF` by default; `ON` builds the shared library `libgaol.so.5.0.0`, whose soname is `libgaol.so.5`, as `configure` and meson do (always a static library with Visual C++);
- `CMAKE_POSITION_INDEPENDENT_CODE`. `ON` by default: GAOL is compiled as position-independent code (`-fPIC`), so that it can be linked into a shared library, as Python bindings; a project building GAOL with `FetchContent` that sets it is followed;
- `WITH_TESTS`. `OFF` by default: builds the unit tests of `tests`, which `make test` and `make check` run (see Section 2.7, page 11);
- `WITH_EXAMPLES`. `OFF` by default: builds the examples of `examples`, which `make check` runs;

- `GAOL_SIMD`. `ON` by default: the intervals are computed with the vector instructions of the processor as soon as it has them, their two bounds in one register: SSE2 instructions on x86 processors (`-msse2 -msse3`; no flag with Visual C++ and clang-cl for x64; on 32-bit Windows with GCC and Clang and `-faligned-new=8`, which the code using GAOL is given too, and with Visual C++ and clang-cl, whose intervals keep their two bounds as two doubles and need no aligned memory), and NEON instructions on 64-bit ARM processors, with every compiler and no flag (see Section 3.2.2, page 21); the FPU computes them otherwise, and on any other processor (32-bit ARM, POWER, s390x, RISC-V...). A universal build of macOS takes the NEON intervals for its arm64 part and the SSE2 intervals for its x86_64 part. `GAOL_ARM64`, `--enable-arm64` and `enable-arm64` were an option of the NEON intervals, independent of `GAOL_SIMD`, and are gone; [GAOL v5]

- `GAOL_PRESERVE_ROUNDING`. `OFF` by default: each operation restores the rounding direction it found, rather than leaving it upward (see Section 3.2, page 17);

- `GAOL_COVERAGE`. `OFF` by default: compiles GAOL and its tests with the counters of gcov, and adds the target `coverage`, which runs the tests and writes the report of their coverage (GCC and Clang, with `Debug`; `--enable-coverage` and the target `make coverage` of `configure`, `enable-coverage` and the target `ninja gaol-coverage` of meson).

These options, and those of CMake itself, are those of the three builds. [GAOL v5] The rest of the configuration has no option: it is decided by checks of the compiler and of the machine, which a program compiled and run by the configuration makes, since GAOL assumes that the compiler, the architecture and the system that configure and compile it are those of the code using it. The checks are told in the messages of `cmake`:

- the fused multiply-add instructions of the processor: GAOL and CORE-MATH are compiled with them whenever the machine building runs a program compiled with them, with the flag the compiler has for them: `-mfma` on x86 (`-mfma -mprefer-vector-width=128` with GCC for Windows, on x86-64 and on 32-bit x86), `/arch:AVX2` with Visual C++ for x64, `-mfpu=neon-vfpv4 -mfloat-abi=hard` on 32-bit ARM; 64-bit ARM, POWER, s390x and RISC-V have them without a flag. The library then needs a processor with them (on x86, Intel Haswell and AMD Piledriver, 2012–2013, and later), and the code using GAOL is given the flag too. The elementary functions are much faster with them: `cos` 2.5 times, `log` 1.6 times, `sin` 1.3 times (Intel i7-1185G7, Clang 21.1), and with GCC 16.2 for Windows `cos` 4.7 times, `log` 3.2 times and `sin` 2.0 times (GitHub runner, MSYS2 UCRT64). GCC for Windows does not align its stack on 32 bytes for the 256-bit registers that `-mfma` alone would have it use (GCC bug 54412): `-mprefer-vector-width=128` keeps it to the 128-bit ones, and the objects of GAOL built so by GCC 16.2 have no instruction on a 256-bit register. In a cross-compilation, where no program can be run, they are left out;

- the AVX-512 path: the four arithmetic operations, the square, the square root and, with FMA, the integer powers beyond 1 and −1 take the AVX-512 instructions, which carry their rounding direction and set none, where `GAOL_PRESERVE_ROUNDING` is on, the compiler takes them and the machine building runs them. With that option every operation gains

(2.8 times at the median of seven operations, from 1.3 to 4.4 times, on an Intel i7-1185G7, whose AVX-512 units are halved); without it they cost (17 % at the median), the direction being written only when it is not upward already, so the path is left out. The library then needs a processor with the instructions, as with the fused multiply-add instructions (see Section 3.2, page 17);

- the 128-bit integer of the accurate phases of CORE-MATH: the type of the compiler where it has one, else two 64-bit halves (Visual C++, and GCC for 32-bit targets);

- the debugging facilities (see Section 18, page 117), for the `Debug` build type: GAOL and CORE-MATH without optimization nor `NDEBUG`, with debugging information and `GAOL_DEBUGGING`.

A developer, or a user who builds a package for any processor, can give these checks the answer they would not give: `-DGAOL_FMA=OFF` (no fused multiply-add instructions), `-DGAOL_PREFER_AVX512=ON` or `OFF` (the AVX-512 path asked for or refused), `-DGAOL_U128_EMULATION=ON` (the two halves of the 128-bit integer), `-DGAOL_DEBUG=ON` (debugging whatever the build type), `-DGAOL_EXCEPTIONS_ENABLED=OFF` (errors end the program), `-DGAOL_VERBOSE_MODE=ON` (a line on the standard error when GAOL initializes and cleans up) and, in a cross-compilation, `-DGAOL_MACHINE_RUNS_FMA=1` and `-DGAOL_MACHINE_RUNS_AVX512=1`. They are not options of the user: `configure -help` does not list them (`--disable-fma`, `--enable-prefer-avx512`, `--enable-verbose-mode`, `--disable-exceptions`), and meson lists them with the mark ‘developers’ (`enable-fma`, `enable-prefer-avx512`, `enable-debug`, `enable-exception`, `enable-verbose-mode`, `machine-runs-fma` and `machine-runs-avx512`).
The CMake build reports errors by exceptions (see Section 17.1, page 114).

Interprocedural optimization is not offered, and `gaol::gaol` carries no such flag: with GCC, it moves floating-point operations across the changes of the rounding direction, and GAOL then gives bounds that do not enclose the exact results.

2.4.2 The archive of the sources and the packages

CPack makes them from the CMake build, in the directory of the build:

```
% cmake -S . -B build -DCMAKE_INSTALL_PREFIX=/usr -DGAOL_FMA=OFF -DGAOL_PRE
% cmake --build build --config Release
% cmake --build build --target package_source
% cmake --build build --target package
```

- `package_source` gives `gaol-`*`version`*`.tar.gz`, the sources, which the three builds build. CPack takes the source tree as it is, less what git ignores and the builds made within it: the archive is made from a clean checkout, and holds `configure` as it was generated. CMake warns, when it configures, if that was for another version than `VERSION.txt` holds: `configure -version` would give the old one in the archive. After a change of `VERSION.txt`, `package_source` configures the build directory again first, whatever the generator;

- `package` gives `gaol-`*`version-system`*`.tar.gz`, what `cmake -install` installs, to extract anywhere, and on Linux `libgaol-dev_`*`version_arch`*`.deb`, the same files in `/usr`.

The packages hold `libgaol` as the build compiled it: where the machine building has the fused multiply-add instructions, only the processors that have them run it, and `-DGAOL_FMA=OFF` makes packages for any processor of the architecture. So does `-DGAOL_PREFER_AVX512=OFF` with `GAOL_PRESERVE_ROUNDING`, on a machine that has the AVX-512 instructions. [GAOL v5]

2.5 Building and installing with autotools

In the root directory of the sources, or in another directory for a build outside the sources:

```
% ./configure --prefix=prefix --with-tests
% make
% make test
% make install
```

`configure` compiles the sources of CORE-MATH into `libgaol`, with the flags it gives to the C code of GAOL. It is committed, generated by autoconf 2.72, automake 1.18.1 and libtool 2.5.4 from `configure.ac` and the `Makefile.am`: `make` never regenerates it (`doc/building.md` says how to). There is no `make dist`: the archive of the sources is made with CMake (see Section 2.4.2, page 8). `configure` accepts the following options:

- `--help`. Displays a list of all options. Note that only those described hereunder are supported;
- `--prefix=`*`prefix-dir`*. The root directory where the library will be installed. It defaults to `/usr/local`;
- `--libdir=`*`lib-dir`*. The directory where to put the libraries. It defaults to *`prefix-dir`*`/lib`;
- `--includedir=`*`include-dir`*. The directory where to put the header files. It defaults to *`prefix-dir`*`/include`;
- `--enable-debug[=`*`yes/no`*`]`. Builds GAOL as the `Debug` build of CMake: with `-g`, without optimization nor `NDEBUG`, and with the debugging facilities (see Section 18, page 117); the `Release` build of CMake otherwise, with `-O3` and `NDEBUG`, and the flags `-funroll-loops`, `-fomit-frame-pointer` and `-fexpensive-optimizations` where the compiler takes them. This option defaults to `no`;
- `--enable-simd[=`*`yes/no`*`]`. The vector intervals, SSE2 on x86 processors and NEON on 64-bit ARM processors, as `GAOL_SIMD` (see Section 2.4.1, page 6). This option defaults to `yes`; [GAOL v5]
- `--enable-preserve-rounding[=`*`yes/no`*`]`. Each operation restores the rounding direction it found (see Section 3.2, page 17). This option defaults to `no`;
- `--enable-coverage[=`*`yes/no`*`]`. Compiles GAOL and its tests with the counters of gcov, and adds the target `make coverage`, as `GAOL_COVERAGE`. This option defaults to `no`; [GAOL v5]
- `--enable-shared[=`*`yes/no`*`]`, `--enable-static[=`*`yes/no`*`]`. The static library `libgaol.a`, which is all that is built by default, as with CMake; `--enable-shared` builds the shared library `libgaol.so.5.0.0` too (it was built by default before GAOL v5); `--enable-pic` (the default) compiles the code as position-independent code; [GAOL v5]

- `--with-tests[=yes/no]`. Builds the unit tests of `tests`, which `make test` and `make check` run, as `WITH_TESTS`. This option defaults to `no`;
- `--with-examples[=yes/no]`. Builds the examples of `examples`, which `make check` runs, as `WITH_EXAMPLES`. This option defaults to `no`.

Note that, as usual, `--disable-xxx` is equivalent to `--enable-xxx=no`. Moreover, `--enable-xxx` is equivalent to `--enable-xxx=yes`. These options are those of the three builds (see Section 2.4.1, page 6): the fused multiply-add instructions, the AVX-512 path, the 128-bit integer and the NEON intervals are decided by checks of the compiler and of the machine, and `--enable-fma`, `--enable-prefer-avx512`, `--enable-verbose-mode` and `--enable-exceptions`, which `configure -help` does not list, are settings of the developers. A universal build of macOS is configured with `-arch` several times in the flags: `./configure CFLAGS="-arch arm64 -arch x86_64" CXXFLAGS="-arch arm64 -arch x86_64" LDFLAGS="-arch arm64 -arch x86_64"`. [GAOL v5]

The targets of the `Makefile` in the root directory are `all`, which builds `libgaol.a` (and the shared library `libgaol.so.5.0.0`, soname `libgaol.so.5`, with `--enable-shared`), `test`, `check`, `perf` and `parser`, those of the three builds (see Section 2.7, page 11), `install`, `clean` and `distclean`. `clean` erases what `make` built, and keeps what `configure` made, so that `make` builds again without it; `distclean` erases that too, which gives the source tree back as it was before `configure`. `make -C manual pdf` builds this manual and the manual of GAOL 4. `configure -help` ends with the address for the bug reports, `jordan.ninin@ensta.fr`, and the page of GAOL v5, `https://github.com/Jordan08/GAOL`.

2.6 Building and installing with meson

```
% meson setup build --prefix=prefix -Dwith-tests=true
% meson compile -C build
% meson test -C build
% meson install -C build
```

meson compiles the sources of CORE-MATH into `libgaol`, as CMake and `configure` do, and the lexer and the parser committed with the sources: neither flex nor bison is needed. `meson compile` needs meson 0.54; with an older one, `ninja -C build` builds GAOL as well. It builds `libgaol.a`, and `libgaol.so.5.0.0` with `-Ddefault_library=shared` (`both` for the two), as `configure` with `--enable-shared`; with Visual C++, the static library alone, as CMake. The options are given to `meson setup`, as `-Doption=value`; the build type gives the optimizations and the debugging facilities, as with CMake (`-buildtype=debug` for the `Debug` build), and the others are those of `configure`:

Option	Default
`buildtype`	`release`: `-O3`, `NDEBUG` and the optimizations of the `Release` build of CMake; `debug` for the `Debug` build
`enable-simd`	`true`
`enable-preserve-rounding`	`false`
`enable-coverage`	`false`; the target `ninja gaol-coverage`
`default_library`	`static`; `shared` builds the shared library
`b_staticpic`	`true`
`with-tests`	`false`; builds the unit tests
`with-examples`	`false`; builds the examples
`with-doc`	`false`; the target `pdf` then builds this manual and the manual of GAOL 4

`meson setup` runs Python to read the version of GAOL in `VERSION.txt`: the first of `python3` and `python` found in the `PATH`, or, if there is none, the Python that runs meson. On Windows, meson 0.53.0 and earlier may take an alias of Python of the Microsoft Store (`python3.exe`, and perhaps `python.exe`, in `WindowsApps`; not checked on Windows), which only opens the Store, and `meson setup` then stops; use meson 0.53.1 or later, or turn off the aliases in the Windows settings (*Manage app execution aliases*). Later versions of meson leave `WindowsApps` out only where the `PATH` names it under the directory of `USERPROFILE`, not for a profile whose directory differs from it.

2.7 Tests

Built with `WITH_TESTS` (CMake), `--with-tests` (`configure`) or `with-tests` (meson), the programs of `tests` compare the bounds GAOL computes with the exact results of the operations, independently of GAOL and of the floating-point environment: the exact results are computed with integers, or were computed with 2000 bits of precision. They check, among others, that the basic operations are the tightest enclosures, that the elementary functions are the tightest or within one double of them, that every operation gives the same result whatever the rounding direction the calling code left, that numbers are read exactly and intervals written rounded outward, that the reverse functions are accurate in the sense of IEEE 1788-2015, and that each name of `gaol_ieee1788` is the operation of the standard it names. The unit tests of GAOL 4, which checked the results of the operations on chosen intervals with CppUnit, are among them, and need CppUnit no more. The page `doc/tests.md` says what each program checks.

The three builds have the same targets:

- `make test` (`meson test`, `ninja test` with meson) runs the unit tests;
- `make check` (`ninja check`) runs the unit tests and the examples of `examples`, built with `WITH_EXAMPLES`, `--with-examples` or `with-examples`, which check what they print;
- `make perf` (`ninja perf`) measures GAOL v5 on the benchmark of `doc/compare/performance.md` and writes its times into the tables, the other libraries keeping the times of the last whole run (Linux, with bash, and Python 3 with numpy);
- `make parser` (`ninja parser`) generates the lexer and the parser of the strings again, with flex and bison.

`tests/find_package` builds the same tests with an installed GAOL, and `tests/fetch_content` with a GAOL brought into a project by `FetchContent`. `gaol_performance` (`tests/performance.cpp`) measures the time per operation of GAOL's arithmetic and elementary functions.

2.8 What is installed

Each build installs:

- the headers, in *`prefix`*`/include/gaol`;
- the library, `libgaol`, which holds CORE-MATH: static with CMake, or shared with `BUILD_SHARED_LIBS`, `--enable-shared` of `configure` and `-Ddefault_library=shared` of meson (static only with Visual C++). The shared library is `libgaol.so.5.0.0`, whose soname is `libgaol.so.5`, with the three builds;
- `gaol.pc`, for pkg-config, in the `pkgconfig` directory of the library directory (except the CMake build with Visual C++);
- with CMake, the CMake package of GAOL, which defines the target `gaol::gaol`.

Both `gaol.pc` and `gaol::gaol` carry the flags that the code including the headers of GAOL has to be compiled with (see Section 3.1, page 14).

3 An overview of GAOL

In this chapter, we will assume that GAOL has already been properly installed, and that the libraries and header files are accessible to your compiler.

Let us consider the following program to enclose the range of the Goldstein-Price function

$$f(x,y) = \big(1+(x+y+1)^2(19-14x+3x^2-14y+6xy+3y^2)\big) \\ \big(30+(2x-3y)^2(18-32x+12x^2+48y-36xy+27y^2)\big)$$

for $x \in [-2, 2]$ and $y \in [-2, 2]$.

Example

```
1  #include <iostream>
2  #include <gaol/gaol.h>
3  using namespace gaol;
4
5  int main(void)
6  {
7    interval
8      x(-2,2),
9      y(-2,2), z;
10
11   z=(1+sqr(x+y+1)*(19-14*x+3*sqr(x)-14*y+6*x*y+3*sqr(y)))*
12     (30+sqr(2*x-3*y)*(18-32*x+12*sqr(x)+
13        48*y-36*x*y+27*sqr(y)));
14
15   std::cout << "z = " << z << std::endl;
16   gaol::cleanup(); // Always, after the last use of GAOL
17   return 0;
18 }
19 z = [-87881320, 147125080]
```

First, note that we have to include the `gaol/gaol.h` header file in order to use all the facilities provided by GAOL. The type `interval` and the functions of GAOL are in the namespace `gaol_core`, which a program names through the namespace `gaol`: the `using` directive of Line 3 imports it. The `gaol/gaol.h` header of GAOL 4 imported the namespace itself; that of GAOL v5 no longer does, and a program adds the directive, or names `gaol::interval` and `gaol::sqr` (see Section 3.4, page 27). [GAOL v5]

Nothing has to be called before using GAOL: `gaol::init()` runs by itself when the program starts, before `main()`, with every compiler (see Section 4, page 35), and each operation of GAOL sets the rounding direction it needs (see Section 3.2, page 17). [GAOL v5]



The program calls `gaol::cleanup()` right after its last use of GAOL, at the latest at the end of `main()`, whatever the compiler. Unless GAOL is built to preserve the rounding direction, its initialization sets the rounding direction upward for the whole program, not only for the operations of GAOL, and `gaol::cleanup()` sets back the one the program started with. GAOL also calls it itself, but only when the program ends: without the call, the code after the last use of GAOL would compute its doubles rounded upward. [GAOL v5]

The `sqr(x)` function stands for *square of* x and is equivalent to `pow(x,2)`: it is the very function that `pow(x,2)` calls, with the same bounds, the sign of a zero bound included. [GAOL v5]

Let `f.cpp` be the name of the file containing the program above. To compile it, with the flags pkg-config gives (see Section 3.1, page 14), we type the following command:

```
% c++ -std=c++11 -O2 $(pkg-config --cflags gaol) -o f f.cpp \
      $(pkg-config --libs gaol)
```

GAOL is the only library to link: CORE-MATH, which computes its elementary functions, is compiled into it, and the C++ compiler links the mathematical library of the system itself.

Executing `f`, we obtain the output shown under the program.

We then know that $f(x, y)$ lies in $[-87881320, 147125080]$ for every x and y in $[-2, 2]$: `z` *encloses* the range of f. It is not the range, which is $[3, 1015690.27\ldots]$: the minimum is $f(0, -1) = 3$, and the maximum is reached on the edge $y = 2$. `z` is 231 times wider. Each occurrence of x and of y in the formula ranges over $[-2, 2]$ on its own, as if the others did not exist: this is the *dependency problem*, which makes an enclosure wider than the range. Cutting x and y into smaller intervals, and taking the hull of the values of f over the pieces, narrows the enclosure, at the price of more evaluations (`examples/03_dependency_problem.cpp` of the sources does it).

3.1 Compiling a program using GAOL

The operations of GAOL on intervals are inline: the code that includes the headers of GAOL has to be compiled with the *flags of interval arithmetic*, not only GAOL itself. [GAOL v5] With GCC and Clang, each where the compiler takes it:

- `-frounding-math -fno-fast-math -ffp-contract=off`, so that the compiler rounds each operation in the direction in effect, as written, and neither evaluates it at compile time in the default rounding nor contracts a multiplication and an addition into a fused one;
- `-msse2 -mfpmath=sse` on 32-bit x86, so that doubles are computed in double precision rather than on the x87 unit;
- `-msse2 -msse3` on x86 processors with the SSE2 intervals (`GAOL_SIMD`);
- `-faligned-new=8` with GCC and Clang on 32-bit Windows (MinGW-w64, MSYS2) with the SSE2 intervals, so that the compiler asks for aligned memory for the intervals, which the C runtime would align on 8 bytes instead of 16; Visual C++ and clang-cl have no such flag, and

need none: their SSE2 intervals keep their two bounds as two doubles there; [GAOL v5]

- nothing more on 64-bit ARM processors with the NEON intervals (`GAOL_SIMD`, see Section 3.2.2, page 21): the first item is all they ask for; [GAOL v5]
- `-ffloat-store` where doubles are still computed on the x87 unit;
- `-mfma` on x86, and `-mfpu=neon-vfpv4 -mfloat-abi=hard` on 32-bit ARM, where GAOL is compiled with the fused multiply-add instructions of the processor (`GAOL_FMA`): the code using GAOL then runs only on a processor with them, as GAOL itself. With GCC for Windows on x86-64 and on 32-bit x86 the flags are `-mfma -mprefer-vector-width=128`, the second keeping the vectorizer of GCC to the 128-bit registers, which it would otherwise spill to a stack that Windows aligns on 16 bytes, not 32 (GCC bug 54412): the objects of GAOL, CORE-MATH and the tests built so by GCC 16.2 have no instruction on a 256-bit register. A program that writes 256-bit intrinsics itself is exposed to that defect, as with `-mavx`. [GAOL v5]

With Visual C++ and clang-cl, `/fp:strict`, `/arch:AVX2` for x64 with `GAOL_FMA`, and `-msse2` with clang-cl for 32-bit x86. Each build installs these flags with GAOL, in `gaol::gaol` and `gaol.pc`. `gaol/gaol_config.h` refuses code compiled by Visual C++, or by clang-cl from Clang 16, without `/fp:strict`. GCC and Clang do not tell the code whether `-frounding-math` and `-ffp-contract=off` were given: there, it only refuses what contradicts them, `-ffast-math`, `-ffinite-math-only` and doubles computed on the x87 unit (see Section 2.3.2, page 4).
With `-frounding-math` on x86, GCC 12.1 to 12.3 and 13.1 to 13.2 (Debian 12 has GCC 12.2) initialize wrongly a double member of a static array of structures given `std::numeric_limits<double>::denorm_min()`: they store the long double 2^{-1074} there, whose first eight bytes read -0 and whose last two overwrite the next member (`{ denorm_min(), DBL_MAX }` read $[-0, \mathrm{NaN}]$, the empty set, on 32-bit x86). A double or an array of doubles initialized so is right, and so are GCC 12.4, 13.3 and later. A program built with these releases computes such a value at run time, as `tests/rounding_direction.cpp` does. [GAOL v5]
The link of the program takes one option too, where the compiler accepts it (GCC 13 and later on x86, and from 11.4 and 12.4 in the series 11 and 12): `-mno-daz-ftz`, which keeps out of the link the file that sets the flush-to-zero and denormals-are-zero modes when a program is linked with `-Ofast` (see Section 3.2.3, page 21). `gaol::gaol` and `gaol.pc` carry it too, where the compiler that built GAOL accepts it: the three builds test the compiler for it with a link. [GAOL v5]

3.1.1 From CMake

Example

```
1 find_package(gaol REQUIRED)
2 target_link_libraries(my_target PRIVATE gaol::gaol)
```

`gaol::gaol` carries the include directory, the flags above, the link option above for the code built by GCC 13 or a later one, or by the compiler that built GAOL, of its major version and not older (another compiler, as Clang 18, would stop on it, and so would GCC 12.0 to 12.3 with a GAOL built by

GCC 11.4: GCC has the option from 11.4 and 12.4 in the series 11 and 12 only), and no longer, for Visual C++, the definition `__GAOL_PUBLIC__=` of GAOL 4: GAOL being a static library there, its declarations carry no `__declspec(dllimport)`. [GAOL v5] There is no other library to link. A library whose headers include those of GAOL links `gaol::gaol PUBLIC`, so that its own users get the flags, and its CMake package finds GAOL again (`find_dependency(gaol)`).

A project can also build GAOL for itself, with the options it wants:

Example

```
include(FetchContent)
FetchContent_Declare(gaol GIT_REPOSITORY
    https://github.com/Jordan08/GAOL.git GIT_TAG master)
FetchContent_MakeAvailable(gaol)
target_link_libraries(my_target PUBLIC gaol::gaol)
```

GAOL is then a target of the project, built with it, and `cmake -install` of the project installs it with it; nothing is downloaded beyond the sources of GAOL. GAOL is compiled as position-independent code, so that `gaol::gaol` can be linked into a shared library.

3.1.2 From pkg-config

Each build installs `gaol.pc` in the `pkgconfig` directory of its library directory, except the CMake build with Visual C++. Its `Cflags` carries the flags above with the include directory, and its `Libs` GAOL itself and the link option above:

```
% export PKG_CONFIG_PATH=prefix/lib/pkgconfig
% c++ -std=c++11 -O2 $(pkg-config --cflags gaol) program.cpp \
      $(pkg-config --libs gaol)
```

Built by CMake with its default options on x86-64 Linux, GAOL gives, on one line:

```
% pkg-config --cflags --libs gaol
-frounding-math -fno-fast-math -ffp-contract=off
-msse2 -msse3 -mfma -Iprefix/include -Lprefix/lib -lgaol
```

(and `-mno-daz-ftz` after `-lgaol` where the compiler that built GAOL takes it: `gaol.pc` is written for that compiler, and a program linked by one that refuses the option, as Clang 18, a GCC before 11.4 or GCC 12.0 to 12.3, is to be linked without it). There is no `-lm`: GAOL still calls functions of the math library of the system (those of `fenv.h`, `sqrt()`, `floor()`, `fma()` where the processor has no such instruction…), which the C++ compiler links itself. [GAOL v5]
In a meson project, `dependency('gaol')` finds it.

A meson project can also build GAOL for itself, as a subproject, the sources of GAOL being in its `subprojects/gaol`:

Example

```
gaol_dep = dependency('gaol', fallback: ['gaol', 'gaol_dep'])
executable('program', 'program.cpp', dependencies: gaol_dep)
```

`dependency()` takes an installed GAOL where it finds one, and builds the subproject otherwise. `gaol_dep` carries the include directory, the flags above and the link option above, as `gaol.pc` does. GAOL is built in `release`,

as CMake and `configure` build it brought into another project, where the project leaves the build type to its default, `debug`: meson 1.8.4 and later give a subproject its own default build type, and GAOL compiles its libraries as in `release` with earlier ones, which give it the build type of the project; it cannot tell there the default from a `--buildtype=debug` asked for (but with Visual C++ before meson 0.57, and with the Visual Studio backend of meson, which keep the build type of the project). `-Dgaol:enable-debug=true` builds it for debugging. [GAOL v5]

3.2 The rounding direction

Floating-point interval arithmetic requires *outward rounding* in order to fulfill the *containment property*: for example, to add intervals $[a, b]$ and $[c, d]$, we compute $[\downarrow a + c \downarrow, \uparrow b + d \uparrow]$, where $\downarrow r \downarrow$ and $\uparrow r \uparrow$ return the greatest (resp. smallest) floating-point number smaller (resp. greater) than the real result of r. These two operations are performed by switching the rounding direction of the FPU towards, respectively, $-\infty$ and $+\infty$.

On most platforms, switching the rounding direction is costly. However, it is possible to cut down the number of switches by relying on the property that $\downarrow -r \downarrow = -\uparrow r \uparrow$. Consequently, one can replace nearly all downward rounding operations by upward rounding ones by negating appropriately twice the operations performed. GAOL stores the lower bound of an interval negated, and computes both bounds rounding upward: the rounding direction can then be set towards $+\infty$ once and for all.

GAOL 4 put on the user the burden to ensure that the rounding direction be always set towards $+\infty$ before any computation involving intervals, which it called the *trust rounding mode*. GAOL v5 no longer does: [GAOL v5] *each operation that computes a bound sets the rounding direction upward when it is not*, whichever way GAOL is built but for the operations of the AVX-512 path (below), and leaves it upward unless GAOL is built to preserve it. The bounds are then right whatever rounding direction the calling code left. The check is an addition, $1 + 2^{-60}$, which is above 1 only when rounded upward, rather than a reading of the rounding direction, which cost far more on some platforms: under Rosetta 2, reading it made an addition of intervals take 800 ns rather than 2.8 ns. On x86-64 processors, and on ARM processors with GCC and Clang, it is $1 + (2^{-1060} + 0)$, which shows the modes that flush the subnormal numbers to zero as well (see Section 3.2.3, page 21); with the NEON intervals of 64-bit ARM processors (see Section 3.2.2, page 21), it is a read of the control register FPCR, which holds both; and on 32-bit x86 processors, whose two units that round, the x87 unit and the SSE instructions, each have a direction, a read of the control word of the first and of MXCSR (`fnstcw` and `stmxcsr`). [GAOL v5] Each function of intervals checks the direction once, and the functions it calls on doubles take it to be upward already.

Code that needs its own rounding direction after the operations of GAOL builds GAOL with the option `GAOL_PRESERVE_ROUNDING` of CMake, which is `--enable-preserve-rounding` with autotools and `-Denable-preserve-rounding=true` with meson: each operation then also restores the rounding direction it found, which makes the arithmetic operations slower, 1.3 to 1.7 times with the AVX-512 path below and several times without it (see Section 3.2.1, page 18); it also restores the modes flushing the subnormals to zero that the operation cleared (see Section 3.2.3, page 21). It is off by default in the three builds. With it, the build takes the AVX-512 path (it was the default of an option, `GAOL_PREFER_AVX512`): where the machine building has the AVX-512 instructions, the four arithmetic

Definition 1 (Round down/up) *Given $\mathbb{R}$ the set of real numbers and $\mathbb{F}$ the set of floating-point numbers (`double`), we have, for all $x \in \mathbb{R}$:*

$$\downarrow x \downarrow = \max\{y \in \mathbb{F} \mid y \leqslant x\}$$
$$\uparrow x \uparrow = \min\{y \in \mathbb{F} \mid y \geqslant x\}$$

operations, the square, the square root and, with FMA, the integer powers beyond 1 and −1 take them, which carry their rounding direction, set none and store none, and the library then needs a processor with them. [GAOL v5]

Without it, the rounding direction stays upward from the initialization of GAOL on, for the whole program, and `cleanup()` sets back the one that the initialization found: every program calls `gaol::cleanup()` right after its last use of GAOL (see Section 4, page 35). [GAOL v5]
Some operations set no direction, and leave the one they found, with or without `GAOL_PRESERVE_ROUNDING`: a program that wants a direction after one of them sets it itself. [GAOL v5]

- Those that round nothing and make no check: the negation and the unary plus, `abs()`, `min()`, `max()`, `sign()`, the roundings to an integer (`floor()`, `ceil()`, `trunc()`, `integer()`, `round_ties_to_even()`...), `&` and `|`, `hull()` and `intersect()`, `invabs_rel()`, `mig()` and `mag()`, `nth_root(x, 0)` and `nth_root(x, 1)`, the constructors from bounds, the access to the bounds, `is_empty()`, the relations and the other predicates, `exact_string()`.

- Those that return before their check. Some return the empty set at once for an empty operand, and some do not: `y * empty` and `exp(empty)` leave the direction, where `empty + y` and `sqrt(empty)` set it upward, so that a program counts on neither. And `hausdorff(x, y)` returns $+\infty$ when a bound is infinite in one of the two intervals only, where it sets the direction upward otherwise.

- The operations of the AVX-512 path, which set none.

3.2.1 What the upward rounding does to the program

The rounding direction is the processor's, and the whole program shares it. The results of GAOL do not depend on it, each operation setting it upward when it is not, in every thread, but the doubles the program computes do. [GAOL v5] Unless GAOL is built with `GAOL_PRESERVE_ROUNDING`, they are rounded upward from the initialization of GAOL, before `main()`, until `cleanup()`, and again after each operation of GAOL that follows it and sets the direction: in a build that takes the AVX-512 path, which without `GAOL_PRESERVE_ROUNDING` only the developers ask for, the arithmetic operations set none, and the direction after one of them is the program's. The compiler does not undo it: with the flags of `gaol::gaol`, `-frounding-math` in particular, an inexact operation on constants written in a function, `1.0 / 3.0`, is not evaluated when compiling, to nearest, but when the program runs, in the direction in effect. Only what the compiler must evaluate when compiling, a `constexpr` or the constant initializer of a static variable, is still rounded to nearest.

What the program computes, with GCC 9.4 or Clang 18 and the GNU C library 2.31 on x86-64 (`examples/13_rounding_environment.cpp` of the sources computes each row and checks it):

The program computes	While GAOL is initialized	After `cleanup()`
`1.0 / 3.0`	0.33333333333333337	0.33333333333333331
`std::stod("0.3") == 0.3`	false	true
`std::lrint(2.3)`	3	2
`std::printf("%.2f", 2.675)`	2.68	2.67
`std::cout << std::setprecision(3) << 3.14159265358979`	3.15	3.14
0.1 added 10^7 times	1000000.0005483569	999999.99983897537
20 000 doubles written with `%.17g`, read back by `std::strtod`	20 000 come back different	none
`TwoSum(1e20, 1e-20)`, the error of a sum	$s + e \neq a + b$	$s + e = a + b$
`TwoSum(a, b)`, 50 000 pairs of doubles of exponents from -60 to 60	$s + e \neq a + b$ for 10 717 pairs	none
`TwoProd(a, b)`, Dekker's splitting, 50 000 pairs of doubles of $[1, 2)$	$p + e \neq ab$ for 1 888 pairs	none
`TwoProd(a, b)`, `e = std::fma(a, b, -a*b)`, the same pairs	none	none

The quotient is the double above 1/3, where the program expects the nearest. The C library rounds in the direction in effect too, as the C standard recommends: with the GNU C library, `std::rint(-2.7)` is -2 where it is -3 to nearest, and `strtod()`, `stod()`, `printf()` and the streams round the decimal digits they read and write up. A double written with 17 digits, which reads back to itself to nearest, then comes back one double above, and one more at each round trip. Another C library may convert to nearest whatever the direction: the example reports what the one it runs with does. Sums are biased: each addition rounds up, so 0.1 added ten million times ends $5.5 \cdot 10^{-4}$ above the exact $10^6 \times 0.1$, where to nearest it ends $1.6 \cdot 10^{-4}$ below. And TwoSum, the algorithm that recovers the error of a sum,

Example

```
1 s = a + b;  bb = s - a;  e = (a - (s - bb)) + (b - bb);
```

gives $s + e = a + b$ only to nearest. Rounded upward, the exact error of $10^{20} + 10^{-20}$, $10^{-20} - 16384$, is no double: `s` is $10^{20} + 16384$ and `e` is -16383.999999999998, whose sum is not $10^{20} + 10^{-20}$. TwoProd, which recovers the error of a product from Dekker's splitting of the operands, breaks the same way (1 888 of the 50 000 pairs of the table, drawn in $[1, 2)$, lose their exactness upward), and is exact in every direction when it is computed with `std::fma` instead: the error of a rounded product is a double, which `std::fma` computes however it rounds (provided it is correctly rounded: with GCC, `std::fma` without optimization is a call of the C library, whose `fma()` is wrong on Cygwin; `__builtin_fma` is the instruction with `-mfma`). Compensated sums, and the double-double arithmetic built on such algorithms, lose their accuracy the same way. Built with `GAOL_PRESERVE_ROUNDING`, the program computes the right-hand column all the time.

Other consequences:

- a thread created after the initialization of GAOL starts upward too (`std::thread` gives a new thread the floating-point environment of the thread constructing it), and `cleanup()` sets back the direction of the calling thread only;

- the libraries the program calls run upward as well: the `exp`, `log` and `pow` of the GNU C library 2.31 go from 0.51 ulp to 1.01 ulp from the exact value, its `cbrt` from 2.8 to 3.8 ulp (20 000 random arguments each, against mpmath), and its `sin`, `cos`, `tan` and `atan` give the same results;
- GCC does not model the rounding direction (GCC bug 34678), even with `-frounding-math`, and reuses after `cleanup()` a double computed before it, still rounded upward, where the program computes it again: from `-O1`, GCC 9.4 does it for `a / b` written on both sides of the call, where Clang 18 computes the quotient again. A program cannot rely on either.

What to do:

- *Call `cleanup()` right after the last use of GAOL*, before the program prints, converts or sums what it computed: its doubles are then computed to nearest. A program that goes back to GAOL afterwards calls `restore_rounding()` (or `std::fesetround(FE_TONEAREST)`) after each use, since an operation of GAOL sets the direction upward again and `cleanup()` sets it back once only (see Section 4, page 35).
- *Read and write numbers through GAOL*, not through the C library: `gaol::textToInterval("0.3")` encloses 3/10 in every direction, where `std::stod("0.3")` does not, and `exact_string()` and `%a` write a number that reads back to the same bits in every direction. GAOL writes its intervals rounded outward whatever the direction.
- *Compute in a block the doubles that need rounding to nearest* (an error-free transformation, `std::lrint()`, a decimal conversion, a call of a library written for that): save the direction with `std::fegetround()`, set `FE_TONEAREST`, and set the saved one back when the block ends, with no operation of GAOL inside, which would set the direction upward again. The example of the sources writes such a guard, `nearest_scope`.
- *Do not count on the compiler* to compute a double where the source code does. One that has to be computed on this side of a change of direction is computed from operands read again through `gaol_core::rnd_keep()`, which writes its argument to volatile memory and returns it: `gaol_core::rnd_keep(a) / gaol_core::rnd_keep(b)` after `cleanup()` is the quotient to nearest with GCC too, where `gaol_core::rnd_keep(a / b)` may be the one computed before.
- *Or build GAOL with `GAOL_PRESERVE_ROUNDING`*, and the program keeps its own direction: each operation reads it, sets it upward, and sets it back when it is done (the operations of the AVX-512 path set none). Nothing else changes: the bounds are the same, and the program still calls `cleanup()`. It costs time, less with the AVX-512 path, which this option turns on by default where the machine has the instructions. On an Intel i7-1185G7 with Clang 21.1 (`tests/performance.cpp`, the best of 9 interleaved runs, three times), with that path `x + y` and `x - y` take 1.3 times longer, `x * y` and `x / y` 1.7 times, `sqr(x)` 1.2 times, `pow(x, 3)` 1.3 times, `exp(x)` 1.3 times, `log(x)` 1.1 times, `sin` and `cos` 1.1 to 1.2 times, and `sqrt(x)` 0.7 times (it is faster); without it (a build for a machine without the instructions, or one that refuses the path with `-DGAOL_PREFER_AVX512=OFF`, a setting of the developers) `x + y` and `x - y` take about 8 times longer, `x * y` 4 times, `x / y` 3.7 times, `sqr(x)` 4.6 times, `pow(x, 3)` and `sqrt(x)` 1.7 times, `exp(x)` 1.3 times, `log(x)`

1.1 times, and `sin` and `cos` 1.2 times. A line of 15 additions of intervals takes 9.6 times longer without the path, and 2.1 times with it.

3.2.2 The NEON intervals

On 64-bit ARM processors (AArch64), GAOL computes its intervals with NEON instructions, as it does with SSE2 instructions on x86: [GAOL v5] the two bounds of an interval are the two lanes of one `float64x2_t`, the lower bound negated as in the `__m128d` of the SSE2 intervals, and `+`, `-`, `*`, `/`, `sqr()` and `sqrt()` take and give their bounds in registers. The sum and the difference are one vector addition rounded upward; the product is the maximum of the four products of the bounds, with no branch on their signs; the quotient is one vector division when the divisor has no zero, and single divisions otherwise; the square roots of both bounds are one vector square root rounded upward, the lower bound being that root where it is exact, and the double below it otherwise.

They are the intervals of GAOL on 64-bit ARM with every compiler (GCC, Clang, AppleClang, clang-cl, Visual C++ and the Clang of MinGW-w64), by default (`GAOL_SIMD` of CMake, `--enable-simd` of `configure`, `enable-simd` of meson, as for the SSE2 intervals; the macro `GAOL_USING_NEON_INSTRUCTIONS` is then 1 in `gaol/gaol_configuration.h`), and need no flag beyond those of interval arithmetic (see Section 3.1, page 14). 32-bit ARM, whose NEON has no vector of doubles, does not take them: the FPU computes the intervals there, as before. A universal build of macOS takes them for its arm64 part, and the SSE2 intervals for its x86_64 part.

64-bit ARM has no instruction that carries its own rounding direction, as the AVX-512 path has: the direction is the field RMode of the control register FPCR (bits 23 and 22, upward being `0x400000`), which the scalar and the vector instructions share, and which holds the modes that flush the subnormal numbers to zero as well (FZ, bit 24, and from Armv8.7 FIZ, bit 0). The NEON intervals therefore check the direction by reading FPCR with `mrs`, which raises no exception flag, and write it with `msr` only if it is not upward with the flush modes clear, where the FPU intervals add $1 + (2^{-1060} + 0)$, which raises the inexact one (see Section 3.3, page 24). `msr` on FPCR is a serializing instruction, slower than `mrs`, which is why it is written when needed only. With `GAOL_PRESERVE_ROUNDING`, each operation reads FPCR once, and writes back the direction and the modes it found at its end only if they differ from what the operation left: a program that computes upward already, with the flush modes clear, pays no write. `round_upward()`, `round_nearest()` and `round_downward()` write FPCR as well, and not through `fesetround()`, and only if the direction changes (see Section 14, page 107). `fegetround()` and `fesetround()` of the C library read and write the same bits (the `FE_*` values of the C libraries of 64-bit ARM are these bits, which a static assertion checks), so that a program may use either.

The bounds are those of the FPU intervals but for the sign of a zero bound (see Section 3.6.6, page 31), which the maximum of 64-bit ARM, ranking $+0$ above -0, can set otherwise than the SSE2 and the FPU intervals do, where the `maxpd` of x86 returns its second operand. The tool `tests/tools/arith` compares the NEON and the FPU intervals on a grid of operands, with the calling code in each of the four rounding directions: it found no other difference.

3.2.3 Flush-to-zero and denormals-are-zero

Processors have modes that flush the subnormal numbers, those below 2^{-1022} in magnitude, to zero. On x86, the *flush-to-zero* (FTZ) and *denormals-are-zero*

(DAZ) modes of the SSE instructions, two bits of the control register MXCSR, make every subnormal result a zero (FTZ) and read every subnormal operand as a zero (DAZ), in the comparisons too; on ARM, the mode FZ of the control register (FPCR, FPSCR) does both. With one of them set, the bounds of every operation with a subnormal result or operand miss the exact result, whatever the rounding direction: `interval(1e-300) * interval(1e-20)` is $[0, 0]$, and so are `sqr(interval(1e-160))` and `interval(3e-308) - interval(2.9e-308)`. [GAOL v5]

A program linked with one of the options `-Ofast`, `-ffast-math` and `-funsafe-math-optimizations` gets them: GCC and Clang then link `crtfastmath.o`, on x86 and on ARM Linux, whose constructor sets them for the whole program. GCC links it last, and it runs after GAOL initialized itself; Clang links it first, and the initialization of a static GAOL, which follows, clears the modes again, unless GAOL is built with `GAOL_PRESERVE_ROUNDING`, or is a shared library, initialized before the program. The option `-fno-fast-math` of `gaol.pc` and `gaol::gaol` does not prevent it: it keeps `crtfastmath.o` out of the link when it comes after `-ffast-math`, but not after `-Ofast` (GCC 9.4 and 13 and Clang 18 link it all the same, and `-fno-fast-math` silences the `#error` of `gaol/gaol_config.h` against `-ffast-math`, which `-Ofast` would raise), nor, with GCC 9.4 and 13, after the option `-funsafe-math-optimizations`. Loading a plug-in or a Python module built with `-Ofast` sets the modes too, when it is loaded, where its compiler linked `crtfastmath.o` into it: Clang 18 does, GCC did before version 13. *Linking a program that uses GAOL with these options, or loading code built with them, makes the bounds wrong*, and GAOL defends itself in two ways:

- on x86 processors, and on ARM processors with GCC and Clang and with the NEON intervals of Visual C++, each operation that computes bounds clears the modes when it starts, in the check that sets the rounding direction upward: the check is one comparison with a subnormal number, which shows the modes as well as the direction, or, with the NEON intervals, a read of FPCR, which holds both; the modes stay cleared for the rest of the program, as the rounding direction stays upward. The check comes before the operation reads, compares or copies a bound, the test of the empty set aside, and the compiler is kept from making a comparison before it, which it would do, not knowing that the check changes how a subnormal compares: with GCC 13 at `-O3`, `asinpi()` compared the bounds it had read with 0 before its check, and with the modes set `asinpi()` of $[100 \cdot 2^{-1074}]$ was $[32 \cdot 2^{-1074}]$, above the exact value. With `GAOL_PRESERVE_ROUNDING`, each operation sets the modes back as it found them, once it has made its result, its maxima and minima of bounds included. The bounds of every operation that makes the check are then the ones it gives with the modes cleared, the tightest ones, and the modes are cleared, or set back, after it. It is the only defence against a plug-in, which sets the modes at any time, and against the compilers and processors without `-mno-daz-ftz`. On an Intel Xeon of the Cascade Lake generation, an addition with a subnormal operand and result took as long as one of normal numbers, and the check as long as the one of the direction alone; through the library, with Clang 18, `x * y` took 4.0 ns rather than 3.4 ns, `sqrt` 11.0 ns rather than 10.3 ns and `pow(x, 3)` 13.7 ns rather than 13.0 ns, as with the same check made of a normal number, and the other operations measured stayed within the noise. Some x86 processors take a microcode assist, of the order of a hundred cycles, for an operation with a subnormal operand or result, which each operation would pay there: on the

processors of the continuous integration (x86-64, i386, arm64 and armhf, under Linux, Windows and macOS), the check took 0.35 to 1.43 ns, as long as the same check made of a normal number, within the noise, and none took such an assist;

- `gaol.pc` and `gaol::gaol` give `-mno-daz-ftz` to the link, where the compiler accepts it (GCC 13 and later on x86, and from 11.4 and 12.4 in the series 11 and 12; GCC 9.4, 12.3, GCC for ARM and Clang 18 refuse it): it keeps `crtfastmath.o` out of the link, so that the program does not get the modes at all, its own code included.

The functions that only read, compare or copy the bounds of an interval make no check, which made each of them 3 to 4.5 ns slower in a micro-benchmark on an Intel i7-1185G7, two to three times the constructor or `certainly_le()`: they compare the bounds as bounds, whatever the modes. [GAOL v5] A mode that reads a subnormal operand as a zero (DAZ, FZ) can make two different doubles compare equal, both zero or subnormal, but never puts two doubles in the wrong order: the doubles are compared first, and only where they compare equal are they compared as integers, by their bits, which no mode changes. These functions are the constructor from two bounds, the relations (`set_contains()`, `certainly_le()`, `set_eq()`, `is_zero()`, `lexicographic_less...`), the intersection `&` and the hull `|`, `max()`, `min()`, `abs()`, `sign()`, `floor()`, `ceil()`, `integer()` and the other roundings to an integer, `invabs_rel()`, `mig()`, `mag()`, `is_canonical()`, `feven()`, `nb_fp_numbers()`, and `inf()` and `sup()` of `gaol_ieee1788`; the roundings to an integer round a subnormal bound from its bits, which the `floor()` and `ceil()` of musl would return as it is. They give the results they give with the modes cleared, and leave the modes as they found them, writing nothing to the control register, with `GAOL_PRESERVE_ROUNDING` or without. On the Intel i7-1185G7, with Clang 18, this left the constructor as fast as before, and made `x <= y`, `x.set_contains(y)`, `x &= y` and `x | y` 0.5 to 0.8 ns slower, and `floor()` 1.7 ns. `midpoint()`, `split()` and `chi()`, which compute, and the writing of an interval (`operator<<`, `intervalToText()`) make the check, which clears the modes, or sets them back after them with `GAOL_PRESERVE_ROUNDING`: the C library writes a subnormal 0 under denormals-are-zero with gdtoa (FreeBSD, macOS) and with the Debug runtime of Visual C++.

What remains: the reader, which makes no check of the modes, takes the numbers apart by their bits, and compares the two numbers of `<a, b>`, which have to be the same double, by theirs too, so that under denormals-are-zero `<1e-310, 1e-309>` is refused, as without it (see Section 10.1.1, page 82). The unary minus makes no check either, but it only exchanges the stored bounds, which the modes do not change, as they do not change the test of the empty set, made before the check of every operation (an operation with an empty operand may return before its check, the modes left as it found them), nor the test by which `hausdorff()` returns $+\infty$ before its check when a bound is infinite in one of its two intervals only. The modes of other processors, and of ARM with the FPU intervals of Visual C++, are neither checked nor cleared, though the functions above compare the bounds right there too, and `operator<<` writes a subnormal bound outward, as 2^{-1022} away from zero and 0 toward it; GCC links `crtfastmath.o` for none of the other processors GAOL is tested on. Link a program that uses GAOL without these options, or with `-mno-daz-ftz`, and compile the code that needs them apart from it.

3.3 The floating-point exceptions

GAOL computes with the floating-point exceptions masked, as a program starts, and leaves them so: an operation that raises one only sets its flag. *A program that enables them* (`feenableexcept()` of the GNU C library, `_controlfp_s()` of Visual C++, `fesetenv()` with an environment that traps) *disables them while GAOL computes*, and enables them again afterwards if it wants them: the operations of GAOL raise some, as listed below, and the processor then stops the program (with `SIGFPE` on Linux) instead of letting the operation give its bounds. [GAOL v5]

- `log(interval(0, 1))`, which is $[-\infty, 0]$, raises the divide-by-zero exception; `interval(1e308)*10`, which is [DBL_MAX, $+\infty$], raises the overflow exception.

- Reading a number above the largest double (`1e400`, `1.8e308`, `0x1p1024`) raises no overflow exception: the reader gave it to `strtod()`, which computes $+\infty$ and raises it, so that `1e400*0`, which is [0, 0], raised the flag that stands for an infinite bound, and a program that had enabled the exception died while it read. The interval read is [DBL_MAX, $+\infty$]; an operation on it raises the flag where its own result overflows, as for any other interval. Nor does reading a number below the least normal double (`1e-310`, `1e-400`) raise the underflow exception, which `strtod()` raises when it rounds such a number to a subnormal or to 0. [GAOL v5]

- Almost every operation raises the inexact exception: most bounds are rounded results, and where GAOL checks the rounding direction with an addition, $1 + 2^{-60}$ or $1 + (2^{-1060} + 0)$, whose result is inexact (see Section 3.2, page 17), even an operation whose bounds are exact raises it. On x86, the addition of the subnormal 2^{-1060} also raises the denormal-operand flag of the SSE instructions, which is none of the five flags of IEEE 754 (not in `FE_ALL_EXCEPT`), and its exception stops the program where the program enabled it; and with flush-to-zero set, the sum $2^{-1060} + 0$, flushed to zero, raises the underflow exception (`FE_UNDERFLOW`) too, before the check clears the mode. With the NEON intervals the check is a read of FPCR (see Section 3.2.2, page 21), which raises no flag: `+`, `-`, `/` and `sqrt()` raise the inexact one only when a bound they compute is rounded, `*`, `sqr()` and `pow(x, 2)`, which is `sqr()`, also when a corner product that is not a bound of the result is. [GAOL v5]

- An empty operand raises no invalid-operation exception, though the bounds of the empty interval are NaN: every operation, relation, function and output of `gaol::interval` and of `gaol_ieee1788` takes it without raising that exception. `is_empty()` and the relations compare the bounds with the quiet comparisons of `<cmath>` (`std::islessequal()`...), each one instruction, as `<=` is, with GCC and Clang on x86-64, and so does the first comparison of the constructor `interval(l, r)`, which gives the empty set for NaN bounds. With Visual C++, where these are calls to the C library, which made `x * y` and `sqrt()` 34 to 61 % slower on x64, GAOL compiles them itself into the same instruction, `ucomisd`, on x64 and on x86 with SSE2; on arm64 they stay calls. With the invalid-operation exception enabled, `is_empty()` of an empty interval killed the program, and so did about 50 operations with an empty operand, which compared its NaN bounds: `x & y` for an empty `y`, `sqrt`, `exp`, `min`, `max`, `floor`,

`set_contains()`, `set_disjoint()`, the output of the empty set…; and `interval::emptyset()` itself in a build without optimization. A NaN the program gives GAOL, the empty set for `interval(d)`, raises it no longer either: `interval(NAN)`, `interval(NAN, 1)`, `x = NAN`, `x += NAN` (and `-=`, `*=`, `/=`, `%=`), `x &= NAN`, `x <= NAN` and `pow(x, NAN)` compared it with `<` or `<=`. An empty operand may still raise the inexact exception: the sum of two empty intervals does where GAOL checks the rounding direction with an addition. The vector comparisons of 64-bit ARM (`fcmgt`, `fcmge`) signal the invalid-operation exception on a NaN: the NEON operations tell the empty set, the infinite bounds and the zeros from the bits of the bounds, and compare no bound as a double before they know it is no NaN. [GAOL v5]

- A compiler may compile a quiet comparison into a signaling one, or compute a comparison that follows a test of the empty set before it, and GAOL is written against the cases seen. GCC turns a choice made on a quiet comparison into a conditional move with a signaling comparison on 32-bit ARM (`vcmpe`, GCC 12 to 14; GCC bug 52258, https://gcc.gnu.org/bugzilla/show_bug.cgi?id=52258, is of this kind) and on POWER9 (`xscmpgedp`, GCC 13 with `-mcpu=power9`): there `is_empty()`, the intersection and the constructors of the FPU intervals tell NaN bounds with `std::isunordered()` first, which stays quiet, so that the choice a program makes on `is_empty()` (`x.is_empty() ? a : b` for an empty `x`), `x &= y` for an empty `y` and `floor()`, `max()`… of an empty interval raise nothing (the armhf jobs of the continuous integration and a POWER9 job under qemu check them, built for size too, where GCC calls the intersection rather than inlining it). GCC 9.4 at `-O3` computed the comparisons of the bounds of `x &= y` before its test of the empty set, in a loop of intersections: they are quiet ones. And GCC vectorizes loops of comparisons of bounds into vector comparisons that raise the exception for an empty element: GCC 9.4 and 13 where AVX is on (`-mavx`, or `-mfma`, which the three builds give where the processor has FMA) and GCC 13 to 15 for 32-bit x86 with SSE2 alone (MinGW-w64) make the quiet comparisons signaling predicates (`vcmpnlepd`, `cmpnlepd`…), GCC 14 for POWER8 uses `xvcmpgedp`, and GCC 11 to 15 for 64-bit ARM compute the second and third comparisons of the constructor for an empty element too. With GCC and the FPU or the NEON intervals, on every processor, the bounds `is_empty()` compares, those of the relations that do not call it (`straddles_zero()`, `set_contains(d)`…) and the doubles given to the constructors go through an empty asm statement, which keeps GCC from vectorizing such loops: loops of `x.set_contains(d)` and of `x.straddles_zero()` took 0.8 and 1.3 ns per element rather than 0.5 ns with GCC 13 on x86-64, about as long as with GCC 9.4, which did not vectorize them and took as long as before. A choice the program makes on a relation, `x.set_contains(y) ? a : b`, and a loop of relations other than those of `tests/rounding_direction.cpp` are not known to raise the exception, but nothing keeps a compiler from compiling them so. [GAOL v5]

- Nonempty operands raised the invalid-operation exception as well, and the cases found no longer do. With the SSE2 intervals, the products `interval(0)*interval::one_plus_infinity()` and `interval::positive()*interval(0)`, which multiply a zero bound by an infinite one, did (the FPU intervals gave the same products

without it), and so did, through them, `pow` of the base `interval(1)` and the exponent `interval::one_plus_infinity()`; `pow` did in every build, through the `pow` of CORE-MATH, for an exponent of extreme magnitude (`pow(interval(1, 2), interval(4.9e-324))`, `pow(interval(1, 2), interval(1e-300))`, an exponent interval with a bound near `DBL_MAX` such as $[10^{300}, 10^{308}]$), which the CORE-MATH of GAOL v5 no longer does, and for a power next to the largest double (`pow(interval(2), interval(1023.9999))`, `pow(interval(0x1.000003f82a1fp+0), 3000000001LL)`), where CORE-MATH multiplied its first approximation by 2^{1024}, an infinity, and computed $\infty - \infty$ (changed in the vendored sources, see `3rd/README.md`); and with the SSE2 intervals, `div_rel(K, J, I)` did where it divided an infinite bound of `K` by an infinite bound of `J` in a half of the register it did not keep ($K = [2, +\infty]$ and $J = [-\infty, 2]$), and raised the division-by-zero exception where it divided a bound by a zero one there ($K = [-2, -2]$ and $J = [-2, 0]$), and so did the division, the relational division and the inverse of a divisor with a zero bound (`interval(1, 2)/interval(0, 2)`, `inverse(interval(0, 2))`, `pow(interval(0, 2), -1)`). This list is not exhaustive. [GAOL v5]

- The overflow and the division-by-zero exceptions are no longer raised for a result that has no infinite bound. Some functions computed beyond the largest double on the way, or divided by zero, and raised them for a finite result, the empty set included: `sqrt(interval(1, DBL_MAX))`, `nth_root(interval(DBL_MAX), 3)` and the roots of order 4 and more, and the relational `sqrt_rel()` and `nth_root_rel()` (the product that proves a root exact is 2^{1024} for the square root of `DBL_MAX`), `sin(interval(-1, DBL_MAX))` and `cos()` (the width of the interval), `tan(interval(DBL_MAX))` ($x + \pi/2$), `div_rel()` (the quotient by a divisor next to 2^{-1074}), `atan_rel()`, `atan2_rel_x()` and `atan2_rel_y()`, `pow_rel()` and `pow_exponent_rel()` (the powers beyond the largest double that their searches try, and the exponents next to 2^{-1074}), and `atanh_rel()` (atanh at -1 and 1). A quotient, a product or a power that would overflow is the infinity, or the largest double, that the operation gives, without being computed, where it is only an intermediate value: the exceptions still tell an infinite bound of the result, as above. This list is not exhaustive either. [GAOL v5]

An empty operand, and the nonempty operands tested, raising no invalid-operation exception therefore does not make that exception safe to leave enabled: every exception stays disabled while GAOL computes. [GAOL v5]

The flags tell nothing of the results: after most operations of GAOL, `fetestexcept(FE_INEXACT)` is raised whatever the result (not after `-x`, `abs()`, `&`, `|`, `floor()`, `max()` or `min()`, which round nothing and check no rounding direction, nor after `+`, `-`, `/` and `sqrt()` with the NEON intervals when their bounds are exact), and `FE_OVERFLOW` or `FE_DIVBYZERO` stands for an infinite bound of the interval, or an infinite result of `width()` or `rad()`, not for an infinite number the program computed: `midpoint()` and `mid()` of [DBL_MAX, DBL_MAX], which overflowed the sum of the bounds, raise none. IEEE 1788-2015 leaves the flags of its operations unspecified. A program that reads them for its own computations clears them (`std::feclearexcept(FE_ALL_EXCEPT)`) just before, and reads them before its next operation of GAOL. [GAOL v5]

GAOL's initialization, which runs before `main()`, sets the default environment (`fesetenv(FE_DFL_ENV)`), which masks every exception, unless GAOL is

built with `GAOL_PRESERVE_ROUNDING`: the initialization then leaves the whole environment as it found it, and an exception a static object enabled before it stays enabled. A program enables the exceptions in `main()` or later, whichever the build, not in a static object initialized before GAOL. `cleanup()` leaves the exceptions and the flags as they are, and so does every operation, whichever way GAOL is built: it leaves the exceptions the program enabled enabled, and the flags it raised raised, and adds the flags it raises itself. With `GAOL_PRESERVE_ROUNDING` and the SSE2 intervals, `+`, `-`, `*`, `/`, `%`, `div_rel()`, `sqr()`, `inverse()`, the integer powers and `+= d`, `-= d`, `*= d`, `/= d`, `%= d` wrote the SSE control register whole, every exception masked and every flag cleared: an exception the program had enabled was masked again by the first of these operations, and the flags it had raised were lost. [GAOL v5]

3.4 The namespaces

The type `interval` of GAOL and its functions are in the namespace `gaol_core`. A program names them through one of two namespaces, which `gaol/gaol.h` declares and does not open: [GAOL v5]

- `gaol`, GAOL under its own names, which this manual uses, to stay compatible with GAOL 4: a program written for GAOL 4 keeps its names there and, but for the results that change (Chapter 22), its results;
- `gaol_ieee1788`, GAOL under the names of IEEE 1788-2015 (Chapter 11), to follow the standard.

The main difference between the two is the power, `pow` and `pown` (see Section 3.5, page 28). The two namespaces hold `interval`, and the functions that have the same name and the same meaning in both (`sin`, `exp`, `sqrt`, `min`...) are the same functions, those of `gaol_core`. `pow` and `textToInterval` are the ones whose meaning differs: each namespace has its own, and none is in `gaol_core`, the namespace where argument-dependent lookup looks for a call `pow(x, y)` on an interval, so that each namespace finds its own only. A program opens one of the two namespaces, not both: with both open, `pow(x, y)` is ambiguous, and so is `textToInterval(s)`, which reads the names of GAOL in `gaol` and those of the standard in `gaol_ieee1788` (see Section 5.1, page 39).

`gaol` names the type and the functions of `gaol_core` one by one: the names GAOL 4 had in `gaol` that GAOL v5 still names there (the class `interval`, the expressions with their nodes and visitor, the functions and the operators of intervals, the functions of doubles and the rounding direction, `round_downward()`, `round_upward()`, `round_nearest()`, `previous_float()`, `next_float()`, `is_finite()`, `version`...), the functions on intervals GAOL v5 adds (`exp2`, `log2`, `hypot`, `trunc`, `sinpi`, `erf`, `fma`, `cancel_minus`, `round_ties_to_even`... and the nodes of their expressions), `restore_rounding()` and `exact_string()`. [GAOL v5]
Some names are in `gaol_core` only, and a program names them there: `gaol_core::rnd_keep`, for instance. [GAOL v5] They are `rnd_keep()` (GAOL v5 adds it, and had it in `gaol` in its 4.3 releases; GAOL 4 has none), `NaN_val`, `eval_stack`, `parse_interval()` and `parsing_names`.
What GAOL's code uses for itself is in `gaol_core` only too, out of the way of the names of a program that opens `gaol`: the functions saving and setting the rounding direction and the flush-to-zero modes (`rounding_state`, `get_rounding()`, `set_rounding()`, and `round_upward_if_needed()`, with `gaol_set_rounding_x86()`, which GAOL v5 had in `gaol` in its 4.3 releases

and GAOL 4 had not, and `get_flush_modes()`...), the functions computing the `pow` of `gaol` (below), and the internals GAOL 4 had in `gaol` without its manual documenting them: the functions `gaol_signbit()`, `modulo_k_pi()` and the `round_*_sse()`, the types `uintdouble`, `ullidouble`, `Interval_struct` and `prec_t`, and the variable `the_null_expr`. A program calling one of them names it in `gaol_core`: `gaol_core::get_rounding()`, for instance.

The `pow` of `gaol` takes the integer power for an integer exponent, a negative base included, and the `pow` of IEEE 1788-2015 for any other exponent (see Section 8.1.2, page 59). In `gaol_core`, its versions are named apart: `gaol_pown()`, `gaol_uipow()`, `gaol_pown_ll()`, `gaol_pow_real()`, `gaol_pow_hybrid()`, which are not in `gaol`.

3.5 The powers in the two namespaces

The power is where `gaol` and `gaol_ieee1788` differ. [GAOL v5] IEEE 1788-2015 has two powers (Table 9.1): `pown`(x, n), the integer power, for an integer n, defined for every x, x^0 being 1 and 0^n having no value for $n < 0$; and `pow`(x, y), the real power $e^{y \log x}$, for an interval y, defined for $x > 0$, and for $x = 0$ when $y > 0$. GAOL 4 had a single `pow`, which took the power of a negative base, and `gaol` keeps it: its `pow` is a hybrid of the two.

- In `gaol_ieee1788`, `pown(x, n)` is the integer power, for an n of any integer type, and `pow(x, y)` the real power: only the part of x in $[0, +\infty]$ counts, whatever y, and `pow(x, p)` with a number p is `pow(x, [p])`, an integer p included.
- In `gaol`, `pow(x, y)` takes the integer power where the exponent is an integer: an integer type, a double that is an integer, or a degenerate interval $[n, n]$ holding an integer, beyond the `int`s too; any other exponent takes the real power of the standard. There is no `pown` in `gaol`.

	`gaol`	`gaol_ieee1788`
`pow`$([-4, -1]$`, 2)`	$[1, 16]$	$\varnothing$ (`pown`: $[1, 16]$)
`pow`$([-2, 3]$`, 3)`	$[-8, 27]$	$[0, 27]$
`pow`$[0, 0]$`, 0)`	$[1, 1]$	$\varnothing$ (`pown`: $[1, 1]$)
`pow`$([-2, 2]$`, -1)`	$[-\infty, +\infty]$	$[0.5, +\infty]$
`pow`$([-2, -1]$`,` $[10^{10}, 10^{10}]$`)`	$[1, +\infty]$	$\varnothing$
`pow`$([-1, 4]$`, 0.5)`	$[0, 2]$	$[0, 2]$
`pow`$[0, 2]$`, -2)`	$[0.25, +\infty]$	$[0.25, +\infty]$

The two agree on $x \subseteq [0, +\infty]$ and on every exponent that holds no integer alone; they differ where x has a negative part and the exponent is an integer, and at 0^0. The `pow` of `gaol` is thus not monotone for the inclusion of its exponent: `pow`$([-4, -1]$`,` $[2, 2]$`)` is $[1, 16]$, and `pow`$([-4, -1]$`,` $[1.5, 2.5]$`)`, whose exponent holds 2, is empty; the `pow` of the standard is. Both give the tightest bounds, but for the integer power, which is one double beyond the tightest where it is within twice the error of its products of a double, and within three doubles of it beyond 2^{53} for an exponent that is no double (Chapter 12).

The rest of GAOL follows the same split. The expressions `pow(e, n)` and `pow(e1, e2)` of `gaol` are the hybrid power, `pown(e, n)` and `pow(e1, e2)` of `gaol_ieee1788` the two powers of the standard. The text `"pow([-4,-1],2)"` is read as $[1, 16]$ by the `textToInterval` of `gaol`, and as the empty set by that of `gaol_ieee1788`, which reads `pow` as the real power and `"pown([-4,-1],2)"`

as the integer power, a name `gaol` does not read. The reverse functions are those of the standard in both namespaces: `pow_rel` and `pow_exponent_rel` (powRev1 and powRev2) reverse the real power, and `nth_root_rel` (pownRev) the integer power.

A program written for GAOL 4, or for IBEX and Codac, whose intervals take the `pow` of GAOL, opens `gaol`; a program that wants the results of the standard, or compares GAOL with other libraries of IEEE 1788-2015, opens `gaol_ieee1788`.

Example

```
#include <iostream>
#include <gaol/gaol.h>

int main(void)
{
  gaol::interval x(-4, -1);
  std::cout << gaol::pow(x, 2) << ' '
            << gaol_ieee1788::pow(x, 2) << ' '
            << gaol_ieee1788::pown(x, 2) << std::endl;
  gaol::cleanup();
  return 0;
}
[1, 16] [empty] [1, 16]
```

3.6 Common errors

In this section, we will review common errors made when using GAOL in the—forlorn?—hope that it will help prevent users from making them.

3.6.1 Floating-point arithmetic and rounding

Programming with floating-point numbers is one of the few activities where one must always consider ones compiler defiantly. For example, let us consider the following piece of code:

Example

```
#include <gaol/gaol>  // We do not import the gaol namespace

using gaol::interval;

int main(void) {
  interval one_tenth(0.1); BEWARE: wrong !

  [some code using one_tenth]
  gaol::cleanup();
}
```

Though a rational perfectly representable in decimal, `0.1` is not representable in binary (at least, not with a finite number of bits) and thus requires rounding. Obviously, the purpose of the user was to define an interval containing this value. However, `0.1` will be rounded *at compile time*, most certainly to the nearest representable floating-point number $\updownarrow 0.1 \updownarrow$. As a consequence, `one_tenth` will be a degenerate interval containing only $\updownarrow 0.1 \updownarrow$, and the *containment property* will be violated.

The right way to deal with rational constants that might not be perfectly representable as floating-point numbers is to stringify them, and to read them

Definition 2 (Round to nearest) *Given $\mathbb{R}$ the set of real numbers and $\mathbb{F}$ the set of floating-point numbers (`double`), $\updownarrow x \updownarrow$ is, for $x \in \mathbb{R}$, the $y \in \mathbb{F}$ that minimizes $|x - y|$:*

$$|x - \updownarrow x \updownarrow| = \min\{|x - y| \mid y \in \mathbb{F}\}$$

the one whose significand is even when two are equally near.

with `gaol::textToInterval()` (see Section 5.1, page 39), such that they can be correctly rounded downward and upward at *runtime*:

Example

```
#include <gaol/gaol>

using gaol::interval;

int main(void)
{
  // OK: this is the right way
  interval one_tenth = gaol::textToInterval("0.1");
  // Another possible way
  interval one_tenth2 = gaol::textToInterval("1/10");

  [some code using one_tenth]
  gaol::cleanup();
  return 0;
}
```

Definition 3 (canonical interval) *A non-empty interval* $I = [a, b]$ *is* canonical *if and only if* $a \geqslant b^-$.

Now, `one_tenth` will be the smallest floating-point interval enclosing `0.1`. An interval like this one, containing at most two consecutive floating-point numbers, is called a *canonical interval*. Written in hexadecimal (see Section 10.2.2, page 86), which shows the bounds exactly, the two intervals are:

Output

```
interval(0.1) = [0x1.999999999999ap-4]
one_tenth     = [0x1.9999999999999p-4, 0x1.999999999999ap-4]
```

3.6.2 Functions of numbers

A function called with a number is the function of the C library, even where the namespace `gaol` is open: `sqrt(2.0)` is a `double`, rounded in the direction in effect, and no enclosure of $\sqrt{2}$. A `double` no longer becomes an interval by itself, the constructor being explicit, so that `interval r = sqrt(2.0);` does not compile; but `interval r(sqrt(2.0));` does, and gives a point that does not contain $\sqrt{2}$. The enclosure is the function of GAOL on an interval, or the text read by `textToInterval()`: [GAOL v5]

Example

```
interval root2 = textToInterval("sqrt(2)");
cout << boolalpha << interval(sqrt(2.0)).set_contains(root2) << ' '
     << sqrt(interval(2.0)).set_contains(root2);
```

false true

For the same reason, `gaol::sin(0.5)` does not compile: it is written `gaol::sin(interval(0.5))`, and `sin(0.5)` alone is the `sin` of C. Writing code inside `namespace gaol {}` has the same effect: the using-declarations of `gaol` are visible there, a call `sqrt(2.0)` finds `gaol::sqrt`, which hides `::sqrt`, and does not compile. This was already the case with GAOL 4.

3.6.3 Midpoint and radius

`interval(m - r, m + r)`, with doubles, is not an interval enclosing $[m - r, m + r]$: the program computes `m - r` and `m + r` itself, in the direction in effect, which is upward until `cleanup()`, so that `m - r` may be above the real $m - r$, and to nearest each bound may be on either side. With `m = 1.0`

and `r = 1e-20`, the interval is $[1, 1 + 2^{-52}]$ while the direction is upward, and does not contain $1 - 10^{-20}$. `interval::midrad(m, r)` (see Section 6, page 45), or `m + interval(-r, r)`, computes both bounds in intervals, and gives $[1 - 2^{-53}, 1 + 2^{-52}]$, which does. [GAOL v5]

Example

```
1 double m = 1.0, r = 1e-20;
2 interval wrong(m - r, m + r);  // Does not enclose [m - r, m + r]
3 interval right = interval::midrad(m, r);  // Does
```

3.6.4 The doubles of the program

The rounding direction that GAOL leaves applies to the rest of the program, which computes its own doubles in it: after an operation of GAOL, `1.0 / 3.0` is the double above 1/3, and `printf()` and the streams of the GNU C library write the decimal digits of a double rounded upward too, until `cleanup()` (see Section 3.2.1, page 18, which gives what changes, and what to do). [GAOL v5]

3.6.5 Zero

`interval x(0, 0)`, `interval x(0)` and `interval::zero()` are $[0, 0]$. The literal `0` is also a null pointer, and the constructors from one and from two strings of GAOL 4 made these calls ambiguous; GAOL v5 has no constructor from strings. [GAOL v5]

3.6.6 The sign of a zero bound

A bound that is zero has a sign, which differs between the builds: `sqr(interval(-1, 2))` is $[-0, 4]$ with the SSE2 intervals, which the default build uses on x86 processors, and with the NEON intervals, which it uses on 64-bit ARM processors, and $[0, 4]$ with the FPU intervals, those of the other processors, of Visual C++ and clang-cl on x86, and of the builds with `GAOL_SIMD` off (Visual C++ and clang-cl for x64 have the SSE2 intervals) [GAOL v5]; `interval::zero()` is $[-0, 0]$ with the first two and $[0, 0]$ with the third. `pow(x, 2)` is `sqr(x)`, and has the same sign. [GAOL v5] The NEON and the SSE2 intervals can differ too: the maximum of 64-bit ARM, which the product takes of its four products, ranks $+0$ above -0, where the one of x86 returns its second operand, so that the lower bound of `interval(-1, 0) * interval(-1, 0)` is -0 with the NEON intervals and $+0$ with the others (see Section 3.2.2, page 21). IEEE 1788-2015 and GAOL take the two zeros for the same number: `x.left() == 0.0` is `true` whichever it is, and a point interval at zero is written `[0]`. A program that reads the sign, with `std::signbit(x.left())` or `1.0 / x.left()`, which is $-\infty$ or $+\infty$, gets one answer with a build and the other with the other: it compares the bounds with `==`, `<` and `<=`. [GAOL v5]

3.6.7 The empty set and the domain of a function

A function is computed on the part of its argument that lies in its domain (IEEE 1788-2015), and GAOL, which has no decorations, says nothing of the rest: `sqrt(interval(-1, 4))` is $[0, 2]$, `log(interval(-1, 0))` is empty, and so are `interval(1, 0)` and the interval of one infinite or NaN double. The empty set then passes the tests as if it were a good answer: it has no point that contradicts a certainly relation, and it is included in every interval, so

that a box outside the domain of a function is both inside and outside the set a paving looks for:

Example

```
interval e = sqrt(interval(-2.0, -1.0)), z(0.0, 1.0);
cout << boolalpha << e.is_empty() << ' ' << (e < z) << ' '
     << (e > z) << ' ' << z.set_contains(e) << ' '
     << e.set_disjoint(z);
true true true true true
```

A program tests `is_empty()` first, and decides itself what a box outside the domain of its function means, as `examples/08_sivia.cpp` does. [GAOL v5]

3.6.8 Relations, sorting and containers

The symbols `<`, `<=`, `>` and `>=` are the certainly relations of IEEE 1788-2015, which do not tell which of two intervals is the greater, and there is no `==`: `interval(1,3) < interval(2,4)` and `interval(1,3) >= interval(2,4)` are both `false` (see Section 7.4, page 52). In C-XSC and in PROFIL/BIAS, `x <= y` means that `x` is a subset of `y`, and `r <= x` that the number `r` is in `x`: code ported from them compiles, and changes its meaning (see Section 11.5, page 97). None of them is an ordering for the standard library: `std::sort`, `std::set` and `std::max` take the comparator `gaol::lexicographic_less` (see Section 7.5, page 53). [GAOL v5]

3.6.9 Both namespaces

A program opens the namespace `gaol` or the namespace `gaol_ieee1788`, not both (see Section 3.4, page 27):

Example

```
#include <gaol/gaol.h>
using namespace gaol;
using namespace gaol_ieee1788;

gaol::interval x(1,2), y = pow(x,x); // Does not compile
```

The compiler reports that the call of `pow` is ambiguous. Not every call is: with an `int` or an `unsigned int` for exponent, `pow(x, 2)` compiles, and takes the `pow` of `gaol`, silently, where that of `gaol_ieee1788` gives another result for a negative base (see Section 3.5, page 28). Only the calls with another type of exponent, `pow(x, 2.0)`, `pow(x, 2L)` or `pow(x, y)`, are ambiguous: [GAOL v5]

Example

```
#include <iostream>
#include <gaol/gaol.h>
using namespace gaol;
using namespace gaol_ieee1788;

int main(void)
{
  gaol::interval x(-4,-1);
  std::cout << pow(x, 2) << std::endl; // The pow of gaol
  gaol::cleanup();
  return 0;
}
[1, 16]
```

3.6.10 Bisection

A loop that cuts an interval until it is narrow, `while (x.width() > eps)`, never ends on an unbounded interval: `bisect(0.5)`, like `split()`, cuts [1, +∞] at `DBL_MAX`, and the half [DBL_MAX, +∞] is cut at `DBL_MAX` again, its width staying infinite. What a loop stops on is `is_bisectable()`, which is `false` for this half, and for an interval of two consecutive doubles, the cut of which gives the interval itself as one half (see Section 9.2, page 78): [GAOL v5]

Example

```
interval x(1.0, GAOL_INFINITY);
x = x.bisect(0.5).second;
cout << boolalpha << x << ' ' << x.width() << ' '
     << x.is_bisectable();
// [1.797693134862315e+308, inf] inf false
```

3.6.11 Newton's method: the division and the relational division

The step $N(X) = m - f(m)/F'(X)$ of the interval Newton method divides with `/`, the set of the quotients a/b of a point of the numerator by a point other than 0 of the denominator. Where $0 \in F'(X)$ and $f(m) = 0$, the quotient $0/F'(X)$ is [0, 0], so that $N(X) = m$ lies inside X and "proves" that X holds exactly one root, where it holds seven for $\sin x - x/10$ on [−10, 10]. A root satisfies a relation, $(x - m)\,d = -f(m)$ for some d in $F'(X)$, which `%`, or `div_rel()`, the reverse of the product (see Section 8.2, page 66), solves: it gives the whole line, rightly, when d and $f(m)$ can both be 0.

Example

```
interval fm(0.0), D(-1.1, 0.9);
cout << fm / D << ' ' << fm % D << ' '
     << div_rel(fm, D, interval(-5.0, 5.0));
// [0] [-inf, inf] [-5, 5]
```

The functional division is the right one where $F'(X)$ holds no 0. `examples/05_interval_newton.cpp` shows the trap and the extended step.

3.6.12 Reading text

`gaol::textToInterval()` throws `std::invalid_argument` for a text that is no interval, and `std::domain_error` for a text that is one but calls a function with an argument it does not take, such as `nth_root(8, 1.5)`, whose order is not an integer: a loop that catches the first lets the second escape. Both derive from `std::logic_error` (Chapter 17). [GAOL v5]

Example

```
try {
  cout << textToInterval("nth_root(8, 1.5)");
} catch (const std::invalid_argument&) {
  cout << "std::invalid_argument";
} catch (const std::domain_error&) {
  cout << "std::domain_error";
}
// std::domain_error
```

`gaol_ieee1788::textToInterval()` throws neither: it gives the empty set for every text that is no interval, as IEEE 1788-2015 has it, so that a malformed text cannot be told from `"[empty]"`:

Example

```
1 cout << gaol_ieee1788::textToInterval("[1, 2") << ' '
2      << gaol_ieee1788::textToInterval("[empty]");
3 [empty] [empty]
```

A null pointer is no text, and the `textToInterval()` of both namespaces and `exactToInterval()` throw `std::invalid_argument` for it: `textToInterval(nullptr)`, `textToInterval(std::getenv("UNSET"))` and `gaol_ieee1788::exactToInterval(nullptr)` do not reach the reader. Their argument is a `const std::string&`, which the call builds from the pointer first, and the C++ standard leaves that undefined (libstdc++ throws `std::logic_error`); the overloads of a C string take the null pointer before. [GAOL v5]

3.6.13 Threads and the global settings

The output format of intervals and their number of digits, set by `interval::format()` and `interval::precision()`, are variables of the program, shared by its threads, and not atomic: setting one while another thread writes an interval, with `operator<<`, the conversion to a `std::string` or `gaol_ieee1788::intervalToText()` (which reads the precision only), is a data race, with the undefined behavior of C++ (ThreadSanitizer reports it). The program that sets them synchronizes with the threads that write: it sets them before starting its threads, or takes a lock of its own around the setting and around each output. Without a setting in between, several threads write intervals at once safely, and so they read strings (see Section 10.1, page 81); `exact_string()` reads neither setting. [GAOL v5]

3.6.14 The flags of interval arithmetic

Compiled without the flags of interval arithmetic (see Section 3.1, page 14), the inline operations of GAOL may be evaluated by the compiler at compile time, rounded to nearest, or contracted into fused multiply-adds, and give bounds that do not enclose the exact results. `gaol/gaol_config.h` stops the compilation where it can tell:

Output

```
% c++ -ffast-math -c program.cpp
gaol/gaol_config.h: error: #error "GAOL cannot be
compiled with -ffast-math (nor -Ofast): the bounds it
computes would not enclose the exact results (remove
the option, or put the flags of gaol.pc and gaol::gaol
after it)"
```

It stops `-ffinite-math-only` in the same way, with which the compiler takes NaN, the bounds of the empty interval, and infinities never to occur (see Section 2.3.2, page 4). Taking the flags from `gaol::gaol` or `gaol.pc` avoids the question. The link is not seen by the header: a program linked with `-Ofast` gets the flush-to-zero mode, which makes the bounds of the operations with subnormal operands and results wrong (see Section 3.2.3, page 21). [GAOL v5]

4 Initialization and cleanup

GAOL initializes itself when the program starts, before `main()`: `init()` need not be called. Since Release 1.0 of GAOL, there is an automatic initialization and cleanup, so that GAOL works even when the program calls neither function. Since GAOL v5, the initialization is made with every compiler, Visual C++ included: each file including the headers of GAOL holds a static object whose constructor initializes GAOL the first time, as `<iostream>` initializes the standard streams, so that GAOL is initialized before the static objects that such a file defines after its `#include`. Each operation also sets the rounding direction upward itself (see Section 3.2, page 17), so that `init()` no longer has to be called again after the calling code changed it. [GAOL v5]



`cleanup()`, on the other hand, is to be called by the program right after its last use of GAOL, at the latest at the end of `main()`, with every compiler. [GAOL v5] Unless GAOL is built to preserve the rounding direction, the initialization leaves the rounding direction upward for the whole program, not only for the operations of GAOL, and `cleanup()` sets back the one the program started with. The automatic cleanup does so too, but only when the program ends, once `main()` has returned and the static objects are destroyed, or when the library is unloaded: until then, the code after the last use of GAOL (the rest of `main()`, the destructors of static objects, the functions registered with `std::atexit()`) computes its doubles rounded upward.

The direction set back is the one GAOL found when it initialized itself. It is not the one the program started with when an operation of GAOL came first: in the initialization of a static object of a file that uses GAOL without including its headers, through a library of the program for instance, or, with Clang, of an `inline` variable or a static member of a class template, whose initialization C++ does not order with the rest of the file.

The constants of GAOL (π, the masks of its SSE2 operations) are initialized when compiling: a static object of the program may compute intervals, before the files of GAOL are initialized, which with the static library of the CMake build come after those of the program, except with MinGW-w64. [GAOL v5]

bool **init** (int *dbg_lvl* = 0)

Initializes the variable `debug_level` (see Section 18, page 117) to the value of *dbg_lvl*.
Unless GAOL was built to preserve the rounding direction, the first call notes the rounding direction it finds, which `cleanup()` sets back, then sets the floating-point environment to its default one, then the rounding direction towards $+\infty$. In addition, it sets the number of digits to display for interval bounds to 16. In a build that takes the AVX-512 path, the first call stops the program, with a message on the standard error, when the processor or the system does not run the AVX-512 instructions, even in a program that computes nothing. [GAOL v5]
Returns `true` if the library was not already initialized and `false` otherwise. After its first call, the function sets the debugging level to its new value and, unless the rounding direction is preserved, the rounding direction upward again: the C runtime of Windows resets it when the program starts, after the library initialized itself.

bool **cleanup** (void)

Unless GAOL was built to preserve the rounding direction, sets back the rounding direction that the first call of `init()` found: the one of the program when GAOL initialized itself, to nearest when the program starts. On x86 processors, the x87 unit and the SSE instructions each get theirs back. The rest of the floating-point environment is left as it is, the exception flags raised since in particular. The rounding direction being one of each thread, it is set back in the thread calling `cleanup()`. [GAOL v5]
The program calls it right after its last use of GAOL: an operation of GAOL after it sets the rounding direction upward again. The automatic cleanup calls it when the program ends, or when the library is unloaded, if the program did not, and frees what the initialization allocated, after the static objects of the program are destroyed: the expressions of `gaol/parser/gaol_expression.h` still alive when the program calls `cleanup()` remain valid.
Returns `true` the first time it is called and `false` afterwards: only the first call sets back the rounding direction.

Example

```
#include <gaol/gaol.h>

int main(void)
{
  gaol::init(1); // First level of debugging requested;
                 // without it, GAOL initializes itself

  [Some code using interval arithmetic]

  gaol::cleanup(); // Always, right after the last use
                   // of GAOL: sets back the rounding
                   // direction
  return 0;
}
```

void **restore_rounding** (void)

Sets back the rounding direction that the first call of `init()` found, as `cleanup()` does at its first call, but as many times as the program calls it. [GAOL v5] A program that goes back to GAOL after `cleanup()`, whose operations set the direction upward again, calls it after each use. Like `cleanup()`, it sets back the direction of the calling thread only, and leaves the rest of the floating-point environment as it is. With `GAOL_PRESERVE_ROUNDING` it does nothing, GAOL leaving the direction of the program alone (see Section 3.2.1, page 18).

5 Interval creation and assignment

The methods for creating an interval and assigning a new value to an already existing one are described in the following.

5.1 Constructors

One can create an interval with four constructors, or with `textToInterval()`:

- by providing its left and right bounds:

Example

```
interval x(l,r);
```

where l and r are `double`s or of a type that is castable into a `double`. The result is the empty set when $[l, r]$ is not an interval: for a left bound $+\infty$, a right bound $-\infty$, bounds in the wrong order ($l > r$) and NaN bounds; [GAOL v5] an integer bound of any integer type but `bool`, the other bound being an integer, a `double` or a `float`, is taken as the integer it is: the left bound is the largest double at most l, the right one the smallest double at least r, so that `interval(1, 9007199254740993LL)` contains $2^{53}+1$, which no double is, and which its conversion to a double made 2^{53} or $2^{53}+2$ according to the rounding direction; and the result is the empty set for $l > r$ compared as numbers, `interval(9007199254740993LL, 9007199254740992LL)` being empty though both lie between the same two doubles; [GAOL v5]

- by providing only one bound, for degenerate point intervals:

Example

```
interval x(v);
```

This is equivalent to: `interval x(v,v);`, and the empty set for an infinite or NaN v. The constructor is `explicit`: a `double` no longer converts silently to an interval, and `interval x = 0.5;`, `return 0.5;` in a function returning an interval, `sin(0.5)` or `min(x, 1.0)` do not compile, where `interval x(0.5)`, `interval(0.5)`,

`sin(interval(0.5))` and `min(x, interval(1.0))` do. The assignment (see Section 5.2, page 42), `&=`, `|=` and the other assignments combined with an operation (see Section 5.3, page 43), the arithmetic operators and the relation symbols (see Section 7.4, page 52) take a `double` as they take `interval(d)`; [GAOL v5] an integer n of any integer type but `bool` gives the tightest interval containing it, $[n, n]$ where n is a double, as every `int` is, and the two doubles around it otherwise: `interval(9007199254740993LL)` is $[2^{53}, 2^{53} + 2]$, where the conversion of $2^{53} + 1$ to a double gave $[2^{53}, 2^{53}]$ or $[2^{53} + 2, 2^{53} + 2]$, neither of which contains it, and `interval(ULLONG_MAX)` contains $2^{64} - 1$. This constructor is `explicit` too, and the same operations take an integer as they take `interval(n)`, the relations comparing it as the integer it is (see Section 7.4, page 52), and `&=` giving `interval(n)` where the interval contains n, the empty set otherwise; [GAOL v5]

- without providing any bound:

Example

```
1  interval x;
```

This is equivalent to:

Example

```
1  interval x(-GAOL_INFINITY,+GAOL_INFINITY);
```

where `GAOL_INFINITY` represents the infinity value of the `double` format (see Section 13.1, page 105);

- by copying an already existing interval (*copy constructor*):

Example

```
1  interval x(-12,12), y=x, z(x);
```

- by reading a string representing an interval in the same format as the one used for input (see Section 10.1.1, page 82), with `textToInterval()`:

interval **textToInterval** (const std::string& *s*)

Returns the interval the string *s* writes, read with the names of the functions of GAOL (see Section 10.1.1, page 82). It is in the namespace `gaol`, under the name IEEE 1788-2015 gives it.

Example

```
interval x = textToInterval("[-23, inf]"),
         y = textToInterval("[5*0.1+dmin, 89*sinh(2.1)]");
```

If the input string does not comply with the expected format, the exception `std::invalid_argument` is thrown if the library was compiled with exception support, which the CMake build always is; in the absence of exception support, the `errno` variable is set to `-1`, and the `gaol_error()` function prints an error message before the program aborts (see Section 17, page 113). The `textToInterval()` of `gaol_ieee1788` reads the names of the functions of the standard, and returns the empty set instead (Chapter 11). A null pointer is no string: the overload for a `const char*`, which `textToInterval(nullptr)` reaches, throws `std::invalid_argument` in both namespaces, and the same error without exception support (a message, then `abort()`). [GAOL v5]
GAOL v5 has no constructor from one string: `textToInterval()` takes the place of the constructor `interval(const char*)` of GAOL 4, which converted any `const char*` implicitly: `interval(0)` was ambiguous, and `x + p` compiled for a null `p`, and crashed. [GAOL v5]

interval **textToInterval** (const std::string& *sl*, const std::string& *sr*)

Returns the interval whose left bound is the one of the interval the string *sl* writes, and whose right bound is the one of the interval *sr* writes, and the empty set when the lower bound of the first is above the upper bound of the second (the intervals the two strings write, each rounded outward, not the numbers: `textToInterval("[1,u]")` orders the numbers).

Example

```
// x=[-4,4]
interval x = textToInterval("[-5,4]+1","[4,6]-[2,3]");
```

It is in the namespace `gaol`, and throws `std::invalid_argument` as `textToInterval(s)` does, for a null pointer in either place too. It takes the place of the constructor from two strings of GAOL 4, which kept the bounds [2, 1] of `interval("2", "1")`: `is_empty()` was true, but its sum with [0, 1] was [2, 2]. [GAOL v5]

interval interval::**midrad** (double *m*, double *r*) static

■ `midrad`$(m, r) = [\downarrow m - r \downarrow, \uparrow m + r \uparrow]$

Returns the interval of midpoint m and radius r, `interval(m) + interval(-r, r)`, which encloses every real of $[m - r, m + r]$, where `interval(m - r, m + r)` rounds both bounds upward. $r = 0$ gives the point m and $r = +\infty$ the whole line; a negative or NaN r gives the empty set, and so does an infinite or NaN m, as `interval(m)` does. [GAOL v5]

Example

```
cout << interval::midrad(1, 0.25) << ' '
     << interval::midrad(1, -1);
[0.75, 1.25] [empty]
```

Caution. you have to be very careful when creating an interval from floating-point constants. Remember that a rational number that is perfectly representable in the decimal base may require rounding in the binary base. For example, if you write the following statement:

Example

```
interval x(0.1);
```

you will *not* have created an interval containing 0.1 since this number has an infinite expansion in the binary base (i.e. it is impossible to represent it perfectly whatever the size of the mantissa may be). As a consequence, the constant 0.1 has very likely been rounded to the nearest floating-point number at compile-time. In such a case, you have to read it from a string instead:

Example

```
interval x = textToInterval("0.1");
```

Note. The degenerate intervals $[+\infty, +\infty]$ and $[-\infty, -\infty]$ do not exist in GAOL v5. [GAOL v5] IEEE 1788-2015 has no such interval, and its constructor fails there and gives the empty set (10.5.8, 12.12.7); the constructors of GAOL give the empty set too, as those of IBEX do, for an infinite point and for all the cases above:

Example

```
cout << interval(GAOL_INFINITY) << ' '
     << interval(-GAOL_INFINITY,-GAOL_INFINITY) << ' '
     << interval(2,1) << ' '
     << interval(1,GAOL_INFINITY) << endl;
[empty] [empty] [empty] [1, inf]
```

GAOL 4 built them as intervals of one infinite point, which its operators did not support. The intervals with one infinite bound, as $[1, +\infty]$, are intervals as any other.

Read from a string, the constant `inf`, an expression, stands for [MAX, $+\infty$], MAX being the largest double, and `-inf` for $[-\infty, -\text{MAX}]$; the interval literals whose left bound is $+\infty$ or whose right bound is $-\infty$ are the empty set, as `numsToInterval` of IEEE 1788-2015 has it:

Example

```
cout << textToInterval("inf") << ' ' << textToInterval("[inf]") << ' '
     << textToInterval("[-inf, -inf]") << endl;
[1.797693134862315e+308, inf] [empty] [empty]
```

The string `"<-inf, -inf>"` throws `std::invalid_argument`, its two bounds being different values (see Section 10.1.1, page 82).

5.2 Straight assignment

It is possible to assign a new value to an already existing interval in three different ways:

- by copying another interval:

Example

```
interval x(-12,12),
         y; // Here, y is [−∞, +∞]
y = x; // Now, y is [−12, 12]
```

- by using a string whose format follows the one expected for input (see Section 10.1, page 81), which `textToInterval()` reads:

Example

```
interval x;
x = textToInterval("[-inf, 123]");
```

GAOL 4 assigned the string itself, `x = "[-inf, 123]"`, which GAOL v5 refuses, having no constructor from one string; [GAOL v5]

- by using a `double`, which `operator=(double)` assigns as `interval(d)`, the constructor from one `double` being explicit: [GAOL v5]

Example

```
interval x;
x = 1234.5; // Now, x is [1234.5, 1234.5]
x = GAOL_INFINITY; // Now, x is empty
```

Both follow the rules of the previous section: assigning an infinity to an interval empties it. An integer n is assigned as `interval(n)`, the tightest interval containing it. [GAOL v5]

Note that there is no method for modifying a bound of an interval since intervals must be considered as an atomic concept.

5.3 Assignment combined with an operation

The following assignment operators combine the value of the interval pointed to by `self` and the value of the right-hand side interval.

In this manual, we note `self` the object to which `this` is a pointer in C++.

interval& interval::**operator&=** (const interval& *I*)
interval& interval::**operator&=** (double *d*)

- self ← self ∩ I
- self ← self ∩ interval(d)

Assigns to `self` the interval resulting from the intersection of `self` and I (resp. `interval(d)`, empty for an infinite or NaN d), which is the empty set when they are disjoint. [GAOL v5]

Example

```
interval x(-12,12);
x &= interval(-6,23); // Now, x is [−6, 12]
```

interval& interval::**operator|=** (const interval& *I*)
interval& interval::**operator|=** (double *d*)

- self ← □(self ∪ I)
- self ← □(self ∪ interval(d))

Assigns to `self` the interval hull of the union of `self` and I (resp. `interval(d)`, which leaves `self` as it is for an infinite or NaN d). [GAOL v5]

Example

```
interval x(-12,12);
x |= interval(-6,23); // Now, x is [−12, 23]
```

interval& interval::**operator+=** (const interval& *I*)
interval& interval::**operator+=** (double *d*)

- self $\leftarrow$ self $+ I$
- self $\leftarrow$ self $+ d$

Assigns to `self` the interval resulting from adding `self` and I (resp. d).

Example

```
interval x(-12,12);
x += interval(-6,23); // Now, x is [−18, 35]
```

interval& interval::**operator-=** (const interval& *I*)
interval& interval::**operator-=** (double *d*)

- self $\leftarrow$ self $- I$
- self $\leftarrow$ self $- d$

Assigns to `self` the interval resulting from subtracting I (resp. d) from `self`.

Example

```
interval x(-12,12);
x -= interval(-6,23); // Now, x is [−35, 18]
```

interval& interval::**operator*=** (const interval& *I*)
interval& interval::**operator*=** (double *d*)

- self $\leftarrow$ self $\times I$
- self $\leftarrow$ self $\times d$

Assigns to `self` the interval resulting from multiplying `self` and I (resp. d).

Example

```
interval x(-12,12);
x *= interval(-6,23); // Now, x is [−276, 276]
```

interval& interval::**operator/=** (const interval& *I*)
interval& interval::**operator/=** (double *d*)

- self $\leftarrow$ self$/I$
- self $\leftarrow$ self$/d$

Assigns to `self` the interval resulting from dividing `self` by I (resp. d).

Example

```
interval x(-12,12);
x /= interval(-6,23);   // Now, x is [−∞, +∞]
x /= interval::zero();  // Now, x is ∅
```

interval& interval::**operator%=** (const interval& *I*)
interval& interval::**operator%=** (double *d*)

Assigns to `self` the interval resulting from dividing `self` by I (resp. d), using a *relational division* (see Section 8.2, page 66).

Example

```
interval x(-12,12);
x %= interval(-6,23);   // Now, x is [−∞, +∞]
x %= interval::zero();  // Now, x is [−∞, +∞]
```

The operators with a double compute as those with an interval, by the sign of the double; an infinite double gives the empty set, as `interval(d)` is empty. [GAOL v5]

6 Interval constants

For convenience, some useful intervals and some canonical intervals enclosing real constants are defined as static functions of the `interval` class:

Function	Value
`interval::emptyset()`	$\varnothing$
`interval::half_pi()`	$[\downarrow \frac{\pi}{2} \downarrow, \uparrow \frac{\pi}{2} \uparrow]$
`interval::minus_one_plus_one()`	$[-1, 1]$
`interval::negative()`	$[-\infty, 0]$
`interval::one()`	1
`interval::one_plus_infinity()`	$[1, +\infty]$
`interval::pi()`	$[\downarrow \pi \downarrow, \uparrow \pi \uparrow]$
`interval::positive()`	$[0, +\infty]$
`interval::two_pi()`	$[\downarrow 2\pi \downarrow, \uparrow 2\pi \uparrow]$
`interval::universe()`	$[-\infty, +\infty]$
`interval::zero()`	0

The enclosures of π, 2π and $\pi/2$ are the tightest ones. The empty set is held as an interval whose two bounds are NaN.

Example

```
1 cout << interval::emptyset() << ' ' << interval::pi() << endl;
2 [empty] [3.141592653589793, 3.141592653589794]
```

7 Interval relations

Interval relations may be divided into two groups. Given I and J two intervals, we have:

1. *set relations*: intervals I and J are considered as sets of reals. For example:

   $$I = J \Leftrightarrow (\forall x \in I, \exists y \in J : x = y) \wedge (\forall y \in J, \exists x \in I : x = y)$$

   Basically, two intervals are equal in that mode if they have the same bounds;

2. *certainly relations*: the relations must be true for any tuple of values in the intervals. For example:

   $$I \leqslant J \Leftrightarrow (\forall x \in I, \forall y \in J : x \leqslant y)$$

   Then, I is less than or equal to J in that mode if its right bound is at most the left bound of J.

GAOL 4 had a third group, the *possibly relations*, true if at least one tuple of values verifies the real relation, and a certain equality, `certainly_eq()` and `certainly_neq()`. They are gone from GAOL v5, and so are the operators `==` and `!=` on intervals: [GAOL v5] `certainly_neq()` was the negation of `certainly_eq()`, which is "possibly not equal", so that [3, 4] was certainly not equal to [3, 4]. IEEE 1788-2015 has neither. Its equality is equal, `set_eq()`; that no value of I equals a value of J is disjoint, `set_disjoint()`; and "possibly equal" is `!set_disjoint()`.

The relation symbols $<$, $\leqslant$, $>$ and $\geqslant$ are the certainly relations in every build (see Section 7.4, page 52).

The relations that IEEE 1788-2015 defines (Tables 10.3 and 10.4) follow its definitions, for the empty set and for the infinite bounds too: [GAOL v5] equal, subset, interior, disjoint, precedes, strictPrecedes, less and strictLess. Each method below says which one it is; the namespace `gaol_ieee1788` gives them their names (Chapter 11).

7.1 Set relations

bool interval::**set_contains** (const interval& *I*) const
bool interval::**set_contains** (double *d*) const

- $I \subseteq$ `self`
- $d \in$ `self`

Returns `true` if I (resp. $\{d\}$) is included in `self`: subset(I, `self`), and isMember(d, `self`) of IEEE 1788-2015 for a finite d. d shall not be a NaN. An integer of any integer type but `bool` is compared as the integer it is, here and in `set_strictly_contains()`: $[2^{53}, 2^{53}]$ does not contain $2^{53}+1$, which its conversion to a double made 2^{53} in the rounding to nearest, and $[2^{53}, 2^{53}+2]$ contains it strictly. [GAOL v5]

Example

```
interval x(-12,34), y(-12,5);

cout << boolalpha << x.set_contains(y) << ' '
     << x.set_contains(interval::emptyset()) << ' '
     << y.set_contains(x) << ' '
     << interval::emptyset().set_contains(x) << ' '
     << interval::emptyset().set_contains(interval::emptyset())
     << endl;
```

```
true true false false true
```

bool interval::**set_strictly_contains** (const interval& *I*) const
bool interval::**set_strictly_contains** (double *d*) const

- I interior to `self`
- d in the interior of `self`

Returns `true` if I is interior to `self`, interior(I, `self`) of IEEE 1788-2015: each bound of `self` is beyond that of I, an infinite bound being beyond the same infinite bound, and the empty set is interior to any interval, itself included. [GAOL v5] The second form returns `true` if d is strictly between the bounds of `self`.

Example

```
interval x(-10,12), y(-10, 11), z, t, u(10.5);

cout << boolalpha
     << x.set_strictly_contains(y) << ' '
     << z.set_strictly_contains(t) << ' '
     << x.set_strictly_contains(u) << ' '
     << interval::emptyset().set_strictly_contains(
                                 interval::emptyset()) << ' '
     << u.set_strictly_contains(interval::emptyset())
     << endl;
```

```
false true true true true
```

bool interval::**set_disjoint** (const interval& *I*) const

- `self` $\cap I = \varnothing$

Returns `true` if the intersection of `self` and I is empty: disjoint of IEEE 1788-2015.

Example

```
interval a(2,4), b(6,8);
cout << boolalpha << a.set_disjoint(b) << ' '
     << interval::emptyset().set_disjoint(interval::emptyset());
```

```
true true
```

bool interval::**set_eq** (const interval& *I*) const

■ $\forall x \in \texttt{self} \exists y \in I : x = y \wedge \forall y \in I, \exists x \in \texttt{self} : y = x$

Returns `true` if intervals `self` and I are equal when considered as sets of reals: equal of IEEE 1788-2015.

Example

```
cout << boolalpha << interval(4,8).set_eq(interval(4,8)) << ' '
     << interval::emptyset().set_eq(interval::emptyset()) << endl;
🖶 true true
```

bool interval::**set_neq** (const interval& *I*) const

■ $\neg(\texttt{self} = I)$

Returns `true` if `self` and I are not equal when considered as sets of reals.

Example

```
cout << boolalpha << interval::universe().set_neq(interval::emptyset());
🖶 true
```

bool interval::**set_le** (const interval& *I*) const

■ `self` interior to I

Returns `true` if `self` is interior to I: `I.set_strictly_contains(self)`, interior(`self`, I) of IEEE 1788-2015. [GAOL v5]

Example

```
interval a(2,GAOL_INFINITY), b(1,GAOL_INFINITY);
cout << boolalpha << interval(-4.5,3).set_le(interval(-10,10)) << ' '
     << interval::emptyset().set_le(interval(5,6)) << ' '
     << interval::emptyset().set_le(interval::emptyset()) << ' '
     << interval::universe().set_le(interval::universe()) << ' '
     << a.set_le(b);
🖶 true true true true true
```

bool interval::**set_leq** (const interval& *I*) const

■ `self` $\subseteq I$

Returns `true` if the real set defined by `self` is included in I: subset(`self`, I) of IEEE 1788-2015.

Example

```
cout << boolalpha << interval(4.5,6).set_leq(interval(4.5,6)) << ' '
     << interval(3.5,9).set_leq(interval(2,6)) << ' '
     << interval::emptyset().set_leq(interval::emptyset());
🖶 true false true
```

bool interval::**set_ge** (const interval& *I*) const

■ I interior to `self`

Returns `true` if I is interior to `self`: `set_strictly_contains(`I`)`.

Example

```
cout << boolalpha << interval(-10,10).set_ge(interval(-4.5,3)) << ' '
     << interval(1,2).set_ge(interval(1,2));
🖶 true false
```

bool interval::**set_geq** (const interval& *I*) const

■ $I \subseteq$ `self`

Returns `true` if the real set defined by `self` contains I: `set_contains(`I`)`.

7.2 Certainly relations

bool interval::**certainly_le** (const interval& *I*) const

■ $\forall x \in \texttt{self}, \forall y \in I\text{: } x < y$

Returns `true` if `self` is certainly strictly less than I: strictPrecedes of IEEE 1788-2015, true when either interval is empty. [GAOL v5]

Example

```
cout << boolalpha << interval(4,5).certainly_le(interval(6,9)) << ' '
     << interval(4,5).certainly_le(interval(5,9)) << ' '
     << interval::emptyset().certainly_le(interval(4,6)) << ' '
     << interval(4,6).certainly_le(interval::emptyset());
true false true true
```

bool interval::**certainly_leq** (const interval& *I*) const

■ $\forall x \in \texttt{self}, \forall y \in I\text{: } x \leqslant y$

Returns `true` if `self` is certainly less or equal to I: precedes of IEEE 1788-2015, true when either interval is empty. [GAOL v5]

Example

```
cout << boolalpha << interval(4,5).certainly_leq(interval(6,9)) << ' '
     << interval(4,5).certainly_leq(interval(5,9)) << ' '
     << interval(5,9).certainly_leq(interval(4,5)) << ' '
     << interval(4,8).certainly_leq(interval(5,9)) << ' '
     << interval::emptyset().certainly_leq(interval(4,6)) << ' '
     << interval(4,6).certainly_leq(interval::emptyset());
true true false false true true
```

bool interval::**certainly_ge** (const interval& *I*) const

■ $\forall x \in \texttt{self}, \forall y \in I\text{: } x > y$

Returns `true` if `self` is certainly strictly greater than I, or if either interval is empty.

Example

```
cout << boolalpha << interval(8,10).certainly_ge(interval(4,8)) << ' '
     << interval(8,10).certainly_ge(interval(4,7)) << ' '
     << interval::emptyset().certainly_ge(interval::emptyset());
false true true
```

bool interval::**certainly_geq** (const interval& *I*) const

■ $\forall x \in \texttt{self}, \forall y \in I\text{: } x \geqslant y$

Returns `true` if `self` is certainly greater or equal to I, or if either interval is empty.

Example

```
cout << boolalpha << interval(8,10).certainly_geq(interval(4,8)) << ' '
     << interval::emptyset().certainly_geq(interval::emptyset());
true true
```

bool interval::**certainly_positive** (void) const

■ self $\subseteq [0, +\infty]$

Returns `true` if the left bound of `self` is greater or equal to zero, or if `self` is empty.

Example

```
cout << boolalpha << interval::emptyset().certainly_positive() << ' '
     << interval(4,5).certainly_positive() << ' '
     << interval(-0.0,6).certainly_positive() << ' '
     << interval(-6,0).certainly_positive();
true true true false
```

bool interval::**certainly_strictly_positive** (void) const

■ self $\subseteq]0, +\infty]$

Returns `true` if the left bound of `self` is strictly greater than zero, or if `self` is empty.

Example

```
cout << boolalpha << interval::emptyset().certainly_strictly_positive() << ' '
     << interval(4,5).certainly_strictly_positive() << ' '
     << interval(-0.0,6).certainly_strictly_positive() << ' '
     << interval(-6,0).certainly_strictly_positive();
true true false false
```

bool interval::**certainly_negative** (void) const

■ self $\subseteq [-\infty, 0]$

Returns `true` if the right bound of `self` is lower or equal to zero, or if `self` is empty.

Example

```
cout << boolalpha << interval::emptyset().certainly_negative() << ' '
     << interval(4,5).certainly_negative() << ' '
     << interval(-6,0).certainly_negative() << ' '
     << interval(-6,-5).certainly_negative();
true false true true
```

bool interval::**certainly_strictly_negative** (void) const

■ self $\subseteq [-\infty, 0[$

Returns `true` if the right bound of `self` is strictly lower than zero, or if `self` is empty.

Example

```
cout << boolalpha << interval::emptyset().certainly_strictly_negative() << ' '
     << interval(4,5).certainly_strictly_negative() << ' '
     << interval(-6,0).certainly_strictly_negative() << ' '
     << interval(-6,-5).certainly_strictly_negative();
true false false true
```

7.3 The order of IEEE 1788-2015

bool interval::**less** (const interval& *I*) const

■ self $\leqslant I \wedge \overline{\texttt{self}} \leqslant \overline{I}$

Returns `true` if each bound of `self` is at most the bound of I on the same side: less of IEEE 1788-2015 (Table 10.3). Two empty sets are in the relation, and an empty set and a nonempty interval are not (Table 10.4). [GAOL v5]

Example

```
1 interval e = interval::emptyset();
2 cout << boolalpha << interval(1,2).less(interval(1,3)) << ' '
3      << interval(1,4).less(interval(2,3)) << ' '
4      << e.less(e) << ' ' << e.less(interval(1,2));
5 🖶 true false true false
```

bool interval::**strictly_less** (const interval& *I*) const

■ self $< I \wedge \overline{\texttt{self}} < \overline{I}$

Returns `true` if each bound of `self` is below the bound of I on the same side, an infinite bound counting as below itself: strictLess of IEEE 1788-2015. The empty sets are as for `less()`. [GAOL v5]

Example

```
1 interval a(-GAOL_INFINITY,1), b(-GAOL_INFINITY,2);
2 cout << boolalpha << interval(1,2).strictly_less(interval(1,3)) << ' '
3      << a.strictly_less(b) << ' '
4      << interval(0,1).strictly_less(interval(2,3));
5 🖶 false true true
```

7.4 Relational Symbols

The relation symbols are the certainly relations, whichever build configured GAOL: the option of GAOL 4 that chose between the set, certainly and possibly relations is gone, and `configure` and meson refuse it. [GAOL v5] `==` and `!=` are not defined on intervals: compare them with `set_eq()`, `set_disjoint()` or `less()`.

An interval and a `double` d compare as the interval and `interval(`d`)`, empty for an infinite or NaN d: `x < 0.0` is `x < interval(0.0)`, and still no `==` nor `!=`. These overloads are what the implicit conversion of a `double` to an interval gave before the constructor from one `double` became explicit. [GAOL v5]

An interval and an integer n of any integer type but `bool` compare as the interval and the integer it is, where it is no double: a double is below n if and only if it is at most the largest double below n, so that `interval(9007199254740992.0) < 9007199254740993LL`, $[2^{53}, 2^{53}] < 2^{53} + 1$, is `true`, where the conversion of $2^{53} + 1$ to the double 2^{53}, in the rounding to nearest, made it `false`. An integer that is a double compares as that double. [GAOL v5]

bool **operator<** (const interval& *I*, double *d*)
bool **operator<** (double *d*, const interval& *I*)
bool **operator<** (const interval& *I1*, const interval& *I2*)

Returns *I1*`.certainly_le(`*I2*`)`: strictPrecedes of IEEE 1788-2015.

bool **operator<=** (const interval& *I*, double *d*)
bool **operator<=** (double *d*, const interval& *I*)
bool **operator<=** (const interval& *I1*, const interval& *I2*)

Returns *I1*.certainly_leq(*I2*): precedes of IEEE 1788-2015.

bool **operator**> (const interval& *I*, double *d*)
bool **operator**> (double *d*, const interval& *I*)
bool **operator**> (const interval& *I1*, const interval& *I2*)

Returns *I1*.certainly_ge(*I2*).

bool **operator**>= (const interval& *I*, double *d*)
bool **operator**>= (double *d*, const interval& *I*)
bool **operator**>= (const interval& *I1*, const interval& *I2*)

Returns *I1*.certainly_geq(*I2*).

Example

```
cout << boolalpha << (interval(1,2) < interval(3,4)) << ' '
     << (interval(1,2) <= interval(2,4)) << ' '
     << (interval(1,3) <= interval(2,4)) << ' '
     << (interval(3,4) > interval(1,2));
true true false true
```

7.5 Intervals in containers and algorithms

The relation symbols do not tell which of two intervals is the greater: `interval(1,3) < interval(2,4)` and `interval(1,3) >= interval(2,4)` are both `false`, and `false` is not the opposite. The empty set, which has no point to contradict a certainly relation, is less than every interval, and greater, itself included: `e < e` is `true`.

None of them is a strict weak ordering, so intervals are not given to the algorithms and the containers of the standard library that compare with `<` (`std::sort`, `std::set`, `std::map`, `std::priority_queue`, `std::max`, `std::min`, `std::clamp`...). Two overlapping intervals are equivalent for `<`, a relation that is not transitive, and the empty set is less than itself, so that their behavior is undefined, and differs with the implementation of the library. A `std::set<interval>` that holds [1, 2] drops [1, 3] (two of the three intervals [1, 2], [1, 3] and [5, 6] stay), `std::sort` of a vector that holds an empty interval can read past the end of the vector (AddressSanitizer stops the program with libstdc++ for 17 intervals or more, at some positions of the empty one), and `std::max`([1, 3], [2, 4]) is [1, 3], its first argument, where `gaol::max` gives [2, 4]. `gaol::max(x, y)` and `gaol::min(x, y)` are the maximum and the minimum of the standard: the set of the greater (or the lesser) of a point of `x` and a point of `y`, and not the greater (or the lesser) of the two arguments.

The comparator to give them is `gaol::lexicographic_less`: [GAOL v5] a strict total order that puts the empty set first and the other intervals by their left bound, then by their right bound, −0 and +0 being the same bound. Two intervals with the same bounds, or two empty sets, are not less than each other, as a container needs. It is also in `gaol_core`; a program that opens `gaol_ieee1788` writes its name in full. Specializing `std::less` for intervals is left to the program, so the comparator is given explicitly:

Example

```
#include <algorithm>
#include <iostream>
#include <set>
#include <vector>
#include <gaol/gaol.h>
using namespace gaol;

int main(void)
{
  const interval boxes[] = { interval(1,2), interval(0,5),
                             interval::emptyset(), interval(1,3) };
  std::set<interval, lexicographic_less> s(boxes, boxes + 4);
  std::vector<interval> v(boxes, boxes + 4);
  std::sort(v.begin(), v.end(), lexicographic_less());
  for (const interval& x : v) std::cout << x << ' ';
  std::cout << s.size() << std::endl;
  gaol::cleanup();
  return 0;
}
[empty] [0, 5] [1, 2] [1, 3] 4
```

An algorithm that needs to know which of two boxes to process first, as a best-first search, orders them by a number of its own, the lower bound of an enclosure for instance, as `examples/06_global_optimization.cpp` does.

7.6 Interval-specific relations

bool interval::**straddles_zero** (void) const

■ $0 \in$ `self`

Returns `true` if `self` contains zero.

Example

```
interval x(0,4), y, z(-12,-5);

cout << boolalpha << x.straddles_zero() << ' '
     << y.straddles_zero() << ' '
     << z.straddles_zero() << endl;
true true false
```

Note. `I.straddles_zero()` ≡ `I.set_contains(0.0)`

bool interval::**strictly_straddles_zero** (void) const

■ $\{0\}$ interior to `self`

Returns `true` if zero is included in the interior of `self`.

Example

```
interval x(0,4), y, z(-12,-5);

cout << boolalpha << x.strictly_straddles_zero() << ' '
     << y.strictly_straddles_zero() << ' '
     << z.strictly_straddles_zero() << endl;
false true false
```

Note. The method call to `I.strictly_straddles_zero()` is equivalent to `I.set_strictly_contains(0.0)`

bool interval::**is_a_double** (void) const

Returns `true` whenever the left and right bounds of the interval are equal: isSingleton of IEEE 1788-2015.

Example

```
interval x(-12.5), y;

cout << boolalpha << x.is_a_double() << ' '
     << y.is_a_double() << ' '
     << interval::emptyset().is_a_double() << endl;
```

```
true false false
```

bool interval::**is_an_int** (void) const

Returns `true` whenever the left and right bounds of the interval are equal and castable into an integer (type `int`).

Example

```
interval x(-12.0), y;

cout << boolalpha << x.is_an_int() << ' ' << y.is_an_int() << ' '
     << interval::emptyset().is_an_int() << ' '
     << interval(3e10).is_an_int() << endl;
```

```
true false false false
```

bool interval::**is_canonical** (void) const

Returns `true` if `self` is nonempty and contains at most two floating-point numbers.

Example

```
interval x(0.0), y(-14,9);

cout << boolalpha << x.is_canonical() << ' '
     << y.is_canonical() << ' '
     << interval::emptyset().is_canonical() << ' '
     << interval::pi().is_canonical();
```

```
true false false true
```

bool interval::**is_empty** (void) const

■ `self` = ∅

Returns `true` if `self` is an empty interval: isEmpty of IEEE 1788-2015. It raises no floating-point exception, the invalid-operation one included, though the bounds of the empty set are NaN (see Section 3.3, page 24). [GAOL v5]

Example

```
cout << boolalpha << interval(4,5).is_empty() << ' '
     << interval(5,4).is_empty() << ' '
     << interval::emptyset().is_empty();
```

```
false true true
```

bool interval::**is_entire** (void) const

■ `self` = $[-\infty, +\infty]$

Returns `true` if `self` is the whole real line: isEntire of IEEE 1788-2015. [GAOL v5]

Example

```
cout << boolalpha << interval::universe().is_entire() << ' '
     << interval::positive().is_entire() << ' '
     << interval::emptyset().is_entire();
true false false
```

bool interval::**is_common_interval** (void) const

Returns `true` if `self` is nonempty and bounded: isCommonInterval of IEEE 1788-2015. [GAOL v5]

Example

```
cout << boolalpha << interval(1,2).is_common_interval() << ' '
     << interval::positive().is_common_interval() << ' '
     << interval::emptyset().is_common_interval();
true false false
```

bool interval::**is_zero** (void) const

■ `self` = $[0, 0]$

Returns `true` if `self` is equal to the interval containing only 0.

Example

```
cout << boolalpha << interval::zero().is_zero() << ' '
     << interval(0.0,0.0).is_zero() << ' '
     << interval(-0.0,+0.0).is_zero() << ' '
     << interval(0,5).is_zero() << ' '
     << interval::emptyset().is_zero();
true true true false false
```

bool interval::**is_symmetric** (void) const

Returns `true` if the left bound of `self` is the opposite of the right bound. An empty interval is not symmetric.

Example

```
cout << boolalpha << interval(-5,5).is_symmetric() << ' '
     << interval::emptyset().is_symmetric() << endl;
true false
```

bool interval::**is_finite** (void) const

Returns `true` if neither bound is infinite. The empty set, whose bounds are NaN, is finite; `is_common_interval()` is not true of it.

Example

```
cout << boolalpha << interval(4,GAOL_INFINITY).is_finite() << ' '
     << interval::emptyset().is_finite() << ' '
     << interval(5,80).is_finite();
false true true
```

8 Interval Arithmetic

The *containment principle* of (floating-point) interval arithmetic imposes that for any operation "$\circ$", and any intervals I and J, the following does hold:

$$I \circ J = \square\{i \circ j \mid i \in I, j \in J\}$$

where $\square$ is a function mapping any real set to the smallest floating-point interval containing it.

For example, if we consider the interval square root, we have:

$$\sqrt{I} = \square\{\sqrt{i} \mid i \in I\}$$

From monotonicity considerations, the square root of $[1, 2]$ is then $\sqrt{[1, 2]} = [1, \sqrt{2}]$. Now, another interpretation of the square root function is as follows:

$$\sqrt{}_r I = \square\{j \in \mathbb{R} \mid \exists i \in I\colon j^2 = i\}$$

This last definition stands for the *relational square root* and permits obtaining both negative and positive values. Hence, we have:

$$\sqrt{}_r[1, 2] = [-\sqrt{2}, \sqrt{2}]$$

This operator arises when we consider the relation

$$x^2 = y$$

which can alternatively be written

$$x = \sqrt{}_r y$$

Here, the *functional square root* is not suitable since it would induce the intersection of the domain of y with $[0, +\infty]$.

Some applications (mainly in the area of *constraint programming*) require the availability of such operators. As a consequence, GAOL offers both functional and relational versions of the main arithmetic operators.

Domains. A function is computed on the part of its argument within its domain, which is that of IEEE 1788-2015 (Tables 9.1 and 10.5): the part outside is left out, and an argument with no point of the domain gives the empty set, as `log`$([-4, 0])$ does. [GAOL v5] Every function gives the empty set for an empty argument. How tight each result is, for each operation, is stated in Chapter 12.

8.1 Functional Arithmetic

8.1.1 Basic operations

interval interval::**operator+** (void) const
interval **operator+** (const interval& *I*, double *d*)
interval **operator+** (double *d*, const interval& *I*)
interval **operator+** (const interval& *I1*, const interval& *I2*)

Addition of two intervals, or of one interval and a double: add of IEEE 1788-2015.

interval interval::**operator-** (void) const
interval **operator-** (const interval& *I*, double *d*)
interval **operator-** (double *d*, const interval& *I*)
interval **operator-** (const interval& *I1*, const interval& *I2*)

Negation (neg), or subtraction (sub) of two intervals, or of one interval and a double.

interval **operator*** (const interval& *I*, double *d*)
interval **operator*** (double *d*, const interval& *I*)
interval **operator*** (const interval& *I1*, const interval& *I2*)

Multiplication of two intervals, or of one interval and a double: mul of IEEE 1788-2015. A zero bound times an infinite bound counts as 0, as the product of a real number of the interval by 0. [GAOL v5]

interval **operator/** (const interval& *I*, double *d*)
interval **operator/** (double *d*, const interval& *I*)
interval **operator/** (const interval& *I1*, const interval& *I2*)

Functional division of two intervals, or of one interval and a double: div of IEEE 1788-2015. The quotient by an interval containing 0 is the hull of the quotients by its nonzero part, and the quotient by [0, 0] is the empty set.

Example

```
1 cout << interval(1,2) * interval(-3,4) << ' '
2      << interval(0,1) * interval(1,GAOL_INFINITY) << ' '
3      << interval(1,2) / interval(0,1) << ' '
4      << interval(1,2) / interval(-1,1) << ' '
5      << interval(1,2) / interval::zero();
6 [-6, 8] [0, inf] [1, inf] [-inf, inf] [empty]
```

interval **operator%** (const interval& *I*, double *d*)
interval **operator%** (double *d*, const interval& *I*)
interval **operator%** (const interval& *I1*, const interval& *I2*)

Relational division of two intervals, or of one interval and a double (see Section 8.2, page 66).

interval interval::**inverse** (void) const
interval **inverse** (const interval& *I*)

■ $1/I$

Inverse of I (resp. `self`): recip of IEEE 1788-2015. The inverse of an interval containing 0 is the hull of the inverses of its nonzero part.

Example

```
cout << inverse(interval(2,4)) << ' ' << inverse(interval(-1,2));
[0.25, 0.5] [-inf, inf]
```

interval **sqr** (const interval& *I*)

■ I^2

Square of I: sqr of IEEE 1788-2015.

interval **sqrt** (const interval& *I*)

■ $\sqrt{I}$

Functional square root of I, on the part of I in $[0, +\infty]$: sqrt of IEEE 1788-2015. The bounds are the tightest ones, whatever the rounding of the square root of the C library. [GAOL v5]

Example

```
cout << sqrt(interval(-4,9)) << ' ' << sqrt(interval(-4,-1));
[0, 3] [empty]
```

interval **fma** (const interval& *X*, const interval& *Y*, const interval& *Z*)

■ $\square\{xy + z \mid x \in X, y \in Y, z \in Z\}$

Fused multiply-add: fma of IEEE 1788-2015, each bound rounded once, the tightest enclosure. [GAOL v5]

Example

```
cout << fma(interval(1,2), interval(3,4), interval(-1,1));
[2, 9]
```

8.1.2 Powers and roots

The following four functions are those of the namespace `gaol`; the namespace `gaol_ieee1788` has its own `pow` (Chapter 11).

interval **pow** (const interval& *I*, int *n*)
interval **pow** (const interval& *I*, unsigned int *n*)
interval **pow** (const interval& *I*, double *p*)
interval **pow** (const interval& *I*, long *n*)
interval **pow** (const interval& *I*, long long *n*)
interval **pow** (const interval& *I*, const interval& *J*)

- I^n
- I^p
- I^J

Power function. `pow` is hybrid in GAOL v5: [GAOL v5]

- an integer exponent takes the *integer power*, pown of IEEE 1788-2015: I to the power n, the negative part of I included, $x^0 = 1$ for every x, 0 included, and no value at 0 for $n < 0$; so do a double p and a degenerate interval $J = [n, n]$ whose value is an integer, beyond the `int`s too; [GAOL v5] an exponent n of another integer type (`long`, `long long`, `short`...), with which `pow` was ambiguous, gives the same integer power, beyond 64 bits that of $\pm(2^{63}-1)$ or $\pm(2^{63}-2)$, of the sign and the parity of n, every power of a double but 0, ±1 and $\pm\infty$ being beyond the doubles there; [GAOL v5]

- any other exponent takes the *real power*, pow of IEEE 1788-2015, on the part of I in $[0, +\infty]$, where 0^y is 0 for $y > 0$ and has no value for $y \leqslant 0$. An exponent $[+\infty, +\infty]$ or $[-\infty, -\infty]$, or an infinite or NaN p, gives the empty set.

The result is thus not monotone for the inclusion of J: `pow([-4, -1], [2, 2])` is $[1, 16]$, and `pow([-4, -1], [1.5, 2.5])` the empty set. GAOL 4 truncated a double exponent p, and computed the powers of a negative base on its magnitude.
The integer powers are computed from exact products within the `int`s, x^{-n} included, and are the tightest enclosures, or one double beyond where the power is closer to a double than twice a bound of their error proved beforehand, 2^{-98} for $n = 3$, 2^{-88} for $n = 1000$, 2^{-62} at most, relatively; they are exact where the power is a double. Beyond the `int`s, from the power of CORE-MATH at the bounds of I, the tightest enclosures where n is a double, and within three doubles of them otherwise. [GAOL v5] The real power takes the power of CORE-MATH at the corners of the box $I \times J$ where it is the least and the greatest: the upper bound is the tightest one, and so is the lower bound, which is the power itself where it is a double, as $4^{0.5}$, $9^{1.5}$ or 2^{-3}, and the double below CORE-MATH's value otherwise. At a corner where the base is 0 or $+\infty$ or the exponent infinite, the bound is the limit of x^y there, 0 or $+\infty$ (1 for a base 1 or an exponent 0): `pow([4, +∞], 0.5)` is $[2, +\infty]$. [GAOL v5]

Example

```
1 cout << pow(interval(-4,-1), 2) << ' '
2      << pow(interval(-4,-1), interval(2.0)) << ' '
3      << pow(interval(-4,9), interval(0.5)) << ' '
4      << pow(interval(-4,-1), interval(1.5, 2.5)) << endl;
5 🖶 [1, 16] [1, 16] [0, 3] [empty]
6 cout << pow(interval(0.0), interval(0.0)) << ' '
7      << pow(interval(-2,2), -2) << ' '
8      << pow(interval(4), 0.5) << endl;
9 🖶 [1] [0.25, inf] [2]
```

interval **nth_root** (const interval& *I*, unsigned int *n*)
interval **nth_root** (const interval& *I*, long long *q*)
interval **nth_root** (const interval& *I*, unsigned long long *q*)
interval **nth_root** (const interval& *I*, int *q*)

■ $\sqrt[n]{I}$

Computes the nth functional root of I: rootn of IEEE 1788-2015. For an odd n, the root of a negative number is the opposite of the root of its magnitude, and the root is defined on the whole real line; for an even n, only the part of I in $[0, +\infty]$ is considered. [GAOL v5] A negative q gives $1/x^{1/|q|}$, defined on $\mathbb{R} \setminus \{0\}$ for an odd q and on $]0, +\infty]$ for an even one. Returns the empty set for $n = 0$.
The bounds are the tightest ones, or up to 2 doubles beyond (4 for a negative q): each is proved with integer powers. The cube root is that of CORE-MATH, the tightest one, and the root of a double that is the nth power of a double is that double. An order of another integer type (`long`, `long long`...), with which `nth_root` was ambiguous, is taken as an `int` or an `unsigned int` where it is one; beyond them, the roots are enclosed by `pow` with the exponent $1/q$, as an interval, $x^{1/q}$ being within 2^{-21} of 1 for every double $x > 0$ there, after one check of the rounding direction for a `long long` or an `unsigned long long` order. [GAOL v5]

Example

```
1 cout << nth_root(interval(-8,27), 3) << ' '
2      << nth_root(interval(-4,16), 2) << ' '
3      << nth_root(interval(4,16), -2) << ' '
4      << nth_root(interval(-8,-1), -3);
5 [-2, 3] [0, 4] [0.25, 0.5] [-1, -0.5]
```

8.1.3 Exponentials and logarithms

The functions of this section, and of the next two, are bounded with CORE-MATH at the bounds of the interval, or at the points where the function reaches its extrema within it. CORE-MATH being correctly rounded, the value at a bound, computed rounding upward, is the upper bound, and the double below it the lower bound: the tightest bounds, unless the exact value is a double, which the functions give exactly where they know it happens ($\exp(0) = 1$, $\log(1) = 0$, $\sin(0) = 0$, $2^3 = 8$...). [GAOL v5]

interval **exp** (const interval& *I*)

Exponential of I.

interval **exp2** (const interval& *I*)
interval **exp10** (const interval& *I*)

2^x and 10^x for x in I: exp2 and exp10 of IEEE 1788-2015. [GAOL v5]

interval **expm1** (const interval& *I*)
interval **exp2m1** (const interval& *I*)
interval **exp10m1** (const interval& *I*)

$e^x - 1$, $2^x - 1$ and $10^x - 1$ for x in I, within $[-1, +\infty]$: expm1, exp2m1 and exp10m1 of IEEE 1788-2015 (Table 10.5). [GAOL v5]

interval **log** (const interval& *I*)

Natural logarithm of I, defined on $]0, +\infty]$: an interval holding no positive number gives the empty set, where GAOL 4 kept its part in $[0, +\infty]$ and gave $[-\infty, -\text{MAX}]$ for `log`([−4, 0]). [GAOL v5]

interval **log2** (const interval& *I*)
interval **log10** (const interval& *I*)

Logarithms in base 2 and 10 of I: log2 and log10 of IEEE 1788-2015. [GAOL v5]

interval **log1p** (const interval& *I*)
interval **log2p1** (const interval& *I*)
interval **log10p1** (const interval& *I*)

$\log(1+x)$, $\log_2(1+x)$ and $\log_{10}(1+x)$ for x in I, defined on $]-1,+\infty]$, $-\infty$ being their limit at -1: logp1, log2p1 and log10p1 of IEEE 1788-2015 (Table 10.5). [GAOL v5]

Example

```
1 cout << exp(interval(0.0)) << ' ' << log(interval(1)) << ' '
2      << log(interval(-4,0.0)) << ' '
3      << log(interval(-4,1)) << endl;
4 [1] [0] [empty] [-inf, 0]
5 cout << exp2(interval(3)) << ' ' << log2(interval(0.25)) << ' '
6      << log10(interval(0.0,1000)) << ' '
7      << expm1(interval(-1e300,0.0)) << ' '
8      << log1p(interval(-2,0.0)) << endl;
9 [8] [-2] [-inf, 3] [-1, 0] [-inf, 0]
```

8.1.4 Trigonometric functions

interval **cos** (const interval& *I*)

Returns the cosine of I, within $[-1, 1]$.

interval **sin** (const interval& *I*)

Returns the sine of I, within $[-1, 1]$.

interval **tan** (const interval& *I*)

Returns the tangent of I: $[-\infty, +\infty]$ when I holds a pole. No double is a pole: `M_PI_2`, the double below $\pi/2$, is $6.1 \cdot 10^{-17}$ short of it, and $[-$`M_PI_2`, `M_PI_2`$]$, whose width is that of the double below π and which holds no pole, has a finite tangent (fourth result of the example); the width of $[-$`M_PI_2`, `M_PI_2`$]$ is the double below π, and intervals of width between that and π are a theoretical case.
The three functions find the pieces of I on which they are monotonic by dividing its bounds by an enclosure of π and, where the quotients cannot tell, next to an extremum or a pole and at the large magnitudes, from the signs of the derivative at the bounds, which CORE-MATH gives exactly: their bounds are within one double of the tightest ones at every magnitude. [GAOL v5] GAOL 4 gave $[-1, 1]$ for `cos(`$[2^{60}, 2^{60}]$`)`, and computed the sine as a shifted cosine, which widened narrow intervals.

Example

```
cout << cos(interval(1,4)) << endl
     << sin(interval(1,2)) << endl
     << tan(interval(1,2)) << endl
     // 1.5707963267948966 is M_PI_2, the double below pi/2
     << tan(interval(-1.5707963267948966, 1.5707963267948966)) << endl
     // 1152921504606846976 is 2 to the 60
     << cos(interval(1152921504606846976.0)) << endl;
[-1, 0.5403023058681398]
[0.8414709848078965, 1]
[-inf, inf]
[-1.633123935319537e+16, 1.633123935319537e+16]
[-0.5567960822766418, -0.5567960822766416]
```

interval **acos** (const interval& *I*)

Returns the arccosine of I, on the part of I in $[-1, 1]$.

interval **asin** (const interval& *I*)

Returns the arcsine of I, on the part of I in $[-1, 1]$.

interval **atan** (const interval& *I*)

Returns the arctangent of I.

Example

```
cout << acos(interval(1,3)) << ' ' << asin(interval(1)) << ' '
     << atan(interval(1));
[0] [1.570796326794896, 1.570796326794897]
[0.7853981633974482, 0.7853981633974484]
```

interval **atan2** (const interval& *Y*, const interval& *X*)

Returns the hull of the angles in $(-\pi, \pi]$ of the points of ordinate in Y and abscissa in X, the point $(0, 0)$ left out, as the atan2 of IEEE 1788-2015: `atan2([0],[0])` is empty, and the result is $[-\pi, \pi]$ whenever the box holds points of the half-line $y = 0$, $x < 0$, where the angle is π, and points below it, where it is next to $-\pi$. GAOL 4 declared the function, but it raised `unavailable_feature_error`. [GAOL v5]

Example

```
interval x(1,GAOL_INFINITY);
cout << atan2(interval(0.0), interval(0.0)) << endl
     << atan2(interval(-1,1), interval(-1)) << endl
     << atan2(x, x) << endl;
[empty]
[-3.141592653589794, 3.141592653589794]
[0, 1.570796326794897]
```

interval **sinpi** (const interval& *I*)
interval **cospi** (const interval& *I*)
interval **tanpi** (const interval& *I*)

$\sin(\pi x)$, $\cos(\pi x)$ and $\tan(\pi x)$ for x in I: sinPi, cosPi and tanPi of IEEE 1788-2015 (Table 10.5). [GAOL v5] Their extrema and poles are at the multiples of $1/2$, which are doubles, and are found exactly, at every magnitude. `tanpi` has no value at the half-integers, which are left out: a half-integer alone gives the empty set.

interval **asinpi** (const interval& *I*)
interval **acospi** (const interval& *I*)
interval **atanpi** (const interval& *I*)
interval **atan2pi** (const interval& *Y*, const interval& *X*)

$\text{asin}(x)/\pi$, $\text{acos}(x)/\pi$, $\text{atan}(x)/\pi$ and $\text{atan2}(y, x)/\pi$: asinPi, acosPi, atanPi and atan2Pi of IEEE 1788-2015 (Table 10.5), within $[-1/2, 1/2]$, $[0, 1]$, $[-1/2, 1/2]$ and $[-1, 1]$. [GAOL v5]

Example

```
cout << sinpi(interval(1e17)) << ' '
     << sin(interval::pi() * 1e17) << endl;
[0] [-1, 1]
cout << tanpi(interval(0.5)) << ' '
     << tanpi(interval(0.25,0.75)) << ' '
     << asinpi(interval(1)) << ' '
     << atan2pi(interval(1), interval(-1)) << endl;
[empty] [-inf, inf] [0.5] [0.75]
```

8.1.5 Hyperbolic functions

interval **cosh** (const interval& *I*)

Returns the hyperbolic cosine of I.

interval **acosh** (const interval& *I*)

Returns the hyperbolic arccosine of I, on the part of I in $[1, +\infty]$.

interval **sinh** (const interval& *I*)

Returns the hyperbolic sine of I.

interval **asinh** (const interval& *I*)

Returns the hyperbolic arcsine of I.

interval **tanh** (const interval& *I*)

Returns the hyperbolic tangent of I, within $[-1, 1]$.

interval **atanh** (const interval& *I*)

Returns the hyperbolic arctangent of I, defined on $]-1, 1[$, $-\infty$ and $+\infty$ being its limits at -1 and 1: the part of I outside is left out, and an interval with no point of it gives the empty set, as `atanh([1, 5])` and `atanh([-1, -1])` do.
The six functions are those of CORE-MATH, where GAOL 4 took them from the mathematical library of the system, which is sometimes further than one double from the exact value. [GAOL v5]

Example

```
cout << cosh(interval(-1,2)) << ' '
     << tanh(interval(20,30)) << endl
     << acosh(interval(0,1)) << ' '
     << atanh(interval(-2,2)) << ' '
     << atanh(interval(1,5)) << endl;
[1, 3.762195691083632] [0.9999999999999998, 1]
[0] [-inf, inf] [empty]
```

8.1.6 Other functions

interval **hypot** (const interval& *X*, const interval& *Y*)

$\sqrt{x^2 + y^2}$ for x in X and y in Y: hypot of IEEE 1788-2015 (Table 10.5), computed at the points of the box nearest to the origin and farthest from it. [GAOL v5]

interval **rsqrt** (const interval& *I*)

$1/\sqrt{x}$ for x in I, defined on $]0, +\infty]$, $+\infty$ being its limit at 0: rSqrt of IEEE 1788-2015 (Table 10.5). [GAOL v5]

Example

```
cout << hypot(interval(3), interval(4)) << ' '
     << hypot(interval(-3,1), interval(4,5)) << endl
     << rsqrt(interval(4,16)) << ' '
     << rsqrt(interval(0.0,4)) << endl;
[5] [4, 5.830951894845301]
[0.25, 0.5] [0.5, inf]
```

interval **compoundm1** (const interval& *X*, const interval& *Y*)

$(1+x)^y - 1$ for x in X and y in Y: compoundm1 of IEEE 1788-2015 (Table 10.5). It keeps the digits of a small x that `pow(1+x, y) - 1` loses, $1+x$ being rounded and the difference cancelling what is left. It is defined for $x > -1$, and for $x = -1$ when $y > 0$, where it is -1; the part of the box outside is left out, and -1 and $+\infty$, the limits at $x = -1$, at $x = +\infty$ and for an infinite y, are bounds. CORE-MATH having no function for it, the value at a corner of the box is computed with integers: 64 bits in a fast phase that settles 71 to 95 % of the pairs, 128 bits and a bound of the error in the others. It takes 0.1 microsecond for each of the one or two corners where the bounds are, where the fast phase settles it, and 0.6 to 0.9 elsewhere. The bounds are the tightest ones, the double itself where the value is one (2.375 for $x = 1.5 - 1$ and $y = 3$, but also $2^m - 1$ for $|m| \leq 53$, and the quotient $-x/(1+x)$ of $y = -1$, divided exactly), but in a case out of 10^{17} or so, where a bound is a double beyond. [GAOL v5]

Example

```
cout << compoundm1(interval(0.5), interval(2)) << ' '
     << compoundm1(interval(1,3), interval(1,2)) << endl
     << compoundm1(interval(-1), interval(0,1)) << ' '
     << compoundm1(interval(-1,0), interval(-1)) << endl
     << compoundm1(interval(1e-20), interval(3)) << endl;
[1.25] [1, 15]
[-1] [0, inf]
[2.999999999999999e-20, 3.000000000000001e-20]
```

interval **erf** (const interval& *I*)
interval **erfc** (const interval& *I*)

The error function $\mathrm{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2}\, dt$, increasing from -1 to 1, and its complement $\mathrm{erfc}(x) = 1 - \mathrm{erf}(x)$, decreasing from 2 to 0, for x in I: the `erf` and `erfc` of C, which CORE-MATH computes, and the tightest bounds, exact at 0 and at the infinities. $\mathrm{erfc}(x)$ is subnormal from $x = 26.55$ on, and below the least subnormal from $x = 27.22$ on, where the lower bound is 0. IEEE 1788-2015 does not name them, and `gaol_ieee1788` does not hold them: a call `erf(x)` on an interval finds them all the same, by argument-dependent lookup. [GAOL v5]

Example

```
cout << erf(interval(0.0)) << ' ' << erf(interval(-1,1)) << ' '
     << erf(interval(6,7)) << endl
     << erfc(interval(0.0,1)) << ' ' << erfc(interval(28,30)) << endl;
[0] [-0.8427007929497149, 0.8427007929497149] [0.9999999999999998, 1]
[0.1572992070502851, 1] [0, 4.940656458412466e-324]
```

8.2 Relational Arithmetic

The relational functions of GAOL are the *reverse functions* of IEEE 1788-2015 (10.5.4, Table 10.1), with their arguments in another order: each computes the hull of the x of an interval I whose image by the function is in J. Their correspondence is given at the end of this section, and the namespace `gaol_ieee1788` provides them under their names (Chapter 11).

interval interval::**operator%** (double *d*) const
interval interval::**operator%** (const interval& *I*) const
interval **operator%** (double *d*, const interval& *I*)

- `self` $\% I = \square\{z \in \mathbb{R} \mid \exists x \in$ `self`$, \exists y \in I : x = yz\}$

Relational division: `div_rel(self, I,` $[-\infty, +\infty]$`)`.

Example

```
cout << interval(1,2) % interval(0,1) << ' '
     << interval(1,2) % interval(-1,1);
[1, inf] [-inf, inf]
```

8.2.1 $(n+1)$-ary relational functions

Consider the relation $y = \cos x$ where x and y are interval variables. One would like to be able to express this relation in the equivalent way: $x = acos\, y$. However, one cannot use the *acos* function because its result is always included into the interval $[0, \pi]$. What we need here is a relational version of the *acos* function. But, since for any value x there are infinitely many values y verifying $x = acos\, y$, we have to take into account the domain of y. As a consequence, we define a new binary operator `acos_rel` whose definition is as follows:

$$\texttt{acos_rel}(Y, X) = \square\{x \in X \mid \exists y \in Y : y = \cos x\}$$

This is to be contrasted with the previous definition of the *acos* function:

$$\texttt{acos}(Y) = \square\{x \in \mathbb{R} \mid \exists y \in Y : x = acos\, y\}$$

Figure 8.1 presents the different results obtained when computing either `acos`(J) or `acos_rel`(J, I). An interval I that meets no piece of the preimage gives the empty set. [GAOL v5]

interval **acos_rel** (const interval& J, const interval& I)

- `acos_rel`$(J, I) = \square\{x \in I \mid \exists y \in J : y = \cos x\}$

Returns the *relational arccosine* of J w.r.t. I.

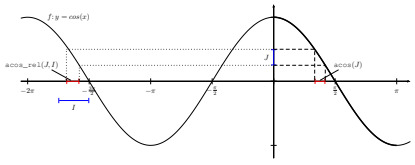


Figure 8.1: Relational cosine

interval **asin_rel** (const interval& J, const interval& I)

- `asin_rel`$(J, I) = \square\{x \in I \mid \exists y \in J : y = \sin x\}$

Returns the *relational arcsine* of J w.r.t. I.

interval **atan_rel** (const interval& J, const interval& I)

- `atan_rel`$(J, I) = \square\{x \in I \mid \exists y \in J : y = \tan x\}$

Returns the *relational arctangent* of J w.r.t. I.
The three functions enclose the multiples of π of the pieces of the preimage in double-double arithmetic, and add the inverse function of J before rounding the sum: their bounds are within one double of the accurate ones. [GAOL v5]

Example

```
cout << asin_rel(interval(0.0), interval(1,7)) << endl
     << atan_rel(interval(1), interval(0,5)) << endl;
[3.141592653589793, 6.283185307179588]
[0.7853981633974482, 3.926990816987242]
```

interval **acosh_rel** (const interval& J, const interval& I)

- `acosh_rel`$(J, I) = \square\{x \in I \mid \exists y \in J : y = \cosh x\}$

Returns the *relational hyperbolic arccosine* of J w.r.t. I.

interval **asinh_rel** (const interval& J, const interval& I)
interval **atanh_rel** (const interval& J, const interval& I)

- `asinh_rel`$(J, I) = \square\{x \in I \mid \exists y \in J : y = \sinh x\}$
- `atanh_rel`$(J, I) = \square\{x \in I \mid \exists y \in J : y = \tanh x\}$

Return the *relational hyperbolic arcsine* and *arctangent* of J w.r.t. I, that is, $\operatorname{asinh}(J) \cap I$ and $\operatorname{atanh}(J) \cap I$, sinh and tanh being one-to-one. As tanh takes its values in $]-1, 1[$, `atanh_rel` gives the empty set for a J with no point of it, as $[1, 1]$.

Example

```
cout << acosh_rel(interval(1,2), interval::universe()) << endl
     << asinh_rel(interval(0,1), interval::universe()) << endl
     << atanh_rel(interval(1), interval::universe()) << endl;
[-1.316957896924817, 1.316957896924817]
[0, 0.8813735870195431]
[empty]
```

interval **sqrt_rel** (const interval& J, const interval& I)

- `sqrt_rel`$(J, I) = \square\{x \in I \mid \exists y \in J : y = x^2\}$

Returns the *relational square root* of J w.r.t. I.

Example

```
cout << sqrt_rel(interval(1,4), interval::universe()) << ' '
     << sqrt_rel(interval(1,4), interval(0,10));
[-2, 2] [1, 2]
```

interval **nth_root_rel** (const interval& *J*, unsigned int *n*,
const interval& *I*)
interval **nth_root_rel** (const interval& *J*, int *p*,
const interval& *I*)

- `nth_root_rel`$(J, n, I) = \square\{x \in I \mid \exists y \in J : y = x^n\}$
- `nth_root_rel`$(J, p, I) = \square\{x \in I \mid x^p \text{ is defined and in } J\}$

Returns the *relational inverse n-th power* of J w.r.t. I, for $n \geqslant 1$. It takes the roots of `nth_root()`. [GAOL v5] For an `int` p, or an integer of another type, it is the reverse of x^p for every integer p, the pownRev of the standard. x^0 is 1 for every x, 0 included, so that the result is I where $1 \in J$, and the empty set elsewhere. For $p < 0$, x^p is $1/x^{|p|}$, which 0 has not: J is cut at 0, each part giving its roots (the hull of the roots of J would keep the part of I between them), and 0, which the roots reach where J is unbounded, is no solution. The bounds are those of `nth_root(J, p)`, the root and then its inverse: up to four doubles beyond the tightest ones for $p < 0$. An exponent of type `double` or `float` is refused at compile time: it was converted to an `unsigned int`, which cut 2.5 to 2.

Example

```
interval J(1,16);
cout << nth_root_rel(J, 4, interval::universe()) << ' '
     << nth_root_rel(J, 4, interval(-10,0.0));
[-2, 2] [-2, -1]
```

Example

```
cout << nth_root_rel(interval(1,8), -3, interval::universe()) << ' '
     << nth_root_rel(interval(-1,2), -3, interval(-0.5,0.5)) << ' '
     << nth_root_rel(interval(1,GAOL_INFINITY), -2, interval(0,3)) << ' '
     << nth_root_rel(interval(1,2), 0, interval(-5,5));
[0.5, 1] [empty] [0, 1] [-5, 5]
```

interval **invabs_rel** (const interval& J, const interval& I)

■ `invabs_rel`$(J, I) = \square\{x \in I \mid |x| \in J\}$

Returns the *relational inverse absolute value* of J w.r.t. I. The negative part of J holds no magnitude: GAOL 4 took it as magnitudes, and gave $[-2, 2]$ where the result is empty. [GAOL v5]

Example

```
1 cout << invabs_rel(interval(1,2), interval::universe()) << ' '
2      << invabs_rel(interval(-2,-1), interval(-10,10));
3 🖶 [-2, 2] [empty]
```

interval **div_rel** (const interval& K, const interval& J, const interval& I)

■ `div_rel`$(K, J, I) = \square\{x \in I \mid \exists z \in K, \exists y \in J : z = xy\}$

Returns the *ternary relational division* of K by J w.r.t. I, which is empty when no x of I is in the relation. [GAOL v5]

Example

```
1 cout << div_rel(interval(1,2), interval(-1,1),
2                 interval(0,10)) << ' '
3      << div_rel(interval(5,6), interval(1,2),
4                 interval(10,20));
5 🖶 [1, 10] [empty]
```

interval **pow_rel** (const interval& K, const interval& J, const interval& I)

■ `pow_rel`$(K, J, I) = \square\{x \in I \mid \exists z \in K, \exists y \in J : z = x^y\}$

Returns the *relational base* of the power $z = x^y$: the hull of the x of I with an x^y in K for a y of J, the arguments in the order of `div_rel()`. [GAOL v5] x^y is the `pow` of IEEE 1788-2015 (Table 9.1), that of `gaol_ieee1788::pow`: $e^{y \log x}$ for $x > 0$, 0 for $x = 0$ and $y > 0$, and no value elsewhere, so that no $x < 0$ is a solution, in either namespace. The `pow` of `gaol` takes the integer power for a degenerate integer exponent $[n]$, which has negative bases too: the reverse of x^n is `nth_root_rel(K, n, I)`.

Example

```
1 cout << pow_rel(interval(4,9), interval(2), interval::universe()) << ' '
2      << pow_rel(interval(2,3), interval(1,GAOL_INFINITY), interval(0,1)) << ' '
3      << pow_rel(interval(1,4), interval(2), interval(-3,3));
4 🖶 [2, 3] [empty] [1, 2]
```

$x^y \geqslant 2$ for a $y \geqslant 1$ asks $x > 1$: the second result is empty.

interval **pow_exponent_rel** (const interval& K, const interval& H, const interval& I)

■ pow_exponent_rel$(K, H, I) = \square\{y \in I \mid \exists z \in K, \exists x \in H : z = x^y\}$

Returns the *relational exponent* of the power $z = x^y$, the same `pow` as `pow_rel()`: the hull of the y of I with an x^y in K for an x of H. [GAOL v5]

Example

```
cout << pow_exponent_rel(interval(8), interval(2), interval::universe()) << ' '
     << pow_exponent_rel(interval(3), interval(2,4), interval::universe());
[3] [0.792481250360578, 1.584962500721157]
```

The bounds of both are the tightest ones: the ends of the solutions, $c^{1/y}$ and $\log c / \log x$ for bounds c, x and y of the arguments, are bounded by the largest and the smallest doubles d whose powers d^y, or x^d, CORE-MATH's `pow`, correctly rounded, proves to be on either side of c. A bound of I where the solutions start between it and the next double is kept when its power is within one double of c, which only knowing whether that power is a double would settle: `pow_rel`$([1+2^{-52}, 1+2^{-52}]$, $[0.5, 0.5]$, $[0, 1+2^{-51}])$ is $[1+2^{-51}, 1+2^{-51}]$, where $(1+2^{-52})^2 = 1+2^{-51}+2^{-104}$: the tightest result for I one double wider.

interval **max_rel** (const interval& Z, const interval& Y, const interval& X)
interval **min_rel** (const interval& Z, const interval& Y, const interval& X)

■ max_rel$(Z, Y, X) = \square\{x \in X \mid \exists y \in Y : \max(x, y) \in Z\}$
■ min_rel$(Z, Y, X) = \square\{x \in X \mid \exists y \in Y : \min(x, y) \in Z\}$

Return the *relational maximum* and *minimum* of Z given Y w.r.t. X, the arguments in the order of `div_rel`: the result, the other operand, then the operand contracted; `max_rel(Z, X, Y)` contracts Y, and `max(X, Y) & Z` contracts Z. $\max(x, y)$ is in Z when x and y are at most $\overline{Z}$ and one of them at least $\underline{Z}$: the set is empty when Y lies above Z, $X \cap [-\infty, \overline{Z}]$ when Y meets Z, and $X \cap Z$ when Y lies below Z, the maximum being x; `min_rel` is its mirror image. The bounds are those of X and Z: each result is the tightest enclosure. [GAOL v5]

Example

```
cout << max_rel(interval(1,2), interval(3,4), interval(-5,5)) << ' '
     << max_rel(interval(1,2), interval(0,1), interval(-5,5)) << ' '
     << max_rel(interval(1,2), interval(-1,0), interval(-5,5)) << endl
     << min_rel(interval(1,2), interval(2,3), interval(-5,5)) << endl;
[empty] [-5, 2] [1, 2]
[1, 5]
```

interval **sign_rel** (const interval& Z, const interval& X)

■ sign_rel$(Z, X) = \square\{x \in X \mid \text{sign}(x) \in Z\}$

Returns the *relational inverse sign* of Z w.r.t. X: the negative x of X where Z holds -1, 0 where Z and X hold it, and the positive x of X where Z holds 1; the empty set where Z holds none of them, as $[0.5, 0.7]$. The negative x go up to 0, which their tightest enclosure takes although $\text{sign}(0) = 0$, the negative doubles next to 0 being in the set. [GAOL v5]

Example

```
cout << sign_rel(interval(-1), interval(-2,3)) << ' '
     << sign_rel(interval(-1), interval(0,3)) << ' '
     << sign_rel(interval(0,1), interval(-2,3)) << endl;
[-2, 0] [empty] [0, 3]
```

interval **floor_rel** (const interval& Z, const interval& X)

■ `floor_rel`$(Z, X) = \square\{x \in X \mid \lfloor x \rfloor \in Z\}$

Returns the *relational inverse floor* of Z w.r.t. X: the part of X in $\left[\lceil \underline{Z} \rceil, \lfloor \overline{Z} \rfloor + 1\right[$, the x whose floor is an integer of Z, and the empty set when Z holds no integer. That interval is open at its upper end, which its tightest closed enclosure takes all the same, the reals just below it being in the set: `floor_rel(interval(0), interval(1, 2))` is empty, X starting at that end. Beyond 2^{53}, where every double is an integer, $\lfloor \overline{Z} \rfloor + 1$ is no double, and the bound is the double above it: `floor_rel` of $[2^{53}, 2^{53}]$ ends at $2^{53} + 2$, and at $+\infty$ for [DBL_MAX, DBL_MAX], where X goes on to it. Each result is the tightest enclosure. [GAOL v5]

Example

```
1 cout << floor_rel(interval(0,2), interval(-5,5)) << ' '
2      << floor_rel(interval(0), interval(1,2)) << ' '
3      << floor_rel(interval(0.5,0.7), interval(-5,5)) << endl;
4 🖶 [0, 3] [empty] [empty]
```

interval **atan2_rel_y** (const interval& *C*, const interval& *X*,
const interval& *Y*)
interval **atan2_rel_x** (const interval& *C*, const interval& *Y*,
const interval& *X*)

■ `atan2_rel_y`$(C, X, Y) = \square\{y \in Y \mid \exists x \in X\colon \operatorname{atan2}(y, x) \in C\}$
■ `atan2_rel_x`$(C, Y, X) = \square\{x \in X \mid \exists y \in Y\colon \operatorname{atan2}(y, x) \in C\}$

Return the *relational* `atan2` with respect to its first and to its second argument: the hulls of the ordinates and of the abscissas of the points of the box $X \times Y$ whose angle is in C, the point $(0, 0)$, which has no angle, left out. [GAOL v5] As in `div_rel`, the interval contracted comes last, and the other argument of `atan2` before it. The angles are those of `atan2`, in $]-\pi, \pi]$: the half-line $y = 0$, $x < 0$ has the angle π, and a C holding $-\pi$ but not π leaves it out; the angles of C beyond π are no angle of any point. The bounds are within two doubles of the tightest ones.

Example

```
1 interval c(1,2), x(-1,1), y(0,1);
2 cout << atan2_rel_y(c, x, y) << ' '
3      << atan2_rel_x(c, y, x) << endl
4      << atan2_rel_x(interval(-4,-3), interval(0.0), x)
5      << endl;
6 🖶 [0, 1] [-0.4576575543602859, 0.6420926159343309]
7 🖶 [empty]
```

The correspondence with the reverse functions of IEEE 1788-2015, whose last argument x is $[-\infty, +\infty]$ when it is left out:

IEEE 1788-2015	GAOL
sqrRev(c, x)	`sqrt_rel(c, x)`
absRev(c, x)	`invabs_rel(c, x)`
pownRev(c, x, p)	`nth_root_rel(c, p, x)`
sinRev, cosRev, tanRev(c, x)	`asin_rel`, `acos_rel`, `atan_rel(c, x)`
coshRev(c, x)	`acosh_rel(c, x)`
mulRev(b, c, x)	`div_rel(c, b, x)`
powRev1(b, c, x)	`pow_rel(c, b, x)` [GAOL v5]
powRev2(a, c, x)	`pow_exponent_rel(c, a, x)` [GAOL v5]
atan2Rev1(b, c, x)	`atan2_rel_y(c, b, x)` [GAOL v5]
atan2Rev2(a, c, x)	`atan2_rel_x(c, a, x)` [GAOL v5]

mulRevToPair(b, c), the two parts of `div_rel(c, b)`, is in the namespace `gaol_ieee1788` under its name (Chapter 11). [GAOL v5] pownRev takes every integer p, $p \leqslant 0$ included. `max_rel`, `min_rel`, `sign_rel` and `floor_rel` are the reverse functions of max, min, sign and floor as the standard defines them (10.5.4), which it neither names nor requires, Table 10.1 not listing them: the namespace `gaol_ieee1788` has no name for them. [GAOL v5]

8.3 Cancellative addition and subtraction

interval **cancel_minus** (const interval& *X*, const interval& *Y*)

The tightest interval Z such that $Y + Z$ contains X, $[\underline{X} - \underline{Y}, \overline{X} - \overline{Y}]$, which exists when X and Y are bounded and X is at least as wide as Y; $[-\infty, +\infty]$ otherwise, and the empty set for an empty X and a bounded Y: cancelMinus of IEEE 1788-2015 (10.5.6). Whether X is at least as wide as Y is decided exactly, which rounding the differences of the bounds cannot do. [GAOL v5]

interval **cancel_plus** (const interval& *X*, const interval& *Y*)

`cancel_minus(X, -Y)`: cancelPlus of IEEE 1788-2015. [GAOL v5]

Example

```
1 cout << cancel_minus(interval(1,5), interval(0,2)) << ' '
2      << cancel_minus(interval(1,2), interval(0,3)) << ' '
3      << cancel_plus(interval(1,5), interval(0,2));
4 [1, 3] [-inf, inf] [3, 5]
```

9 Interval functions

The numeric functions of IEEE 1788-2015 (Table 10.2, 12.12.8) are those of this chapter; the namespace `gaol_ieee1788` names them as the standard does, and returns $+\infty$ and $-\infty$ for the bounds of the empty set, where `left()` and `right()` return NaN (Chapter 11).

double interval::**rad** (void) const

- $\texttt{rad}([a,\,b]) = \min\{r \mid [m-r, m+r] \supseteq [a,\,b]\}$

Returns the radius of `self`, the smallest double r such that $[m-r, m+r]$ contains `self`, m being `self.midpoint()`: $+\infty$ for an unbounded interval, NaN for the empty set (rad of IEEE 1788-2015). [GAOL v5]

void interval::**mid_rad** (double& *m*, double& *r*) const

Sets m to `self.midpoint()` and r to `self.rad()` (midRad of IEEE 1788-2015). [GAOL v5]

Example

```
1 double m, r;
2 interval(1,4).mid_rad(m, r);
3 cout << m << ' ' << r << ' ' << interval(1,GAOL_INFINITY).rad();
4 🖶 2.5 1.5 inf
```

double interval::**width** (void) const

- $\texttt{width}([a,\,b]) = \uparrow b - a \uparrow$

Returns the width of `self`: wid of IEEE 1788-2015. Returns NaN whenever the interval is empty, as the standard has it, where GAOL 4 returned -1. [GAOL v5]

Example

```
1 cout << boolalpha << interval(4,6).width() << ' '
2      << (interval(1,next_float(1)).width()
3          == std::numeric_limits<double>::epsilon()) << ' '
4      << interval::emptyset().width();
5 🖶 2 true nan
```

interval interval::**width_enclosure** (void) const

■ `width_enclosure`$([a, b]) = [\downarrow b - a \downarrow, \uparrow b - a \uparrow]$

Returns an interval enclosing the exact width of `self`, of which `width()` is the upper bound only: `interval(b) - interval(a)`, the point $[0, 0]$ for a point interval and the empty set for the empty set. The width of an unbounded interval, $+\infty$, being no real number, the result is then $[\text{MAX}, +\infty]$, MAX being the largest double, as `textToInterval("inf")` is. [GAOL v5]

Example

```
cout << interval(1,3).width_enclosure() << ' '
     << interval(-1,1e16).width_enclosure() << endl
     << interval(1,GAOL_INFINITY).width_enclosure();
[2] [1e+16, 1.000000000000001e+16]
[1.797693134862315e+308, inf]
```

interval interval::**inflate** (double *r*) const

■ `inflate`$([a, b], r) = [\downarrow a - r \downarrow, \uparrow b + r \uparrow]$

Returns `self` widened by the absolute radius r, `self + interval(-r, r)`, which encloses every $x + e$ for x in `self` and $|e| \leqslant r$, however small r is next to the bounds. $r = 0$ gives `self` and $r = +\infty$ the whole line; the empty set stays empty, and a negative or NaN r gives the empty set. [GAOL v5]

Example

```
cout << boolalpha << interval(1,2).inflate(0.5) << ' '
     << interval(1,2).inflate(1e-20).set_eq(interval(1,2));
[0.5, 2.5] false
```

[6] Eldon Robert Hansen. *Global Optimization Using Interval Analysis*. Pure and Applied Mathematics. Marcel Dekker Inc., 1992.

double interval::**mig** (void) const

Returns the *mignitude* of `self`: mig of IEEE 1788-2015. See the book by Hansen [6, chap. 3]. The mignitude of an interval $[a, b]$ is the smallest absolute value of the numbers in the interval, that is: 0 if the interval straddles 0, a if the interval is strictly positive, and $-b$ otherwise.

Note. The mignitude of the empty interval is a NaN.

Example

```
cout << interval(4,5).mig() << ' '
     << interval(-6,-3).mig() << ' '
     << interval(-3,8).mig();
4 3 0
```

[14] Volker Stahl. *Interval Methods for Bounding the Range of Polynomials and Solving Systems of Nonlinear Equations*. Phd. thesis, Johannes Kepler Universität, Linz, September 1995.

double interval::**smig** (void) const

Returns the *signed mignitude* of `self`. See Stahl's thesis [14, def. 1.3.28]. The signed mignitude of an interval $[a, b]$ is 0 if the interval straddles 0, a if the interval is strictly positive, and b otherwise.

Note. The signed mignitude of the empty interval is a NaN.

Example

```
cout << interval(4,5).smig() << ' '
     << interval(-6,-3).smig() << ' '
     << interval(-3,8).smig();
4 -3 0
```

double interval::**mag** (void) const

Returns the *magnitude* of `self`: mag of IEEE 1788-2015. The magnitude of an interval $[a, b]$ is the greatest absolute value of the numbers in the interval.

Note. The magnitude of the empty interval is a NaN.

Example

```
cout << interval(4,5).mag() << ' '
     << interval(-6,-3).mag() << ' '
     << interval(-10,5).mag();
5 6 10
```

double **hausdorff** (const interval& *I1*, const interval& *I2*)

Returns the Hausdorff distance between the two sets defined by intervals *I1* and *I2*, that is:

$$\texttt{hausdorff}(I_1, I_2) = \max(|\underline{I_1} - \underline{I_2}|, |\overline{I_1} - \overline{I_2}|)$$

rounded upward: the tightest upper bound of the distance. [GAOL v5]
Two equal bounds, infinite ones included, are at distance 0: the distance of $[1, +\infty]$ to itself is 0, and to $[2, +\infty]$ is 1 (GAOL 4 returned $+\infty$ for both). It is $+\infty$ when a bound is infinite in one of the two intervals only, and a NaN when one of them is empty. [GAOL v5]

Example

```
cout << hausdorff(interval(4,8),interval(5,10)) << ' '
     << hausdorff(interval(1,GAOL_INFINITY),interval(2,GAOL_INFINITY)) << ' '
     << hausdorff(interval(1,GAOL_INFINITY),interval(1,2));
2 1 inf
```

double interval::**midpoint** (void) const

Returns the midpoint of `self`, mid of IEEE 1788-2015. Given a and b two finite floating-point numbers, we have the following cases, MAX being the largest positive double, `std::numeric_limits<double>::max()`:

$$\begin{cases} \texttt{midpoint}(\varnothing) & = \text{NaN} \\ \texttt{midpoint}([-\infty, +\infty]) & = 0 \\ \texttt{midpoint}([-\infty, b]) & = -\text{MAX} \\ \texttt{midpoint}([a, +\infty]) & = \text{MAX} \\ \texttt{midpoint}([a, b]) & = \updownarrow (a+b)/2 \updownarrow \end{cases}$$

The midpoint of $[a, b]$ is rounded to nearest, ties to even: it is the half of $a + b$, or the sum of the halves of a and b where $a + b$ may overflow [5], that is where $|a|$ or $|b|$ is 2^{1023} or more, which is tested first: `midpoint()` of [MAX, MAX] raises no overflow exception. [GAOL v5]

[5] Frédéric Goualard. How do you compute the midpoint of an interval? *ACM Transactions on Mathematical Software*, 40(2), 2014.

interval interval::**mid** (void) const

Returns the tightest interval enclosing the midpoint of `self`, which is included in `self`. With the same notations as for `midpoint()`, we have the cases:

$$\begin{cases} \texttt{mid}(\varnothing) & = \varnothing \\ \texttt{mid}([-\infty, +\infty]) & = [0, 0] \\ \texttt{mid}([-\infty, b]) & = [-\text{MAX}, -\text{MAX}] \\ \texttt{mid}([a, +\infty]) & = [\text{MAX}, \text{MAX}] \\ \texttt{mid}([a, b]) & = [\downarrow (a+b)/2 \downarrow, \uparrow (a+b)/2 \uparrow] \end{cases}$$

The midpoint is computed as `midpoint()` computes it, with the rounding outward. GAOL 4 did not enclose the midpoint of some intervals whose bounds are subnormal numbers. [GAOL v5]

double interval::**left** (void) const

- `left`$([x, y]) = x$

Returns the left bound of `self`: NaN for the empty set, whose bounds are NaN. `gaol_ieee1788::inf()` returns $+\infty$ there, as IEEE 1788-2015 has it.

double interval::**right** (void) const

- `right`$([x, y]) = y$

Returns the right bound of `self`: NaN for the empty set. `gaol_ieee1788::sup()` returns $-\infty$ there.

interval **abs** (const interval& *I*)

- $\text{abs}(I) = \{|x| \in \mathbb{R}^+ \mid x \in I\}$

Returns the absolute value of *I*.

Example

```
1 cout << abs(interval(-5,6)) << ' '
2      << abs(interval(-4,-2));
3 [0, 6] [2, 4]
```

[12] Helmut Ratschek and Jon Rokne. Interval methods. In *Handbook of Global Optimization*, pages 751–828. Kluwer Academic, 1995.

double **chi** (const interval &*I*)

This function, introduced by Ratscheck and Rokne [12], characterizes the degree of symmetry of intervals. Its definition is as follows:

$$\text{For} I = [a, b], \quad \text{chi}(I) = \begin{cases} -1 & \text{if } I = 0 \\ a/b & \text{if } |a| \leqslant |b| \\ b/a & \text{otherwise} \end{cases}$$

It is NaN for the empty set, 1 for $[-\infty, +\infty]$, and 0 for the other intervals with an infinite bound.

Example

```
1 cout << chi(interval(3,6)) << ' '
2      << chi(interval(-6,3)) << ' '
3      << chi(interval::emptyset()) << ' '
4      << chi(interval::universe()) << ' '
5      << chi(interval(-5,GAOL_INFINITY));
6 0.5 -0.5 nan 1 0
```

interval **min** (const interval &I, const interval &J)

- $\min([a,\ b],[c,\ d]) = [\min(a,c),\ \min(b,d)]$

Returns the minimum of two intervals.

Example

```
cout << min(interval(5,6),interval(3,9)) << ' '
     << min(interval::emptyset(),interval(3,8));
[3, 6] [empty]
```

interval **max** (const interval &*I*, const interval &*J*)

- $\max([a,\ b],[c,\ d]) = [\max(a,c),\ \max(b,d)]$

Returns the maximum of two intervals.

Example

```
cout << max(interval(5,6),interval(3,9)) << ' '
     << max(interval::emptyset(),interval(3,8));
[5, 9] [empty]
```

9.1 Integer functions

The functions of this section are those of IEEE 1788-2015 (Table 9.1): each is non-decreasing, so that the bounds of the result are its values at the bounds of the interval, and each is exact. None of them depends on the rounding direction.

interval **floor** (const interval &*I*)

- [`floor`(I.`left`()), `floor`(I.`right`())]

Example

```
cout << floor(interval(4.5,6.5)) << ' '
     << floor(textToInterval("[-10.4,3.5]"));
[4, 6] [-11, 3]
```

interval **ceil** (const interval &*I*)

- [`ceil`(I.`left`()), `ceil`(I.`right`())]

Example

```
cout << ceil(interval(4.5,6.5)) << ' '
     << ceil(textToInterval("[-10.4,3.5]"));
[5, 7] [-10, 4]
```

interval **integer** (const interval &*I*)

- [`ceil`(I.`left`()), `floor`(I.`right`())]

Narrows down the bounds to the closest integers: the empty set when I holds no integer. Note that the resulting bounds are still `double` numbers, and may therefore not be representable with integral types.

Example

```
cout << integer(interval(4.5,6.5)) << ' '
     << integer(interval(4.2,4.8));
[5, 6] [empty]
```

interval **sign** (const interval &*I*)

The signs of the elements of I: -1 below 0, 0 at 0, 1 above (sign). [GAOL v5]

interval **trunc** (const interval &*I*)

The elements of I rounded toward zero (trunc). [GAOL v5]

interval **round_ties_to_even** (const interval &*I*)
interval **round_ties_to_away** (const interval &*I*)

The elements of I rounded to the nearest integer, halfway values to the even one, respectively away from zero (roundTiesToEven, roundTiesToAway). [GAOL v5]

Example

```
cout << sign(interval(-2,3)) << ' '
     << trunc(interval(-2.5,3.7)) << ' '
     << round_ties_to_even(interval(0.5,2.5)) << ' '
     << round_ties_to_away(interval(0.5,2.5));
[-1, 1] [-2, 3] [0, 2] [1, 3]
```

9.2 Splitting methods

void interval::**split** (interval& *I1*, interval& *I2*) const

Splits `self` into two parts using `midpoint()`; returns the left part in *I1* and the right part in *I2*.
I1 or *I2* may be equal to `self`.

Example

```
interval I1a, I2a,
         I3(1.0,next_float(1.0)),
         I1b, I2b;
interval(4,5).split(I1a,I2a);
I3.split(I1b,I2b);
cout << boolalpha << I1a << ' ' << I2a << ' '
     << I1b.set_eq(interval(1.0)) << ' ' << I2b.set_eq(I3) << endl;
[4, 4.5] [4.5, 5] true true
```

interval interval::**split_left** (void) const

- `split_left`$([a, b]) = [a, m]$

Splits `self` into two parts using m = `midpoint()` and returns the left part.

Example

```
cout << interval(4,5).split_left();
[4, 4.5]
```

interval interval::**split_right** (void) const

■ split_right($[a, b]$) = $[m, b]$

Splits `self` into two parts using m = `midpoint()` and returns the right part.

Example

```
cout << interval(4,5).split_right();
🖶 [4.5, 5]
```

std::pair<interval, interval> interval::**bisect** (double *ratio*) const

■ bisect($[a, b], \rho$) = $([a, c], [c, b])$

Returns the two parts of `self` on either side of the cut point c, a double of $[a, b]$ that both parts share, so that they cover `self` exactly. For $0 < \rho < 1$, c is $a + \rho(b - a)$, computed as $a(1 - \rho) + b\rho$, which cannot overflow, and kept within $[a, b]$; when `is_bisectable()` is true, c is moreover strictly between a and b, so that both parts are smaller than `self`. $\rho = 0.5$ cuts at `midpoint()`, as `split()` does, and so does every ratio for an unbounded interval: a half-line is cut at the largest double or at its opposite, the whole line at 0. The empty set gives two empty sets, and a point interval itself twice. A ratio outside $(0, 1)$, or NaN, throws `std::domain_error`. [GAOL v5]

Example

```
std::pair<interval, interval> p = interval(0,4).bisect(0.25);
cout << p.first << ' ' << p.second;
🖶 [0, 1] [1, 4]
```

bool interval::**is_bisectable** (void) const

Returns `true` when a double lies strictly between the bounds of `self`, so that `bisect()` gives two parts, each smaller than `self`; `false` for the empty set, a point interval, two consecutive doubles, and the half-lines [MAX, $+\infty$] and $[-\infty, -\text{MAX}]$, which `split()` cuts at their finite bound. [GAOL v5]

Example

```
cout << boolalpha << interval(1,2).is_bisectable() << ' '
     << interval(1,next_float(1)).is_bisectable();
🖶 true false
```

9.3 Union and intersection

interval **operator&** (const interval& I1, const interval& I2)

■ $I1 \cap I2$

Returns the interval resulting from the intersection of *I1* and *I2*: intersection of IEEE 1788-2015. The intersection of disjoint intervals is the empty set `interval::emptyset()`, whose bounds are NaN. GAOL 4 kept the greatest left bound and the smallest right bound, [3, 2] for $[1, 2] \cap [3, 4]$, which the operations computing on the bounds did not take for empty. [GAOL v5]

Example

```
cout << (interval(4,6) & interval(5,9)) << ' '
     << (interval(1,2) & interval(3,4));
🖶 [5, 6] [empty]
```

interval **operator|** (const interval& I1, const interval& I2)

■ □($I1 \cup I2$)

Returns the hull of the union of *I1* and *I2*: convexHull of IEEE 1788-2015.

Example

```
1 cout << (interval(3,6) | interval(9,12));
2 🖶 [3, 12]
```

interval **hull** (const interval& I1, const interval& I2)
interval **intersect** (const interval& I1, const interval& I2)

`I1 | I2` and `I1 & I2` under a name, for an algorithm written with names rather than operators, and for `std::accumulate` or a pointer to a function. They are in the namespace `gaol` too. [GAOL v5]

Example

```
1 cout << hull(interval(1,2), interval(4,5)) << ' '
2      << intersect(interval(1,3), interval(2,5));
3 🖶 [1, 5] [2, 3]
```

10 Input/output

10.1 Reading intervals

istream& **operator**>> (istream& *in*, interval& I)

Reads a line from the input stream *in*, and assigns to I the interval it writes. The blanks before the line, the ends of lines included, are skipped first, as before a number, even under `std::noskipws`, which keeps a number from skipping them: an empty line, or one holding only blanks, is never a line to read. An interval cannot be written on several lines. If the line is syntactically ill-formed, I is set to the empty set, and a `std::invalid_argument` exception is thrown (see Section 17.1, page 114) if the library was compiled with exceptions enabled (see Section 2.5, page 9); alternatively, it prints an error message to `cerr` and aborts if exceptions were disabled. An ill-formed line also sets the `failbit` of *in*, which `in.clear()` resets before reading on; a function called with an argument it does not take throws `std::domain_error` instead. Where no line is left, *in* gets its `failbit`, I is left as it was, and nothing is thrown, as for a number: `while (in >> x)` reads the lines one after the other, the blank ones skipped, and ends with the input, a file ending with an empty line included. [GAOL v5] GAOL 4 read the empty text left, and threw `input_format_error`, as it did on an empty line, so that `in >> d >> x` over the lines `1.5` and `[1, 2]` threw at the end of the first one.

Example

```
1 std::istringstream in("[1, 2]\n[0.1, 0.2]\n");
2 interval a, b;
3 in >> a >> b;
4 cout << a << ' ' << b;
5 🖶 [1, 2] [0.09999999999999999, 0.20000000000000001]
```

Example

```
1 std::istringstream in("1.5\n\n[1, 2]\n\n");
2 double d;
3 interval x;
4 in >> d >> x;
5 cout << d << ' ' << x << ' ';
6 cout << (in >> x ? "more" : "end");
7 🖶 1.5 [1, 2] end
```

10.1.1 Input format

A string to be translated into an interval must have the following syntax (with terminals in lower case and non-terminals in slanted upper case):

```
EXPR
        : NUMBER
        | UNCERTAIN_NUMBER          // An interval literal, never a bound
        | LITERAL
        | dmin          // Smallest positive normal floating-point number
        | dmax                  // Largest positive floating-point number
        | pi
        | inf                       // Floating-point positive ‘‘infinity’’
        | infinity                                          // The same
        | empty                                       // Empty interval
        | EXPR + EXPR
        | EXPR - EXPR
        | EXPR * EXPR
        | EXPR / EXPR
        | - EXPR
        | + EXPR
        | FUNCTION_CALL
        | ( EXPR )
        ;

LITERAL
        : [ EXPR ]
        | [ EXPR , EXPR ]
        | [ EXPR , ]                                      // Right bound +∞
        | [ , EXPR ]                                       // Left bound -∞
        | [ , ]                                         // Entire real line
        | [ entire ]                                    // Entire real line
        | [ ]                                             // Empty interval
        | < EXPR , EXPR >
        ;

FUNCTION_CALL
        : NAME ( EXPR )
        | NAME ( EXPR , EXPR )
        | NAME ( EXPR , EXPR , EXPR )
        ;
```

Every string is an expression, computed with interval arithmetic: the literals of intervals, the numbers, the constants and the functions combine with the operators in any order, and a literal stands wherever a number may, in a bound too. [GAOL v5] `1+[1,2]`, `2*cos([0,1])` and `[cos([0,1]), 2]` are read. GAOL 4 read a string with two grammars, one for the intervals between brackets and one for the numbers, and refused `1+[1,2]` and `[1,2]+1+[1,2]` while it read `[1,2]*2`. `[empty]` is the form `[`*EXPR*`]` of the expression `empty`.

Spaces, tabs and newlines are not significant except in numbers, and the case of letters is ignored, as IEEE 1788-2015 has it for its literals: `[Empty]`, `[1, Inf]`, `SIN(1)` and `1E3` are read, whatever the locale of the program. [GAOL v5] The “+” sign before numbers and `inf` is optional.

Numbers. A *NUMBER* is written in decimal, with an optional point and exponent (`12`, `0.1`, `1e-3`), in hexadecimal (`0x1f`, `0x1.3p-1`), in octal (`0o17`) or in binary (`0b101`, `0b1.1p3`), the digits after `0x`, `0o` or `0b` having an optional point and an optional binary exponent after a `p`. [GAOL v5] A number that begins

with 0 is decimal: `010` is 10, as IEEE 1788-2015 has it (9.7.2), and octal is written `0o10`, as in Python. Numbers are read exactly, whatever their number of digits: a number that is a double is read as that double, and any other one is replaced by the tightest interval of doubles containing it. [GAOL v5] GAOL 4 read them with the `strtod()` of the C library, which did not enclose `0.1` where it rounds to nearest in every rounding direction (the C runtime of Windows, musl on 64-bit ARM processors). The point of a number is a point whatever the locale of the program, a locale writing a decimal comma included (as `setlocale(LC_ALL, "")` sets under a French one). [GAOL v5] GAOL 4 read `0.1` as 0 there, `strtod()` stopping at the point. The modes of the processor that flush the subnormal numbers to zero (flush-to-zero and denormals-are-zero, which a program linked with `-Ofast` has) do not change what a number is read as: `1e-310` is read as the tightest interval of doubles containing it, and `0` as 0. [GAOL v5] Reading a long number takes a time quadratic in its number of digits: `"0."` followed by n digits `3` takes 0.017 s for $n = 20\,000$, 0.28 s for 80 000 and 4.4 s for 320 000, and the uncertain form 2.3 times as long. [GAOL v5]

The uncertain form. An *UNCERTAIN_NUMBER* of IEEE 1788-2015 (12.11.3) is a decimal number followed by `?`, a radius in units of its last digit (half a unit when it is left out, and an infinite radius for a second `?`), an optional direction, `u` or `d`, which keeps only the radius above or below, and an optional exponent: `3.56?1` is [3.55, 3.57], `3.56?1e2` is [355, 357], and `-10??u` is $[-10, +\infty]$. [GAOL v5] It is an interval literal of its own, not a bound: `"[5?1]"` and `"[1, 3.56?1]"` throw `std::invalid_argument`, as IEEE 1788-2015 has it (12.11.4), while `"3.56?1*2"` is [7.1, 7.14]. A sign written next to an uncertain number, as `10-3.56?1` or `10+3.56?1`, is read as a part of the number, and the text throws `std::invalid_argument`, as does `10 -3.56?1`: a blank is needed after the sign, `10 - 3.56?1`. [GAOL v5]

Functions. A *NAME* is the name of a function of GAOL, with its arguments, whatever the case of its letters: `gaol::textToInterval()`, from one string or from two, and `operator>>` read the names of GAOL, and `gaol_ieee1788::textToInterval()` those of IEEE 1788-2015 (see Section 11.3, page 96), as each namespace has its own `pow`. [GAOL v5] The names of GAOL are:

`sqr`, `sqrt`, `cbrt`, `pow`, `nth_root`, `exp`, `exp2`, `exp10`, `expm1`, `exp2m1`, `exp10m1`, `log`, `log2`, `log10`, `log1p`, `log2p1`, `log10p1`, `sin`, `cos`, `tan`, `asin`, `acos`, `atan`, `atan2`, `sinh`, `cosh`, `tanh`, `asinh`, `acosh`, `atanh`, `sinpi`, `cospi`, `tanpi`, `asinpi`, `acospi`, `atanpi`, `atan2pi`, `hypot`, `compoundm1`, `rsqrt`, `sign`, `ceil`, `floor`, `trunc`, `integer`, `round_ties_to_even`, `round_ties_to_away`, `abs`, `min`, `max`, `inverse`, `fma`

Each is the function of GAOL of that name (Chapters 8 and 9); `pow` is the `pow` of the namespace `gaol` (see Section 8.1.2, page 59), and `cbrt(x)` is `nth_root(x,3)`. A name that is not in the list, or a call with a wrong number of arguments, throws **`std::invalid_argument`**. The second argument of `nth_root` shall be a point interval that can be evaluated as an integer, negative ones included; otherwise, a **`std::domain_error`** exception is thrown (or an error is reported, depending on whether exceptions were enabled or not at configuration time—see Section 2.5, page 9). [GAOL v5] The integer may be of any size a double holds: `"nth_root(8,3000000000)"` is `nth_root(8, 3000000000LL)`, where it was refused beyond the `int`s. One of 2^{63} or more in magnitude is taken as $\pm(2^{63} - 2)$: the integer powers are the same, and

the enclosure of the roots of that order holds those of the larger orders. The second argument of `pown` and `rootn`, with the names of the standard, is read in the same way. GAOL 4 read the trigonometric and hyperbolic functions, `exp`, `log`, `pow`, `sqrt` and `nth_root` only.

Bounds. The notations "n" and "$[n]$" are equivalent. Expressions in bounds are evaluated using interval arithmetic, intervals included; the left (resp. right) bound of the result is then used, depending on the side it appeared in: `"[cos([0,1]), 2]"` is $[\cos 1, 2]$. A bound left out is infinite. An interval literal `[l, u]` whose left bound is above its right bound is the empty set, as in IEEE 1788-2015: `"[2,1]"`, and also `"[9007199254740993, 9007199254740992]"` and `"[1.0000000000000001, 1]"`, whose numbers are compared exactly with the doubles around them, a number that is not a double lying strictly between the two doubles around it. [GAOL v5] They were read as the points 2^{53} and 1. Two numbers that are not doubles and lie between the same two doubles are compared exactly too: `"[1.0000000000000002, 1.0000000000000001]"` is the empty set, and `"[1.0000000000000001, 1.0000000000000002]"` is the interval from the double below to the double above them, as is the same number written twice. `pi` and another number lying between the same two doubles, a number far beyond the doubles (about 10^{311} or more, or 10^{-331} or less, in decimal, about 2^{1026} or more, or 2^{-1081} or less, in the other bases), and bounds that are computed, as `1/3`, are not told apart: the literal is then the interval from the lower bound of the first to the upper bound of the second. The two bounds in the string "`<a, b>`" must be expressions that evaluate to the same double even for different rounding directions: an `std::invalid_argument` exception is raised otherwise, as for `"<0.1, 0.1>"`, a number that is not a double being no bound of `<a, b>`. [GAOL v5] `"<0.1, 0x1.9999999999999p-4>"`, the double just below 0.1, was read as the point [`0x1.9999999999999p-4`]. A bound that is computed is a double when its value is a point, as `0.5`, `(0.25+0.25)` and `[0.5]` are; `"<[0.1], 0x1.9999999999999p-4>"` and `"<0x1.5555555555556p-2, 1/3>"`, whose first or second bound has the two doubles around the number for its value, throw `std::invalid_argument` too, where they were read as the point of the other bound. [GAOL v5] The two doubles are compared by their bits, -0 and $+0$ being the same bound, so that the mode of the processor that reads the subnormal numbers as zero (denormals-are-zero) changes nothing: `"<1e-310, 1e-309>"` throws under it too. [GAOL v5] Compared as doubles, which two subnormal numbers are equal as under that mode, they were read $[10^{-310}, 10^{-309}]$. GAOL 4 wrote the degenerate intervals in that form, and the texts it wrote for those whose digits are exact, as `<4, 4>`, are still read; `operator<<` writes such an interval `[a]`, the literal of IEEE 1788-2015 for a point, and any other with its two bounds (see Section 10.2.2, page 86).

Infinities. An interval literal whose left bound is $+\infty$ or whose right bound is $-\infty$ is the empty set, as `numsToInterval` of IEEE 1788-2015 has it: `"[inf]"` and `"[-inf, -inf]"` are empty. [GAOL v5] The expression `"inf"` alone is still [MAX, $+\infty$], and `"-inf"` [$-\infty$, $-$MAX] (see Section 5.1, page 42).

Threads. Several threads may read strings at once, in parallel: the lexer is reentrant and the parser pure, and each reading of a string has a state of its own. [GAOL v5] GAOL 4 kept that state in global variables, and crashed there.

Long and nested strings. A string may be as long as memory allows: a sum of 100000 terms is read in a time proportional to its length, and

takes no more of the stack of the program than a sum of two terms. [GAOL v5] The reader built a tree with a node for each operation, which was evaluated and deleted with one call within another per node: a sum of 100000 terms overflowed the stack and crashed the program. Operations are now computed as they are read, and the expressions that a program builds through `gaol/parser/gaol_expression.h` are evaluated and deleted without recursion too. A token, a number, a name or a run of blanks, holds 2^{29} characters at most, the buffer of the lexer: a longer one throws `std::bad_alloc`. The nesting of a string is limited by the stack of the parser, which holds 10000 entries: an open parenthesis or bracket, or a sign, takes one until the expression it opens is complete, the name and the parenthesis of a call two, and an operand and the operator or the comma after it two, until the operand that follows them is read. About 10000 nested parentheses, brackets or signs, 5000 nested calls of one argument and 3300 levels of `1+(1+(...))` are read; a string nested deeper is refused as any string that is not an expression, and throws **`std::invalid_argument`**.

A `std::string`, given to `textToInterval()` or read by `operator>>`, is read whole: a NUL character is no character of any token, and a string holding one is not an expression. [GAOL v5] The reader read it up to the NUL, so that `"[1,2]"` followed by a NUL and any text was [1, 2], and a string of $2^{32} + 3$ characters was read as its first 3, as `"1+1"` for `"1+1+1+..."`: the lexer of flex held the length of a string in an `int`.

Errors. A string that is not an expression throws **`std::invalid_argument`**. An error found anywhere in the string stops the reading, and no interval is given: `"[nth_root(8,1.5)]+[1,2]"` throws, where the two grammars of GAOL 4 gave $[-\infty, +\infty]$. [GAOL v5] Where memory runs out, the reading throws `std::bad_alloc`, and frees all it allocated. [GAOL v5] flex, which wrote "out of dynamic memory" and ended the program, and the parser, which read a stack it could not grow as a string nested too deep, threw no such exception. Without exceptions, an error that the reader finds as it computes, a wrong call, an uncertain number as a bound or two different bounds in `<a, b>`, or a bound of it that is no double, makes the reading fail, as a syntax error does: `parse_interval()` returns `false`, and `gaol::textToInterval()` and `operator>>` report the error and abort, as for any string that is not an expression. [GAOL v5] The error was reported, and the program aborted, inside the reader, so that `gaol_ieee1788::textToInterval()` could not give the empty set (see Section 11.3, page 96).

Example

```
interval x = textToInterval("[4, 6*7]");
interval y = textToInterval("[-inf, dmax]");
interval z = textToInterval("[3.14,3.15]/8", "[3.14,3.15]/7");
interval t = textToInterval("[3.14,3.15]/[7,8]");
cout << textToInterval("[1,]") << ' ' << textToInterval("[ ]") << ' '
     << textToInterval("[Entire]") << endl;
[1, inf] [empty] [-inf, inf]
cout << textToInterval("[0x1.3p-1, 2/3]") << ' '
     << textToInterval("3.56?1") << endl;
[0.59375, 0.6666666666666668] [3.549999999999999, 3.5700000000000001]
cout << textToInterval("1+[1,2]") << ' '
     << textToInterval("[cos([0,1]), 2]") << endl;
[2, 3] [0.5403023058681396, 2]
```

10.2 Writing intervals

Intervals may be printed into a stream like any other C++ primitive type by using the "`<<`" operator.

ostream& **operator<<** (ostream& *out*, const interval& I)

> Prints the interval I to the output stream *out*. The way the intervals are actually displayed depends on the active format (see next section). However, whatever the format, an empty interval is always displayed as `[empty]`, as IEEE 1788-2015 writes the empty set (12.11.3). [GAOL v5] The formats **`width`** and **`center`** of GAOL 4 wrote `empty`, which is no interval literal.

In the decimal formats, each bound is rounded outward, whatever the C library: its magnitude is written by the C library, rounded to nearest, then compared exactly with the bound, and its last digit is moved by one when it is on the wrong side. The text read back therefore encloses the interval written. [GAOL v5] GAOL 4 let the C library round each bound in the direction it set, which the C runtime of Windows and musl do not honour.

std::string **exact_string** (const interval& *I*)

> Returns what `operator<<` writes in the format `hexa` (see next section), without reading nor changing the global output format: `"[empty]"` for the empty set, `"[a]"` for a degenerate interval (`"[0x0p+0]"` for a zero of either sign), and otherwise the two bounds in the hexadecimal-significand form of IEEE 1788-2015 (13.4.1), which `textToInterval()` reads back as the same doubles, a zero degenerate interval as [+0, +0], the same set. This is the exact text representation of the standard, its intervalToExact. [GAOL v5] The text is the same whatever the locale of the program, a locale writing a decimal comma included, and whatever the C library.

Example

```
1 cout << exact_string(textToInterval("0.1"));
2 [0x1.9999999999999p-4, 0x1.999999999999ap-4]
```

10.2.1 Converting intervals to strings

For convenience, the `interval` class provides a conversion operator into the standard C++ type `string`, which writes the interval in the active format.

Example

```
1 interval I(3,4);
2 string s = "test line embedding " + string(I) + " as a string";
3 // Now, s is "test line embedding [3, 4] as a string"
```

10.2.2 Output format

Intervals may be displayed following five different formats:

`agreeing`. By printing all the digits that are the same in the left and right bounds followed by an interval containing the remaining digits:

> "`3.141~[5926, 6001]`" stands for "[3.1415926, 3.1416001]"

Each bound is written with the digits of the precision and the flag `showpoint`, in the global locale of the program. The digits are shared only by two bounds that are finite, not zero and of the same sign, written with as many digits before the point and the same exponent, and that share their first digit that is not 0: the other intervals are printed as in the format **bounds**. The digits of each bound are kept: GAOL 4 dropped from both bounds what followed the last character of the left one that is not a zero, and wrote [1.25, 1.2567] `1.25~[, ]`. [GAOL v5] It shared the digits of every interval whose right bound is at most ten times the left one, [1, 10] being `1~[., 0.]`, [4, 6] `~[4., 6.]`, without a shared digit, and a zero `~[-0., 0.]`, and printed every interval with a negative bound with its bounds. A point interval that the digits write exactly is printed as a single number, without the brackets: `interval(4.0)` is `4.000000000000000`, where the format **bounds** prints `[4]`; any other point interval is printed as any interval is, `interval(0.1)` being `0.100000000000000~[0, 1]`. The point zero is printed `[0]`, as in the format **bounds** (the formats **width** and **center** print it `0`, and **hexa** `[0x0p+0]`).

Example

```
1 interval::format(interval_format::agreeing);
2 std::cout << interval(4,6) << ' ' << interval(1.25,1.2567) << ' '
3           << interval(-1.2567,-1.25) << std::endl;
4 [4, 6] 1.25~[0, 67] -1.25~[67, 0]
```

bounds. By printing the bounds between square brackets. A degenerate interval whose value is written exactly by the digits displayed is printed as that value alone between square brackets, the literal of IEEE 1788-2015 for a point (12.11), which `textToInterval()` reads back as the same point:

Example

```
1 interval::format(interval_format::bounds);
2 std::cout << interval(3,5) << ' ' << interval(4) << std::endl;
3 [3, 5] [4]
```

A zero is printed `[0]` whatever the signs of its bounds: [−0, +0], which `interval::zero()` is with the SSE2 and the NEON intervals, [+0, +0] and [−0, −0] are the same set, which `[0]` and `[-0]` are both read back as, and the text is then the same on every build (the format **hexa** prints it `[0x0p+0]`). A degenerate interval whose value has more digits than those displayed is printed like any other interval, with its two bounds rounded outward: `interval(0.1)` is printed `[0.1, 0.1000000000000001]`, which is read back, as any other text, as an interval containing it. The bounds are printed with a decimal point, whatever the locale of the stream, so that the reader, whose decimal point is a point whatever the locale, reads the text back: under a locale writing a decimal comma, it would take `[-2,5]` for the interval [−2, 5], and refuses `[0,5, 2,5]`. [GAOL v5] GAOL 4 printed every degenerate interval between angles, the text of its value rounded downward and upward: `interval(4)` was `<4, 4>`, a form of GAOL that is no literal of the standard, and `interval(0.1)` `<0.1, 0.1000000000000001>`, which its reader refused (`std::invalid_argument`), the angles standing for one double only. The reader still takes the angles (see Section 10.1.1, page 82): a text such as `<4, 4>`, written by GAOL 4, is read back.

width. by printing their midpoint and their radius, the `midpoint()` and the `rad()` of IEEE 1788-2015 (12.12.8): the interval is contained in the midpoint plus or minus the radius, which is the smallest number that makes it so:

Example

```
interval::format(interval_format::width);
std::cout << interval(4,5) << std::endl;
4.5 (+/- 0.5)
```

The midpoint is written rounded to nearest and the radius rounded upward, so that a radius that is not zero is never written 0. Rounded upward, a radius may look noisy with the default 16 digits: [0.1, 0.3] is written `0.2 (+/- 0.1000000000000001)`, its radius being the double nearest to 0.1, which is a little above 0.1; this is intended. A degenerate interval has no radius: it is written as its midpoint alone, and an unbounded interval has an infinite radius, its midpoint being 0 or plus or minus the largest double (`[3, inf]` is `1.797693134862316e+308 (+/- inf)`). The midpoint and the radius are written with the decimal point of the locale of the stream and its grouping of the digits, the radius without a sign, even with the flag `showpos`, and a midpoint 0 as `0`. [GAOL v5] GAOL 4 wrote $(a + b)/2$ and $(b - a)/2$ rounded to nearest, which do not contain the interval ($[1, 1 + 2^{-52}]$ was `1 (+/- 1.11e-16)`), a radius 0 for $[0, 5 \cdot 10^{-324}]$, and `inf` for the midpoint of $[10^{308}, 1.7 \cdot 10^{308}]$; the manual and the header described this number as the width.

center. by printing their midpoint only, as the width format does. [GAOL v5]

hexa. by printing their left and right bounds in the hexadecimal-significand form of IEEE 1788-2015 (13.4.1), which is exact and which `textToInterval()` reads back as the same doubles, without the round-off error of a binary-to-decimal conversion: the recovery requirement of the standard (13.4), whatever the precision, the flags and the locale of the stream. As in the format **bounds**, a degenerate interval is printed as its value alone between square brackets, `[0x1.8p+0]` for `interval(1.5)`, and a zero `[0x0p+0]` whatever the signs of its bounds, which is read back as $[+0, +0]$, the same set. GAOL 4 printed the sixteen hexadecimal digits of each double, which its parser did not read. [GAOL v5]

Example

```
interval::format(interval_format::hexa);
std::cout << textToInterval("0.1") << std::endl;
[0x1.9999999999999p-4, 0x1.999999999999ap-4]
```

Note that *the* **bounds** *and* **hexa** *formats are the ones read back as an input* (see previous section), whatever the locale of the stream. The formats **width** and **center** are for the eye: the midpoint is written with the digits of the precision, so that the text `c (+/- w)` does not necessarily contain the interval, where the text of the formats **bounds**, **agreeing** and **hexa** does. A radius that took the rounding of the midpoint into account would show the interval `0.1`, which is one number, as `0.1 (+/- 5.6e-18)`, and would not show that an interval is narrower than the digits of its midpoint, which is what the format is for.

The choice of the format to use is made through the following static methods:

void interval::**format** (interval_format::format_t *f*) static
interval_format::format_t interval::**format** (void) static

The first form of the method allows modifying the format to use in subsequent printing of intervals. The second form reports what is the current form in use. It returns a value of type `interval_format::format_t` (see below and 10.2.4 for an example of use). The format is global to the program: `exact_string()` gives the hexadecimal text without changing it.

interval_format struct

Structure type used to choose the output format for intervals. It has five possible values of type `interval_format::format_t`:

- `interval_format::agreeing`.
- `interval_format::bounds`.
- `interval_format::width`.
- `interval_format::center`.
- `interval_format::hexa`.

Example

```
interval I(interval::pi());

interval::format(interval_format::agreeing);
cout << I << "\n";
// Prints 3.14159265358979~[3, 4]
// The ~[] part is dropped if the bounds agree on all digits

interval::format(interval_format::bounds);
cout << I << "\n";
// Prints [3.141592653589793, 3.141592653589794]

interval::format(interval_format::width);
cout << I << "\n";
// Prints 3.141592653589793 (+/- 4.440892098500627e-16)

interval::format(interval_format::center);
cout << I << "\n";
// Prints 3.141592653589793

interval::format(interval_format::hexa);
cout << I << "\n";
// Prints [0x1.921fb54442d18p+1, 0x1.921fb54442d19p+1]
```

10.2.3 Choosing the number of digits to display

You can manipulate the number of digits to print by using the `precision()` static methods of the interval class. The precision of the stream, which `std::setprecision` sets, is not used for intervals, and is left as it was: the numbers written after an interval keep it. The width of the stream (`std::setw`) and its adjustment apply to the whole interval; with `std::internal`, the fill goes between the sign of the midpoint and its digits in the formats `width` and

`center`, and of the shared digits in the format `agreeing`, as it does for a number, and before the `[` in the others, as `std::right` has it. [GAOL v5] GAOL 4 set the precision of the stream to that of the intervals and left it so, and `std::setw` padded the `[` of an interval alone.

std::streamsize interval::**precision** (void) static

Returns the current number of digits used for printing bounds of intervals, 16 by default.
See example below.

std::streamsize interval::**precision** (std::streamsize *n*) static

Set the number of digits to use for printing bounds to *n*. In addition, returns the number of digits previously used.
See example below.

10.2.4 Example

Example

```
1  #include <iostream>
2  #include <gaol/gaol.h>
3  using namespace gaol;
4
5  using std::cout;
6  using std::endl;
7
8  int main(void)
9  {
10   interval::precision(4);
11   interval::format(interval_format::bounds);
12   cout << interval::pi() << endl;
13
14   if (interval::format() != interval_format::bounds) {
15     cout << interval::pi() << endl;
16   } else {
17     interval::precision(16);
18     interval::format(interval_format::width);
19     cout << interval::pi() << endl;
20   }
21   gaol::cleanup();
22   return 0;
23 }
24 [3.141, 3.142]
25 3.1415926535897931 (+/- 4.4408920985006262e-16)
```

The previous program has the output shown under it.

The first call to `interval::format()` is unnecessary since the default format is `interval_format::bounds`.

Note. An interval written in decimal and read back gives an interval that encloses it, whatever the precision chosen, the bounds being rounded outward; the fewer the digits, the wider. The format `hexa` gives the same interval back.

The `interval::pi()` constant is a predefined *canonical* interval containing π (see Section 6, page 45). Here, its bounds are two consecutive doubles, 2^{-51} apart: the midpoint written is one of them, and the radius is their distance,

the smallest that has the interval within the midpoint plus or minus it. [GAOL v5] GAOL 4 wrote half of it.

The `interval_format::width` format may be useful whenever the number of digits displayed is insufficient to know whether the result is a single floating-point number or an interval whose size is very small (consider for example the first result above), because *we have the guarantee that if an interval is not a single number, the radius displayed will be different from zero*, whatever its size and the number of digits: the radius is rounded upward. A degenerate interval is displayed without a radius.

11

The names of IEEE 1788-2015

GAOL names its operations its own way: the reverse function coshRev(c, x) of IEEE 1788-2015 is `acosh_rel(c, x)`, mulRev(b, c, x) is `div_rel(c, b, x)`, with its arguments in another order, and roundTiesToEven(x) is `round_ties_to_even(x)`. The namespace `gaol_ieee1788`, which `gaol/gaol.h` brings along, gives each operation of the standard that GAOL provides the name and the argument order the standard gives it, so that a reader of the standard finds it without looking for its translation. It is not in `gaol`, and it holds the type `interval` as well: one directive lets a program use GAOL under the names of the standard. [GAOL v5]

Example

```
#include <iostream>
#include <gaol/gaol.h>
using namespace gaol_ieee1788;

int main(void)
{
  interval x = numsToInterval(1, 2);
  interval y = mulRev(numsToInterval(2, 2), x);   // x / 2
  interval z = sinPi(x) + rootn(y, 3);
  bool b = strictLess(x, entire());
  std::cout << std::boolalpha
            << x << ' ' << y << ' ' << z << ' ' << b << std::endl;
  gaol::cleanup();
  return 0;
}
[1, 2] [0.5, 1] [-0.2062994740159004, 1] false
```

A program opens `gaol` or `gaol_ieee1788`, not both (see Section 3.4, page 27). A name of the program's own that one of the standard shadows, a constant `inf` for instance, is to be qualified: `gaol_ieee1788::inf(x)`. So is `less(x, y)` next to `using namespace std;`, where it is ambiguous with the class template `std::less`.

11.1 Where the standard and GAOL differ

The functions of GAOL that have the name and the meaning the standard gives them (`sin`, `exp`, `sqrt`, `min`...) are the same functions in `gaol_ieee1788`. Where the standard and GAOL differ, the names of `gaol_ieee1788` follow the standard:

- `pow(x, y)` is the `pow` of Table 9.1, defined for $x > 0$, and for $x = 0$ when $y > 0$, where the `pow` of `gaol` takes the integer power for an integer exponent, a negative base included (see Section 8.1.2, page 59). `pow(x, p)` with a number p is `pow(x, [p])`, `pow(x, 2)` included, `[p]` containing an integer that is no double: the power with an integer exponent is `pown(x, n)` in the standard, whose n is an integer rather than an interval. An integer exponent beyond the `int`s gives the power of CORE-MATH at the bounds of x, as the `pow` of `gaol` does on the whole of x;
- `inf(x)` and `sup(x)` are $+\infty$ and $-\infty$ for the empty set (Table 10.2), where `left()` and `right()` are NaN, and `inf` returns -0 for a left bound 0 (12.12.8);
- `isMember(m, x)` is `false` for an infinite m (10.6.3);
- `intervalToText(x)` writes the bounds of `x` rounded outward, `[l, r]`, `[a]` for a degenerate interval whose value the digits write exactly, as `[4]`, or `[empty]`, with the digits of `interval::precision()` and a decimal point, an interval literal of the standard (12.11) that `textToInterval` reads back as an interval containing `x` (13.3): whatever the output format that `interval::format()` sets and the locale of the program, where `operator<<` writes the current format, which is no literal in the format **`width`**; [GAOL v5]
- `textToInterval` and `exactToInterval` read the names of the functions of the standard, and return the empty set for a string that is no interval (12.1.3), where `gaol::textToInterval` reads the names of GAOL and throws (see Section 11.3, page 96); a null pointer is no string, and they throw `std::invalid_argument` for it too; [GAOL v5]
- `pownRev(c, x, p)` takes every integer p, $p \leqslant 0$ included: x where $1 \in c$ for $p = 0$, and for $p < 0$ the hull of the $x \neq 0$ with x^p in c, c being cut at 0. Its p, and the q of `rootn(x, q)`, may be of any integer type: `pownRev(c, 3000000000u)` converted p to a negative `int` and gave the reverse of that power; beyond the `unsigned int`s, `pownRev` gives the hull of the parts of x in the roots of order p of c and, for an even p, in their opposites. [GAOL v5]

Example

```
cout << pow(interval(-4,-1), interval(2.0)) << ' '
     << pown(interval(-4,-1), 2) << ' '
     << pow(interval(2,3), interval(1e10)) << endl;
[empty] [1, 16] [1.797693134862315e+308, inf]
cout << boolalpha << inf(empty()) << ' ' << sup(empty()) << ' '
     << isMember(GAOL_INFINITY, entire()) << ' '
     << textToInterval("[1, 2") << endl;
inf -inf false [empty]
```

11.2 The operations

The last argument x of the reverse functions is optional, and $[-\infty, +\infty]$ when it is left out (10.5.4).

gaol_ieee1788	GAOL
Constants and constructors (10.5.2, 10.5.8)	
`empty()`, `entire()`	`interval::emptyset()`, `interval::universe()`
`numsToInterval(l, u)`	`interval(l, u)`
`textToInterval(s)`	`gaol::textToInterval(s)` with the names of the standard, the empty set for a wrong string
Forward elementary functions (Table 9.1)	
`neg`, `add`, `sub`, `mul`, `div`	`-x`, `x + y`, `x - y`, `x * y`, `x / y`
`recip(x)`	`inverse(x)`
`sqr`, `sqrt`, `fma`	the same
`pown(x, p)`	`gaol::pow(x, p)` for an integer p of any type
`pow(x, y)`, `pow(x, p)`	the `pow` of the standard
`exp`, `exp2`, `exp10`, `log`, `log2`, `log10`	the same
`sin`, `cos`, `tan`, `asin`, `acos`, `atan`, `atan2`	the same
`sinh`, `cosh`, `tanh`, `asinh`, `acosh`, `atanh`	the same
`sign`, `ceil`, `floor`, `trunc`	the same
`roundTiesToEven`, `roundTiesToAway`	`round_ties_to_even`, `round_ties_to_away`
`abs`, `min`, `max`	the same
Recommended functions (Table 10.5)	
`rootn(x, q)`	`nth_root(x, q)`
`expm1`, `exp2m1`, `exp10m1`	the same
`logp1`, `log2p1`, `log10p1`	`log1p`, `log2p1`, `log10p1`
`hypot`, `rSqrt`	`hypot`, `rsqrt`
`compoundm1`	the same
`sinPi`, `cosPi`, `tanPi`	`sinpi`, `cospi`, `tanpi`
`asinPi`, `acosPi`, `atanPi`, `atan2Pi`	`asinpi`, `acospi`, `atanpi`, `atan2pi`
Reverse functions (Table 10.1)	
`sqrRev(c, x)`, `absRev(c, x)`	`sqrt_rel(c, x)`, `invabs_rel(c, x)`
`pownRev(c, x, p)`	`nth_root_rel(c, p, x)`
`sinRev`, `cosRev`, `tanRev(c, x)`	`asin_rel`, `acos_rel`, `atan_rel(c, x)`
`coshRev(c, x)`	`acosh_rel(c, x)`
`sinhRev`, `tanhRev(c, x)` (not in the standard, named after `coshRev`)	`asinh_rel`, `atanh_rel(c, x)`
`mulRev(b, c, x)`	`div_rel(c, b, x)`
`powRev1(b, c, x)`, `powRev2(a, c, x)`	`pow_rel(c, b, x)`, `pow_exponent_rel(c, a, x)` [GAOL v5]
`atan2Rev1(b, c, x)`, `atan2Rev2(a, c, x)`	`atan2_rel_y(c, b, x)`, `atan2_rel_x(c, a, x)` [GAOL v5]
`mulRevToPair(b, c)` (10.5.5)	the two parts of `div_rel(c, b)`, in a `std::pair`
Cancellative operations and set operations (10.5.6, 10.5.7)	
`cancelMinus`, `cancelPlus`	`cancel_minus`, `cancel_plus`

gaol_ieee1788	GAOL
`intersection(x, y)`, `convexHull(x, y)`	`x & y`, `x | y`
Numeric functions (Table 10.2)	
`inf(x)`, `sup(x)`	`x.left()`, `x.right()`, but for the empty set
`mid`, `wid`, `rad`, `mag`, `mig(x)`	`x.midpoint()`, `width()`, `rad()`, `mag()`, `mig()`
`midRad(x, m, r)`	`x.mid_rad(m, r)`
Boolean functions (10.5.10, Tables 10.3 and 10.4)	
`isEmpty(x)`, `isEntire(x)`	`x.is_empty()`, `x.is_entire()`
`equal(a, b)`, `subset(a, b)`	`a.set_eq(b)`, `a.set_leq(b)`
`interior(a, b)`, `disjoint(a, b)`	`a.set_le(b)`, `a.set_disjoint(b)`
`less(a, b)`, `strictLess(a, b)`	`a.less(b)`, `a.strictly_less(b)`
`precedes(a, b)`, `strictPrecedes(a, b)`	`a.certainly_leq(b)`, `a.certainly_le(b)`
`isCommonInterval(x)`, `isSingleton(x)`	`x.is_common_interval()`, `x.is_a_double()`
`isMember(m, x)`	`x.set_contains(m)`, false for an infinite m
Input and output (Clause 13)	
`intervalToText(x)`	the bounds `[l, r]`, `[a]` or `[empty]`, with the current precision, whatever the format and locale
`intervalToExact(x)`	`exact_string(x)`
`exactToInterval(s)`	`textToInterval(s)`
Expressions	
`pown(e, n)`, `pow(e1, e2)`	the expressions of `pown` and of the `pow` of the standard, declared by `gaol/parser/gaol_expression.h`

`mulRevToPair(b, c)` returns the set $\{x \mid \exists \beta \in b,\ \beta x \in c\}$ in two intervals, the pair (u, v) of the standard: u before v when the set has two parts, which happens when 0 is strictly between the bounds of b and c does not contain 0, $(u, \varnothing)$ when it is one interval, and $(\varnothing, \varnothing)$ when it is empty. The convex hull of u and v is `mulRev(b, c)`. Where b is unbounded on both sides, the two parts share 0. [GAOL v5]

Example

```
1 std::pair<interval, interval> p = mulRevToPair(numsToInterval(-1, 1),
2                                                numsToInterval(1, 2));
3 std::pair<interval, interval> q = mulRevToPair(numsToInterval(2, 4),
4                                                numsToInterval(1, 2));
5 cout << p.first << ' ' << p.second << ' '
6      << q.first << ' ' << q.second << endl;
7 [-inf, -1] [1, inf] [0.25, 1] [empty]
```

11.3 Reading with the names of the standard

`textToInterval(s)` reads a string as `gaol::textToInterval()` does (see Section 10.1.1, page 82), with the names of the functions of IEEE 1788-2015: those of Tables 9.1 and 10.5 that GAOL provides, whatever the case of their letters. [GAOL v5]

`neg`, `add`, `sub`, `mul`, `div`, `recip`, `sqr`, `sqrt`, `fma`, `pown`, `pow`, `exp`, `exp2`, `exp10`, `log`, `log2`, `log10`, `sin`, `cos`, `tan`, `asin`, `acos`, `atan`, `atan2`, `sinh`, `cosh`, `tanh`, `asinh`, `acosh`, `atanh`, `sign`, `ceil`, `floor`, `trunc`, `roundTiesToEven`, `roundTiesToAway`, `abs`, `min`, `max`, `rootn`, `expm1`, `exp2m1`, `exp10m1`, `logp1`, `log2p1`, `log10p1`, `hypot`, `compoundm1`, `rSqrt`, `sinPi`, `cosPi`, `tanPi`, `asinPi`, `acosPi`, `atanPi`, `atan2Pi`

`pow` is the pow of Table 9.1, and the second argument of `pown` and `rootn` is an integer. The names of GAOL alone (`nth_root`, `cbrt`, `inverse`, `integer`, `log1p`, `round_ties_to_even`...) are no functions there, and give the empty set, as any string that is no interval: a wrong call (`"pown([2,5],2.5)"`, `"sin(1,2)"`) and a string holding a NUL character included, whether GAOL was configured with exceptions or not. [GAOL v5] Without them, a wrong call ended the program. Where memory runs out, `textToInterval` throws `std::bad_alloc`, which it turned into the empty set. As `pow`, the function `textToInterval` is in the namespace `gaol` too, where it reads the names of GAOL and throws `std::invalid_argument` for a wrong string: a program calls the one of the namespace it opens.

Example

```
1 cout << textToInterval("pown([2,5],5)") << ' '
2      << textToInterval("pow([-4,-1],2)") << ' '
3      << textToInterval("nth_root(8,3)") << endl;
4 🖶 [32, 3125] [empty] [empty]
```

With `using namespace gaol;`, `textToInterval("pow([-4,-1],2)")` is [1, 16], and `textToInterval("pown([2,5],5)")` throws.

11.4 What is not provided

GAOL has no decorations: only bare intervals are provided (Clause 11). Nor does it provide the slope functions (Table 10.6), overlap (10.6.4), the reduction operations sum, dot, sumSquare and sumAbs (12.12.12), and the exact operations of 12.13.5. The end of `gaol/gaol_ieee1788.h` keeps this list.

11.5 Coming from another library

The names of interval libraries differ, and the same symbol can mean another relation. The first table gives, for what an algorithm uses, the name of GAOL v5, the one of `gaol_ieee1788`, and those of IBEX and Codac, which share them, of C-XSC and of Boost.Interval. A dash means that the sources read have no function of the kind, not that the algorithm cannot be written. [GAOL v5]

	GAOL v5	`gaol_ieee1788`	IBEX, Codac
Lower bound	`x.left()`	`inf(x)`	`x.lb()`
Upper bound	`x.right()`	`sup(x)`	`x.ub()`
Midpoint, a double	`x.midpoint()`	`mid(x)`	`x.mid()`
Midpoint, an interval	`x.mid()`	—	—
Width, rounded upward	`x.width()`	`wid(x)`	`x.diam()`
Radius	`x.rad()`	`rad(x)`	`x.rad()`
Smallest, greatest absolute value	`x.mig()`, `x.mag()`	`mig(x)`, `mag(x)`	`x.mig()`, `x.mag()`

	GAOL v5	gaol_ieee1788	IBEX, Codac
Hull, intersection	`x | y`, `x & y`	`convexHull(x, y)`, `intersection(x, y)`	`x | y`, `x & y`
Is empty	`x.is_empty()`	`isEmpty(x)`	`x.is_empty()`
Subset	`x.set_leq(y)`	`subset(x, y)`	`x.is_subset(y)`
Interior	`x.set_le(y)`	`interior(x, y)`	`x.is_interior_subset(y)`
Contains a number	`x.set_contains(d)`	`isMember(d, x)`	`x.contains(d)`
Disjoint	`x.set_disjoint(y)`	`disjoint(x, y)`	`x.is_disjoint(y)`
Equal	`x.set_eq(y)`	`equal(x, y)`	`x == y`
Inflate by `r`	`x.inflate(r)`	—	`x.inflate(r)`
Bisect	`x.bisect(ratio)`	—	`x.bisect(ratio)`, ratio 0.5 (IBEX) or 0.49 (Codac)
Two-piece division	two `div_rel`, or `mulRevToPair(b, c)`	`mulRevToPair(b, c)`	`div2` (IBEX only)

	GAOL v5	C-XSC	Boost.Interval
Lower bound	`x.left()`	`Inf(x)`	`lower(x)`
Upper bound	`x.right()`	`Sup(x)`	`upper(x)`
Midpoint, a double	`x.midpoint()`	`mid(x)`	`median(x)`
Width, rounded upward	`x.width()`	`diam(x)`	`width(x)`
Smallest, greatest absolute value	`x.mig()`, `x.mag()`	`AbsMin(x)`, `AbsMax(x)`	`norm(x)`, the greatest only
Hull, intersection	`x | y`, `x & y`; `hull(x, y)`, `intersect(x, y)`	`x | y`, `x & y`	`hull(x, y)`, `intersect(x, y)`
Is empty	`x.is_empty()`	`IsEmpty(x)`	`empty(x)`
Subset	`x.set_leq(y)`	`x <= y`	`subset(x, y)`
Interior	`x.set_le(y)`	`x < y`, `in(x, y)`	—
Contains a number	`x.set_contains(d)`	`d <= x`, `in(d, x)`	`in(d, x)`
Disjoint	`x.set_disjoint(y)`	`Disjoint(x, y)`	`!overlap(x, y)`
Equal	`x.set_eq(y)`	`x == y`	`equal(x, y)`
Inflate by `r`	`x.inflate(r)`	`Blow(x, eps)`, relative to the width	`widen(x, r)`

What the symbols `<`, `<=` and `==` mean is where code ported from another library compiles and changes meaning:

	`x < y`	`x <= y`	`x == y`
GAOL v5	every point of `x` is below every point of `y` (strictPrecedes)	every point of `x` is at most every point of `y` (precedes)	not defined: `x.set_eq(y)`
IBEX	not defined	not defined	the same bounds, the empty set equal to itself
Codac	a `BoolInterval`: true, false, unknown, or empty if an operand is	not defined	the same bounds, as IBEX
C-XSC	`x` in the interior of `y`	`x` is a subset of `y`; `r <= x`: the number `r` is in `x`	the same bounds
PROFIL/BIAS	`x` in the interior of `y`	`x` is a subset of `y`; `r <= x`: `r` is in `x`	the same bounds

	`x < y`	`x <= y`	`x == y`
Boost.Interval	true if every point of `x` is below every point of `y`, false if none is, else it throws `comparison_error`	true if every point of `x` is at most every point of `y`, false if none is, else it throws	true if `x` and `y` are the same point, false if they are disjoint, else it throws

So `0.0 <= f` is "0 is in `f`" in C-XSC and in PROFIL/BIAS, and in GAOL v5 the certainly relation "0 is at most every point of `f`": a program ported from them compiles and no longer tests what it did (`f.set_contains(0.0)` does). A Boost.Interval program that catches `comparison_error` to detect an uncertain comparison never sees it with GAOL v5, whose `x < y` is false for overlapping intervals, and the empty set, which Boost refuses, passes every certainly relation (see Section 7.5, page 53). The names are those of IBEX 2.9.1 (`ibex_Interval.h`), Codac 2 (`codac2_Interval.h`), C-XSC 2.5.4 (`interval.hpp` and `interval.inl`), PROFIL/BIAS 2.0.8 (`Interval.h`) and Boost 1.86 (`boost/numeric/interval/`), read in their sources.

12 Accuracy of the operations

IEEE 1788-2015 requires an implementation to document the tightness of each of its interval operations (12.10.3). This chapter does so for `interval`, from the algorithm of each operation; the tests of `tests` check each statement on random and special arguments (see Section 2.7, page 11), and the page `doc/accuracy.md` gives, for each, the algorithm and the largest distance the tests found. [GAOL v5]

12.1 Accuracy modes

IEEE 1788-2015 defines three accuracy modes (12.10.1), for an operation f, its exact value f^*, and an input box x:

- *tightest*: $f(x)$ is the hull, in doubles, of $f^*(x)$: each bound is the nearest double on its side of the exact bound;
- *accurate*: $f(x)$ encloses $f^*(x)$, and is within

  $$\text{nextOut}(f_{\text{tightest}}(\text{nextOut}(x))),$$

  nextOut moving each bound one double outward: roughly, $f(x)$ is no wider than the tightest result for an input one double wider, plus one double on each side;
- *valid*: $f(x)$ encloses $f^*(x)$.

The standard requires the basic operations, the integer functions and `abs`, `min` and `max` to be tightest; the other elementary functions, the reverse functions and the recommended functions to be valid, and recommends them to be accurate (12.10.2). Below, "within k doubles" means that each bound is at most k doubles beyond the tightest one.

GAOL bounds its elementary functions with CORE-MATH, which is correctly rounded in the rounding direction in effect: computing upward, GAOL takes the value at a bound as the upper bound, with nothing to add, and the double below it as the lower one. Those are the tightest bounds, the second one unless the exact value is a double, which the operations give exactly where it happens. Where an algorithm evaluates the function at the bounds of the interval, the bounds of the interval are therefore the tightest ones.

The statements hold for GAOL as the three builds make it, with the code including its headers compiled with the flags of interval arithmetic (see Section 3.1, page 14), for the SSE2, the NEON and the FPU intervals, whatever the rounding direction the calling code left, with the floating-point exceptions masked as a program starts (see Section 3.3, page 24), and on every architecture and with every compiler alike. The intervals of floats are not covered.

12.2 Tightness of each operation

Operations	Tightness
Basic operations (Table 9.1): tightest required	
`-x`, `+`, `-`, `*`, `/`, `inverse`, `sqr`	tightest
`sqrt`	tightest, whatever the rounding of the square root of the C library
`fma`	tightest
Integer and absmax functions: tightest required	
`floor`, `ceil`, `integer`, `sign`, `trunc`, `round_ties_to_even`, `round_ties_to_away`, `abs`, `min`, `max`	tightest, and exact, in every rounding direction
Powers, exponentials and logarithms: valid required, accurate recommended	
`pow(x, n)`, integer n (`pown`)	the tightest bounds, of either sign, or one double beyond where the power is within twice a bound of their error of a double, relatively (never for $n = 2$, `sqr(x)`; 2^{-98} for $n = 3$, 2^{-62} at most), below 2^{-968} and beyond the largest double too; exact where the power is a double; the same bounds with the SSE2, the NEON and the FPU intervals. Beyond the `int`s: tightest where n is a double, within three doubles otherwise
`pow(x, y)` (`pow`)	tightest, at the corners of the box and at their limits where the base is 0 or $+\infty$ or the exponent infinite; for a degenerate integer exponent within the `int`s, the integer power of the row above, one double beyond the tightest where it is within twice the error of its products (`pow([0x1.fffffffffffffp+1], [7])` has its lower bound one double below the tightest)
`exp`, `exp2`, `exp10`, `log`, `log2`, `log10`	tightest; exact where the value is a double ($\exp(0)$, $\log(1)$, 2^n, 10^n for $0 \leqslant n \leqslant 22\ldots$)
Trigonometric and hyperbolic functions: valid required, accurate recommended	
`cos`, `sin`	tightest where the function is evaluated at the bounds, at every magnitude; -1 and 1 exactly at the extrema; $[-1, 1]$ from the width $2\underline{\pi}$ on, $\underline{\pi}$ being the double below π
`tan`	tightest between two poles, at every magnitude; $[-\infty, +\infty]$ exactly when a pole is within x, but for the widths above $\underline{\pi}$ and below π
`asin`, `acos`, `atan`, `atan2`	tightest; exactly 0, or the tightest enclosures of $\pm\pi/4$, $\pm\pi/2$ and π, where the value is one of them
`sinh`, `cosh`, `tanh`, `asinh`, `acosh`, `atanh`	tightest
Recommended functions (Table 10.5)	

Operations	Tightness
`nth_root(x, n)`, $n > 0$	$n = 1, 2, 3$: tightest. Otherwise accurate: the tightest bounds or up to 2 doubles beyond, proved with integer powers; beyond the `unsigned int`s, CORE-MATH's `pow` at the corners, within one double
`nth_root(x, q)`, $q < 0$	accurate: the root above, then the tightest division, up to 4 doubles beyond the tightest bounds
`expm1`, `exp2m1`, `exp10m1`, `log1p`, `log2p1`, `log10p1`, `hypot`, `rsqrt`	tightest
`compoundm1`	tightest but in a case out of 10^{17} or so, where a bound is a double beyond [GAOL v5]
`sinpi`, `cospi`, `tanpi`, `asinpi`, `acospi`, `atanpi`, `atan2pi`	tightest, at every magnitude
The error functions, which IEEE 1788-2015 does not name	
`erf`, `erfc`	tightest; exact at 0 and at the infinities [GAOL v5]
Reverse functions (Table 10.1): valid required, accurate recommended	
`sqrt_rel`, `nth_root_rel`, `acosh_rel`, `asinh_rel`, `atanh_rel`	accurate
`nth_root_rel(c, p, x)`, $p < 0$	accurate: the root of each part of c and then its inverse, up to four doubles beyond the tightest [GAOL v5]
`invabs_rel`	tightest
`max_rel`, `min_rel`, `sign_rel`, `floor_rel` (not in Table 10.1)	tightest
`asin_rel`, `acos_rel`, `atan_rel`	accurate, or one double beyond; a bound of x beyond 2^{52} is kept
`div_rel`, `%`	accurate; tightest where the bounds of the quotients are doubles
`pow_rel`, `pow_exponent_rel`	tightest, but for a bound of x next to the solutions whose power is within one double of c, which is kept: the tightest result for x one double wider [GAOL v5]
`atan2_rel_y`, `atan2_rel_x`	accurate; within two doubles of the tightest bounds where these are not empty
`cancel_minus`, `cancel_plus`	tightest
Set operations, constructors, numeric and boolean functions	
`&`, `|`, `interval(l, u)`	tightest (exact)
`textToInterval(s)`, `operator>>`	tightest for the literals of IEEE 1788-2015; valid for expressions, computed with the operations above
`midpoint()`, `width()`, `rad()`, `mag()`, `mig()`	as 12.12.8 requires
`mid()`, `hausdorff()`	tightest
relations	exact, as Tables 10.3 and 10.4
`operator<<`	`hexa`: exact, read back as the same interval. Decimal formats: valid, and the tightest with the digits asked for where the C library rounds to nearest as it should

Where a bound of J is one double from the image of a bound of the preimage, the reverse functions keep a point of I whose image is just outside J:

`sqrt_rel`$([1 + 2^{-52}, +\infty]$`,` $[-1, 1 + 2^{-52}]$`)` is $[-1, 1 + 2^{-52}]$, whose tightest enclosure is $[1, 1 + 2^{-52}]$. They are accurate all the same, the widened J of nextOut holding that image.

Before GAOL v5, the elementary functions were correctly rounded to nearest and moved one double outward, which is at most one double beyond the tightest bound, and the hyperbolic functions were taken from the mathematical library of the system, whose values are sometimes further than one double from the exact ones.

13 Floating-point numbers

13.1 Floating-point constants

In addition to the constants available through `numeric_limits<double>`, GAOL defines the following `double` constants:

Constant (`double`)	Value
`GAOL_NAN`	NaN (quiet)
`GAOL_INFINITY`	$+\infty$

GAOL 4 also defined the doubles `two_pi`, `pi`, `half_pi`, `pi_dn`, `pi_up`, `half_pi_dn`, `half_pi_up`, `ln2_dn`, `ln2_up`, `two_power_51` and `two_power_52`, in the namespace of GAOL: a program opening it with `using namespace gaol` found them among its own names, and a `pi` of its own was ambiguous. GAOL v5 no longer declares them for the code using it. The bounds of π, $\pi/2$ and 2π are those of `interval::pi()`, `interval::half_pi()` and `interval::two_pi()`, `interval::pi().left()` being $\downarrow \pi \downarrow$, and those of $\ln 2$ those of `log(interval(2.0))`. [GAOL v5]

13.2 Floating-point functions

bool **feven** (double *x*)

Returns true whenever x is even.
This function should not be used with infinity and NaN arguments.

Example

```
1 cout << boolalpha << feven(3.0) << ' ' << feven(4.0) << ' ' << feven(3.5);
2 false true false
```

double **next_float** (double *x*)

Returns the smallest `double` greater than x.

double **previous_float** (double *x*)

Returns the greatest `double` smaller than x.

bool **is_signed** (double *x*)

Returns true whenever x is signed. No provision is made concerning the fact that x is a NaN. If you only want to test for negative numbers (and -0), you will have to test also whether x is a NaN by using `std::isnan()`.

double **minimum** (double *x*, double *y*)

Returns the minimum `double` value of x and y. This function is commutative and returns -0 when comparing -0 and $+0$, i.e.:

$$\begin{cases} \min(x,y) & = \min(y,x), & \forall x \neq \mathrm{NaN}, \forall y \neq \mathrm{NaN} \\ \min(x,\mathrm{NaN}) & = \min(\mathrm{NaN},x) = \mathrm{NaN}, & \forall x \\ \min(-0,0) & = \min(0,-0) = -0 \end{cases}$$

double **maximum** (double *x*,double *y*)

Returns the maximum `double` value of x and y. This function is commutative and returns $+0$ when comparing -0 and $+0$, i.e.:

$$\begin{cases} \max(x,y) & = \max(y,x), & \forall x \neq \mathrm{NaN}, \forall y \neq \mathrm{NaN} \\ \max(x,\mathrm{NaN}) & = \max(\mathrm{NaN},x) = \mathrm{NaN}, & \forall x \\ \max(-0,0) & = \max(0,-0) = 0 \end{cases}$$

unsigned long long **nb_fp_numbers** (double *a*, double *b*)

Returns the number of floating-point numbers in the interval $[a, b]$. In particular, we have:

- `nb_fp_numbers(a,next_float(a)) == 2`
- `nb_fp_numbers(a,a) == 1`

The two zeros are the same number: `nb_fp_numbers(-0.0,0.0)` is 1, and `nb_fp_numbers(-0.0,1.0)`, `nb_fp_numbers(-1.0,0.0)` and `nb_fp_numbers(0.0,1.0)` are all 4607182418800017409 (GAOL 4 returned 13830554455654793217 for the first two). [GAOL v5]

Note. As a precondition, a shall be lower or equal to b.
The result is an `unsigned long long`, where GAOL 4 named its type with the macro `ULONGLONGINT`, which GAOL v5 no longer defines, as it prefixes with `GAOL_` the macros of its headers. [GAOL v5]
Throws a `std::domain_error` exception (see Section 17.1, page 114) if either a or b is a NaN or an infinity, or if a is greater than b, or calls `gaol_error` and aborts where the exceptions are disabled: it returns no number then, where this manual gave `numeric_limits<unsigned long long>::max()`. [GAOL v5]

Example

```
1 cout << nb_fp_numbers(1.0, next_float(1.0)) << ' '
2      << nb_fp_numbers(1.0, 2.0);
3 2 4503599627370497
```

14 Manipulating the FPU

GAOL provides functions to set the rounding direction of the floating-point unit. The operations of GAOL do not need them: each sets the rounding direction upward itself (see Section 3.2, page 17).

On x86 processors, with GCC, Clang and Visual C++, they write the control registers of the x87 and SSE units directly (`fnstcw`/`fldcw`, `stmxcsr`/`ldmxcsr`), rather than calling `fesetround()`, which cost 50 to 250 ns per call with the C runtime of Windows; with Visual C++ for x64, the SSE register only, the x87 unit being unused there. [GAOL v5] Elsewhere (ARM, POWER, s390x, RISC-V), they call `fesetround()`, except on 64-bit ARM processors with the NEON intervals (see Section 3.2.2, page 21), where they write the rounding direction field of the control register FPCR (`mrs`/`msr`) themselves, and only when the direction changes. [GAOL v5]

14.1 Rounding functions

void **round_downward** (void)

Sets the rounding direction mode towards $-\infty$.

void **round_nearest** (void)

Sets the rounding direction mode to the nearest/even.

void **round_upward** (void)

Sets the rounding direction mode to $+\infty$.

15 Version information

The library provides four constants to allow programs to determine at runtime with which version they are dynamically linked with. The versioning scheme adopted is the one used by the Apache Software Foundation described at http://apr.apache.org/versioning.html. They are named by the namespaces `gaol` and `gaol_ieee1788`. [GAOL v5]

unsigned int **version_major** const

Major version of the library: 5 for GAOL v5.

unsigned int **version_minor** const

Minor version of the library.

unsigned int **version_micro** const

Patch version of the library.

const char *const **version** const

Version of the library as a string.

Example

```
1 cout << gaol::version;
2 🖶 5.0.0
```

16 Additional functions

The following functions are utility functions not necessarily related to intervals or floating-point numbers.

template <typename T>
bool **odd** (const *T*& *x*)

> Returns `true` if x is odd and false otherwise. The T type may be any type providing the `&` ("bitwise `and`") operator with the same semantics as the one for `int`s.

template <typename T>
bool **even** (const *T*& *x*)

> Returns `true` if x is even and false otherwise. The T type may be any type providing the `&` ("bitwise `and`") operator with the same semantics as the one for `int`s.

17 Error handling

A program that uses GAOL may report errors in two different ways:

- by throwing an exception;
- or by printing a message and aborting.

The exceptions are those of the standard library, declared in `<stdexcept>`. GAOL 4 had classes of its own (`gaol_exception`, `input_format_error`, `invalid_action_error` and `unavailable_feature_error`), which GAOL v5 no longer has (see Section 17.1, page 114). [GAOL v5]

The mechanism in use depends on the way the library is configured. If you use the option `--enable-exceptions=yes`, the default, all errors are reported through exception throwing; otherwise, a message is printed to the standard error, and the program aborts. The CMake build always reports errors by exceptions. Relying on exceptions is more in the C++ spirit, though it may incur some overhead.

It is up to the user to comply with this mechanism when adding error reporting code to ones program. GAOL defines the following macro to be used whenever one wants to report an error.

gaol_ERROR (*excep*,*msg*) macro

The behavior of the macro depends on the value chosen for the option `--enable-exceptions`: if exceptions are enabled, an exception of the class *excep*, derived from `std::exception` (the compiler refuses any other type, with exceptions or without) and in practice one of `<stdexcept>`, is thrown with the text "`file:line:` *msg*", `file` and `line` being those of the macro (`__FILE__` and `__LINE__`); otherwise, the program aborts with message *msg*. [GAOL v5]

Example

```
interval x;

[Code manipulating x]

if (x.is_empty()) {
   gaol_ERROR(std::domain_error,"Emptiness of one interval");
}
```

The `gaol_error()` function is defined as follows:

void **gaol_error** (const char *const *err*)
void **gaol_error** (const char **file*, int *line*, const char **err*)

Displays a message on the standard error output. The ternary version should be called with the `GAOL_FILE_POS` macro for the first two parameters. A null *file* is written `(unknown file)`, and a null *err* is an empty text. [GAOL v5]

The `GAOL_FILE_POS` macro is described in the next section.

17.1 Exceptions

GAOL throws the exceptions of the standard library, and none of its own. [GAOL v5] It throws two of them to report an error:

- `std::invalid_argument`, when a string is no interval (see Section 17.1.1, page 115);
- `std::domain_error`, when a function is called with an argument it does not take (see Section 17.1.2, page 115), as `bisect()` is for a ratio outside $(0, 1)$.

Both derive from `std::logic_error`, which derives from `std::exception`. A single handler of `std::logic_error` catches both, but the handler of one leaves the other alone, and none of them catches the `std::bad_alloc` that GAOL throws where memory runs out.

The `what()` of these exceptions is "`file:line:` *explanation*": the name of the source file of GAOL, without its directories, and the line that threw the exception, then what went wrong, as in `gaol_interval.cpp:48: Syntax error in interval initialization: [1, 2.` For the errors that the reader of strings finds in a call, the file is that of the grammar, `gaol_interval_parser.ypp`, and the line one of that file, which the directives `#line` of the generated parser give. A handler of `std::exception`, and the message of an exception that nothing catches, say what went wrong, where GAOL 4 gave `std::exception` whatever the error. The text is valid as long as the exception is.

GAOL_FILE_POS macro

Expands itself into the first two arguments of `gaol_error()` and `gaol_warning()`, the name of the file and the line:

Example

```
1 if ([some condition]) {
2   gaol_error(GAOL_FILE_POS, "No additional information");
3 }
```

What GAOL 4 had is gone: the base class `gaol_exception`, its accessors `file()`, `line()` and `explanation()`, and the "`<<`" operator that wrote them. The classes become those of the standard library:

- `gaol_core::input_format_error` becomes `std::invalid_argument`;
- `gaol_core::invalid_action_error` becomes `std::domain_error`.

What `explanation()` gave follows the file and the line in `what()`. `unavailable_feature_error` was thrown by no operation of GAOL v5: `atan2()`, which did, is implemented. [GAOL v5]

17.1.1 The `std::invalid_argument` exception

The `std::invalid_argument` exception is thrown whenever one attempts to create an interval from an invalid string. This situation may occur when reading an interval from a stream with the `>>` operator, or when reading a string with `gaol::textToInterval()`, from one string or from two, a null pointer being no string. The example prints what follows the first colon and the blank of `what()`, the explanation, which the file and the line precede.

Example

```
try {
  interval x = textToInterval("sin(1)+");
} catch (const std::invalid_argument& e) {
  string text = e.what();
  cout << text.substr(text.find(": ") + 2);
}
```

```
Syntax error in interval initialization: sin(1)+
```

17.1.2 The `std::domain_error` exception

This exception is thrown whenever a function is called with invalid arguments (e.g. calling `nb_fp_numbers()` with NaNs as parameters, or `bisect()` with a ratio outside $(0, 1)$), or a string calls a function with an argument it does not take (`"nth_root(8, 1.5)"`).

17.2 Warnings

void **gaol_warning** (const char **warn*)
void **gaol_warning** (const char **file*, int *line*, const char **warn*)

> Prints the message *warn* on the standard error output. The second form should be called with the `GAOL_FILE_POS` macro for the first two parameters. A null *file* is written `(unknown file)`, and a null *warn* is an empty text. [GAOL v5]

18 Debugging facilities

The debugging facilities described hereunder are available only if GAOL has been configured with the debugging facilities enabled, which the `Debug` build of each build does: `-DCMAKE_BUILD_TYPE=Debug` with CMake, the `--enable-debug` option of `configure` (see Section 2.5, page 9), or `--buildtype=debug` with meson, and `-DGAOL_DEBUG=ON` with CMake and `-Denable-debug=true` with meson whatever the build type. [GAOL v5] They define `GAOL_DEBUGGING` to 1 in `gaol/gaol_configuration.h`, where it is 0 otherwise, as every option there, so that it is read with `#if` (`#ifdef` is true of it in every build). The Debug builds compile without `NDEBUG` too, so that `assert()` stays active. [GAOL v5]

int **debug_level**

Global variable used to remember the current value of the debugging level. This variable is set when initializing the library. The variable is declared in the `gaol_core` namespace, and named in `gaol` too.

GAOL_DEBUG_VERBOSE (*lvl*,*cmd*) macro

Executes *cmd* if *lvl* is lower or equal to the current debugging level (see the variable `debug_level` above). It was named `GAOL_DEBUG` before GAOL v5, which is now the option of CMake that builds GAOL for debugging. [GAOL v5]
This macro defaults to nothing if the library was not configured with the debugging facilities.

A possible use for this macro is as follows:

Example

```
1 interval x(-10,10);
2
3 [Some code]
4
5 GAOL_DEBUG_VERBOSE(1,cout << "The value of x is " << x);
6 x += interval(3.5,4.5);
7 GAOL_DEBUG_VERBOSE(2,cout << "Now the value of x is " << x);
```

The first message will be displayed whenever the debugging level is at least 1, and the second one only if it is at least 2.

19 Additional Documentation

19.1 Documentation on GAOL

The primary reference is this manual. The directory `doc` of the sources holds the pages that give the details of GAOL v5, with the measures behind its choices:

- `doc/building.md`: the CMake, autotools and meson builds, and their options;
- `doc/using.md`: the flags of interval arithmetic, GAOL from CMake and from pkg-config, the namespaces and the rounding direction;
- `doc/three-builds.md`: what the three builds agree on, what the options are worth, and the compilers and options they refuse;
- `doc/tests.md`: what the programs of `tests` check;
- `doc/accuracy.md`: the tightness of each operation, with its algorithm and what the tests found;
- `doc/differences.md`: each change GAOL v5 brings to GAOL 4.2.2, and where it comes from;
- `doc/continuous-integration.md`: the systems, processors and compilers GAOL is built and tested on;
- `doc/compare/README.md`: GAOL compared with libieeep1788, filib++, PROFIL/BIAS, Solaris Studio and Boost.Interval, their special cases against IEEE 1788-2015, the enclosure of the elementary functions of GAOL and Boost.Interval, and their performance;
- `3rd/README.md`: the version of CORE-MATH GAOL takes, what differs from it, and the fixes to propose upstream.

The manual of GAOL 4, by Frédéric Goualard, is kept in `manual/v4`.

19.2 References

The following articles, books and standards have inspired in some way or another the devising of the GAOL library and/or the writing of this manual.

- *IEEE Standard for Interval Arithmetic.* IEEE Std 1788-2015 [9].
- *The CORE-MATH Project.* Alexei Sibidanov, Paul Zimmermann and Stéphane Glondu [13], and https://core-math.gitlabpages.inria.fr/.
- *How do you compute the midpoint of an interval?* Frédéric Goualard [5].
- *Fast and Correct SIMD Algorithms for Interval Arithmetic.* Frédéric Goualard [4].
- *Interval Extensions of Multivalued Inverse Functions.* Frédéric Goualard [3].
- *Interval Arithmetic Specification.* Dmitri Chiriaev and G. William Walster. Draft revised May 1998.
- *The Extended Real Interval System.* G. William Walster. April 1998.
- *C++ Interval Arithmetic Programming Reference.* Sun Microsystems, Inc. October 2000, revision A.
- *Interval Arithmetic: From Principles to Implementation.* T. Hickey, Q. Ju, and M. H. van Emden [7].

20 Reporting bugs

Bugs and suggestions for improvement of GAOL v5 shall be submitted as issues of its repository:

https://github.com/Jordan08/GAOL/issues

A report is most useful with the program that shows the problem, the interval it gives and the one expected, the system, the processor, the compiler and the options GAOL was built with.

21 Contributors

The main implementor and lead designer for the GAOL library is Frédéric Goualard (https://frederic.goualard.net).

GAOL v5 is written by Jordan Ninin (ENSTA), for Codac.

The `interval pow(const interval&, const interval&)` function was designed by Marc Christie.

The code for the multiplication and the division is largely inspired from the one presented by Tim Hickey, Qun Ju and Maarten Van Emden in *Interval Arithmetic: from Principles to Implementation*, Journal of the ACM 48(5):1038–1068, september 2001.

GAOL v5 takes up:

- the patch IBEX applies to GAOL, by Gilles Chabert (initialization of the floating-point unit, rounding direction of `midpoint()` and `operator<<`);
- the fork of GAOL by Fabrice Le Bars, for Visual C++, MinGW and the systems the `configure` of GAOL does not know;
- the CMake build of GAOL in IBEX, by Cyril Bouvier and Gilles Chabert, with the compilation flags of the IBEX fork of Fabrice Le Bars;
- the rounding tests of Codac, which its tests follow.

Its elementary functions are those of CORE-MATH, by Alexei Sibidanov, Paul Zimmermann and the other authors of the project (Inria).

22

Moving a program from GAOL 4 to GAOL v5

Most programs written for GAOL 4 compile and run with GAOL v5 once the namespace is opened and `textToInterval()` reads the strings they gave to `interval(const char*)`. This chapter lists what a program may have to change, and what it will see differently. The page `doc/differences.md` gives each change with its reasons and its measures.

22.1 Compiling and linking

- `gaol/gaol.h` no longer opens the namespace `gaol`: add `using namespace gaol;`, or name `gaol::interval` (see Section 3.4, page 27). The type and the functions are in `gaol_core`, and the symbols of the library name it: a program compiled against the headers of GAOL 4 is compiled again.
- `gaol` holds the names GAOL 4 had there that GAOL v5 still has, except those GAOL uses for itself, which a program names in `gaol_core` (see Section 3.4, page 27): `gaol_signbit()`, `uintdouble`, `ullidouble`, `Interval_struct`, `the_null_expr`, `prec_t`, `modulo_k_pi()`, the `round_*_sse()`, which the manual of GAOL 4 did not document, and, of the 4.3 releases of GAOL v5, `rounding_state`, `get_rounding()`, `set_rounding()`, `round_upward_if_needed()` and `gaol_set_rounding_x86()`.
- With Visual C++, the code using GAOL no longer defines `__GAOL_PUBLIC__` empty: GAOL is a static library there, and its declarations carry no `__declspec(dllimport)`. A definition left in a project changes nothing. The macro is `GAOL_PUBLIC` now.
- The allocator of containers `aligned_allocator` is gone, with its header, `gaol/gaol_allocator.h`.
- GAOL is the only library to link: the mathematical library GAOL 4 was built with is no longer used, CORE-MATH being compiled into GAOL, and the options of the builds that chose it are gone.

- The code including the headers of GAOL is compiled with the flags of interval arithmetic, which `gaol::gaol` and `gaol.pc` give (see Section 3.1, page 14); `gaol/gaol_config.h` refuses the options and the compilers that give wrong bounds (see Section 2.3.2, page 4).
- The intervals of floats, `gaol::intervalf` and `gaol::interval2f`, are compiled by none of the builds, which have no option for them: only a developer of GAOL compiles them, defining `GAOL_FLOAT_INTERVALS` in `gaol/gaol_config.h`.
- The functions of doubles rounded downward and upward, `exp_dn()`, `sin_up()` and the others of `gaol/gaol_double_op.h`, are no longer part of the interface: the header is no longer installed.
- The static intervals `interval::cst_one`, `cst_pi` and the other `cst_` members are gone: `interval::one()`, `interval::pi()` and the other functions give these constants, right even in the initialization of a static object (see Section 4, page 35).
- `gaol::init()` need not be called again after the program changed the rounding direction: each operation sets it upward itself (see Section 3.2, page 17). It need not be called at all, GAOL initializing itself with every compiler, Visual C++ included (see Section 4, page 35).
- The program calls `gaol::cleanup()` right after its last use of GAOL: unless the rounding direction is preserved, it sets back the rounding direction that the initialization found, which GAOL 4 left upward (see Section 4, page 35). It no longer frees what the initialization allocated, which the automatic cleanup frees when the program ends.
- The operators `==` and `!=` on intervals are gone, with the possibly relations (`possibly_eq()`, `possibly_le()`...), `certainly_eq()` and `certainly_neq()`, and so is the option `--enable-relations` of `configure` and meson: a program compares intervals with `set_eq()`, `set_disjoint()` and `!set_disjoint()`, and `<`, `<=`, `>` and `>=` are the certainly relations (see Section 7, page 47).
- The constructor from one string, `interval(const char*)`, is gone, and with it the conversion of a string wherever an interval is expected: `textToInterval("0.1")` reads a string in its place, and throws as it did; `x = "[1, 2]"` becomes `x = textToInterval("[1, 2]")`, and `x + "0.1"` becomes `x + textToInterval("0.1")` (see Section 5.1, page 39). The constructor from two strings is gone too: `interval("0.1", "0.3")` becomes `textToInterval("0.1", "0.3")`, which gives the empty set for bounds in the wrong order. `interval(0)` and `interval(0, 0)`, which these constructors made ambiguous, compile.

22.2 Results that change

- **Static objects.** The intervals computed in the initialization of the static objects of a program linked with the static library GAOL are right: `interval::pi()` was $[-0,0]$ there, and $[1,2]/[-1,1]$ was $[-0,0]$ rather than $[-\infty,+\infty]$, the constants of GAOL being computed after these objects, except with MinGW-w64 (see Section 4, page 35).

- **The bounds are tighter**: the elementary functions give the tightest bounds, rather than one double beyond, and are exact where their value is a double; the integer powers and the nth roots are computed from exact products; `sin`, `cos` and `tan` are within one double of the tightest bounds at every magnitude (Chapter 12).

- **The empty set.** The constructors give it for $[+\infty, +\infty]$, $[-\infty, -\infty]$, bounds in the wrong order and NaN bounds, and assigning an infinity to an interval empties it (see Section 5.1, page 39); the intersection of disjoint intervals is it, with NaN bounds; so are `log` of an interval holding no positive number, `invabs_rel` of a negative interval and `div_rel` with no solution in I. `width()` of the empty set is NaN rather than -1.

- **Powers.** `pow(x, p)` for a double p no longer truncates p; `pow(x, y)` keeps the negative part of x for an integer exponent only, and takes the `pow` of IEEE 1788-2015 otherwise (see Section 8.1.2, page 59). `nth_root` of an odd order is defined on the whole real line, and takes a negative order.

- **The comparisons** follow IEEE 1788-2015 for the empty set and the infinite bounds: `certainly_leq()` and its companions are true when either interval is empty, and `set_strictly_contains()` takes an infinite bound as beyond itself (see Section 7, page 47).

- **Input and output.** Numbers are read exactly, and the literals of IEEE 1788-2015 are read, in any case of letters; `"[inf]"` is the empty set. Every string is an expression, where an interval may stand wherever a number may (`"1+[1,2]"`), an uncertain number being no bound (`"[5?1]"` is refused), and an error anywhere stops the reading. The reader knows every function of GAOL, and `gaol_ieee1788::textToInterval` reads the names of IEEE 1788-2015 (`pown`, `rootn`...). The format `hexa` writes interval literals, which are read back bit for bit; the decimal formats round the bounds outward whatever the C library, and the format `agreeing` keeps the digits of each bound (Chapter 10).

- **New operations**: `atan2()` is implemented, and the functions of IEEE 1788-2015 GAOL did not provide are added (`exp2`, `hypot`, `sign`, `trunc`..., Chapter 8).

Library Copying

GNU LIBRARY GENERAL PUBLIC LICENSE

Version 2, June 1991

Copyright © 1991 Free Software Foundation, Inc. 675 Mass Ave, Cambridge, MA 02139, USA

Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed.

[This is the first released version of the library GPL. It is numbered 2 because it goes with version 2 of the ordinary GPL.]

Preamble

The licenses for most software are designed to take away your freedom to share and change it. By contrast, the GNU General Public Licenses are intended to guarantee your freedom to share and change free software–to make sure the software is free for all its users.

This license, the Library General Public License, applies to some specially designated Free Software Foundation software, and to any other libraries whose authors decide to use it. You can use it for your libraries, too.

When we speak of free software, we are referring to freedom, not price. Our General Public Licenses are designed to make sure that you have the freedom to distribute copies of free software (and charge for this service if you wish), that you receive source code or can get it if you want it, that you can change the software or use pieces of it in new free programs; and that you know you can do these things.

To protect your rights, we need to make restrictions that forbid anyone to deny you these rights or to ask you to surrender the rights. These restrictions translate to certain responsibilities for you if you distribute copies of the library, or if you modify it.

For example, if you distribute copies of the library, whether gratis or for a fee, you must give the recipients all the rights that we gave you. You must make sure that they, too, receive or can get the source code. If you link a program with the library, you must provide complete object files to the recipients so that they can relink them with the library, after making changes to the library and recompiling it. And you must show them these terms so they know their rights.

Our method of protecting your rights has two steps: (1) copyright the library, and (2) offer you this license which gives you legal permission to copy, distribute and/or modify the library.

Also, for each distributor's protection, we want to make certain that everyone understands that there is no warranty for this free library. If the library is modified by someone else and passed on, we want its recipients to know that

what they have is not the original version, so that any problems introduced by others will not reflect on the original authors' reputations.

Finally, any free program is threatened constantly by software patents. We wish to avoid the danger that companies distributing free software will individually obtain patent licenses, thus in effect transforming the program into proprietary software. To prevent this, we have made it clear that any patent must be licensed for everyone's free use or not licensed at all.

Most GNU software, including some libraries, is covered by the ordinary GNU General Public License, which was designed for utility programs. This license, the GNU Library General Public License, applies to certain designated libraries. This license is quite different from the ordinary one; be sure to read it in full, and don't assume that anything in it is the same as in the ordinary license.

The reason we have a separate public license for some libraries is that they blur the distinction we usually make between modifying or adding to a program and simply using it. Linking a program with a library, without changing the library, is in some sense simply using the library, and is analogous to running a utility program or application program. However, in a textual and legal sense, the linked executable is a combined work, a derivative of the original library, and the ordinary General Public License treats it as such.

Because of this blurred distinction, using the ordinary General Public License for libraries did not effectively promote software sharing, because most developers did not use the libraries. We concluded that weaker conditions might promote sharing better.

However, unrestricted linking of non-free programs would deprive the users of those programs of all benefit from the free status of the libraries themselves. This Library General Public License is intended to permit developers of non-free programs to use free libraries, while preserving your freedom as a user of such programs to change the free libraries that are incorporated in them. (We have not seen how to achieve this as regards changes in header files, but we have achieved it as regards changes in the actual functions of the Library.) The hope is that this will lead to faster development of free libraries.

The precise terms and conditions for copying, distribution and modification follow. Pay close attention to the difference between a "work based on the library" and a "work that uses the library". The former contains code derived from the library, while the latter only works together with the library.

Note that it is possible for a library to be covered by the ordinary General Public License rather than by this special one.

Terms and Conditions for Copying, Distribution and Modification

0. This License Agreement applies to any software library which contains a notice placed by the copyright holder or other authorized party saying it may be distributed under the terms of this Library General Public License (also called "this License"). Each licensee is addressed as "you".

   A "library" means a collection of software functions and/or data prepared so as to be conveniently linked with application programs (which use some of those functions and data) to form executables.

   The "Library", below, refers to any such software library or work which has been distributed under these terms. A "work based on the Library" means either the Library or any derivative work under copyright law: that is to say, a work containing the Library or a portion of it, either

verbatim or with modifications and/or translated straightforwardly into another language. (Hereinafter, translation is included without limitation in the term "modification".)

"Source code" for a work means the preferred form of the work for making modifications to it. For a library, complete source code means all the source code for all modules it contains, plus any associated interface definition files, plus the scripts used to control compilation and installation of the library.

Activities other than copying, distribution and modification are not covered by this License; they are outside its scope. The act of running a program using the Library is not restricted, and output from such a program is covered only if its contents constitute a work based on the Library (independent of the use of the Library in a tool for writing it). Whether that is true depends on what the Library does and what the program that uses the Library does.

1. You may copy and distribute verbatim copies of the Library's complete source code as you receive it, in any medium, provided that you conspicuously and appropriately publish on each copy an appropriate copyright notice and disclaimer of warranty; keep intact all the notices that refer to this License and to the absence of any warranty; and distribute a copy of this License along with the Library.

   You may charge a fee for the physical act of transferring a copy, and you may at your option offer warranty protection in exchange for a fee.

2. You may modify your copy or copies of the Library or any portion of it, thus forming a work based on the Library, and copy and distribute such modifications or work under the terms of Section 1 above, provided that you also meet all of these conditions:

   a. The modified work must itself be a software library.

   b. You must cause the files modified to carry prominent notices stating that you changed the files and the date of any change.

   c. You must cause the whole of the work to be licensed at no charge to all third parties under the terms of this License.

   d. If a facility in the modified Library refers to a function or a table of data to be supplied by an application program that uses the facility, other than as an argument passed when the facility is invoked, then you must make a good faith effort to ensure that, in the event an application does not supply such function or table, the facility still operates, and performs whatever part of its purpose remains meaningful.

      (For example, a function in a library to compute square roots has a purpose that is entirely well-defined independent of the application. Therefore, Subsection 2d requires that any application-supplied function or table used by this function must be optional: if the application does not supply it, the square root function must still compute square roots.)

   These requirements apply to the modified work as a whole. If identifiable sections of that work are not derived from the Library, and can be reasonably considered independent and separate works in themselves, then this License, and its terms, do not apply to those sections when you

distribute them as separate works. But when you distribute the same sections as part of a whole which is a work based on the Library, the distribution of the whole must be on the terms of this License, whose permissions for other licensees extend to the entire whole, and thus to each and every part regardless of who wrote it.

Thus, it is not the intent of this section to claim rights or contest your rights to work written entirely by you; rather, the intent is to exercise the right to control the distribution of derivative or collective works based on the Library.

In addition, mere aggregation of another work not based on the Library with the Library (or with a work based on the Library) on a volume of a storage or distribution medium does not bring the other work under the scope of this License.

3. You may opt to apply the terms of the ordinary GNU General Public License instead of this License to a given copy of the Library. To do this, you must alter all the notices that refer to this License, so that they refer to the ordinary GNU General Public License, version 2, instead of to this License. (If a newer version than version 2 of the ordinary GNU General Public License has appeared, then you can specify that version instead if you wish.) Do not make any other change in these notices.

   Once this change is made in a given copy, it is irreversible for that copy, so the ordinary GNU General Public License applies to all subsequent copies and derivative works made from that copy.

   This option is useful when you wish to copy part of the code of the Library into a program that is not a library.

4. You may copy and distribute the Library (or a portion or derivative of it, under Section 2) in object code or executable form under the terms of Sections 1 and 2 above provided that you accompany it with the complete corresponding machine-readable source code, which must be distributed under the terms of Sections 1 and 2 above on a medium customarily used for software interchange.

   If distribution of object code is made by offering access to copy from a designated place, then offering equivalent access to copy the source code from the same place satisfies the requirement to distribute the source code, even though third parties are not compelled to copy the source along with the object code.

5. A program that contains no derivative of any portion of the Library, but is designed to work with the Library by being compiled or linked with it, is called a "work that uses the Library". Such a work, in isolation, is not a derivative work of the Library, and therefore falls outside the scope of this License.

   However, linking a "work that uses the Library" with the Library creates an executable that is a derivative of the Library (because it contains portions of the Library), rather than a "work that uses the library". The executable is therefore covered by this License. Section 6 states terms for distribution of such executables.

   When a "work that uses the Library" uses material from a header file that is part of the Library, the object code for the work may be a derivative work of the Library even though the source code is not. Whether this is true is especially significant if the work can be linked without the

Library, or if the work is itself a library. The threshold for this to be true is not precisely defined by law.

If such an object file uses only numerical parameters, data structure layouts and accessors, and small macros and small inline functions (ten lines or less in length), then the use of the object file is unrestricted, regardless of whether it is legally a derivative work. (Executables containing this object code plus portions of the Library will still fall under Section 6.)

Otherwise, if the work is a derivative of the Library, you may distribute the object code for the work under the terms of Section 6. Any executables containing that work also fall under Section 6, whether or not they are linked directly with the Library itself.

6. As an exception to the Sections above, you may also compile or link a "work that uses the Library" with the Library to produce a work containing portions of the Library, and distribute that work under terms of your choice, provided that the terms permit modification of the work for the customer's own use and reverse engineering for debugging such modifications.

   You must give prominent notice with each copy of the work that the Library is used in it and that the Library and its use are covered by this License. You must supply a copy of this License. If the work during execution displays copyright notices, you must include the copyright notice for the Library among them, as well as a reference directing the user to the copy of this License. Also, you must do one of these things:

   a. Accompany the work with the complete corresponding machine-readable source code for the Library including whatever changes were used in the work (which must be distributed under Sections 1 and 2 above); and, if the work is an executable linked with the Library, with the complete machine-readable "work that uses the Library", as object code and/or source code, so that the user can modify the Library and then relink to produce a modified executable containing the modified Library. (It is understood that the user who changes the contents of definitions files in the Library will not necessarily be able to recompile the application to use the modified definitions.)

   b. Accompany the work with a written offer, valid for at least three years, to give the same user the materials specified in Subsection 6a, above, for a charge no more than the cost of performing this distribution.

   c. If distribution of the work is made by offering access to copy from a designated place, offer equivalent access to copy the above specified materials from the same place.

   d. Verify that the user has already received a copy of these materials or that you have already sent this user a copy.

   For an executable, the required form of the "work that uses the Library" must include any data and utility programs needed for reproducing the executable from it. However, as a special exception, the source code distributed need not include anything that is normally distributed (in either source or binary form) with the major components (compiler, kernel, and so on) of the operating system on which the executable runs, unless that component itself accompanies the executable.

It may happen that this requirement contradicts the license restrictions of other proprietary libraries that do not normally accompany the operating system. Such a contradiction means you cannot use both them and the Library together in an executable that you distribute.

7. You may place library facilities that are a work based on the Library side-by-side in a single library together with other library facilities not covered by this License, and distribute such a combined library, provided that the separate distribution of the work based on the Library and of the other library facilities is otherwise permitted, and provided that you do these two things:

    a. Accompany the combined library with a copy of the same work based on the Library, uncombined with any other library facilities. This must be distributed under the terms of the Sections above.

    b. Give prominent notice with the combined library of the fact that part of it is a work based on the Library, and explaining where to find the accompanying uncombined form of the same work.

8. You may not copy, modify, sublicense, link with, or distribute the Library except as expressly provided under this License. Any attempt otherwise to copy, modify, sublicense, link with, or distribute the Library is void, and will automatically terminate your rights under this License. However, parties who have received copies, or rights, from you under this License will not have their licenses terminated so long as such parties remain in full compliance.

9. You are not required to accept this License, since you have not signed it. However, nothing else grants you permission to modify or distribute the Library or its derivative works. These actions are prohibited by law if you do not accept this License. Therefore, by modifying or distributing the Library (or any work based on the Library), you indicate your acceptance of this License to do so, and all its terms and conditions for copying, distributing or modifying the Library or works based on it.

10. Each time you redistribute the Library (or any work based on the Library), the recipient automatically receives a license from the original licensor to copy, distribute, link with or modify the Library subject to these terms and conditions. You may not impose any further restrictions on the recipients' exercise of the rights granted herein. You are not responsible for enforcing compliance by third parties to this License.

11. If, as a consequence of a court judgment or allegation of patent infringement or for any other reason (not limited to patent issues), conditions are imposed on you (whether by court order, agreement or otherwise) that contradict the conditions of this License, they do not excuse you from the conditions of this License. If you cannot distribute so as to satisfy simultaneously your obligations under this License and any other pertinent obligations, then as a consequence you may not distribute the Library at all. For example, if a patent license would not permit royalty-free redistribution of the Library by all those who receive copies directly or indirectly through you, then the only way you could satisfy both it and this License would be to refrain entirely from distribution of the Library.

If any portion of this section is held invalid or unenforceable under any particular circumstance, the balance of the section is intended to apply, and the section as a whole is intended to apply in other circumstances.

It is not the purpose of this section to induce you to infringe any patents or other property right claims or to contest validity of any such claims; this section has the sole purpose of protecting the integrity of the free software distribution system which is implemented by public license practices. Many people have made generous contributions to the wide range of software distributed through that system in reliance on consistent application of that system; it is up to the author/donor to decide if he or she is willing to distribute software through any other system and a licensee cannot impose that choice.

This section is intended to make thoroughly clear what is believed to be a consequence of the rest of this License.

12. If the distribution and/or use of the Library is restricted in certain countries either by patents or by copyrighted interfaces, the original copyright holder who places the Library under this License may add an explicit geographical distribution limitation excluding those countries, so that distribution is permitted only in or among countries not thus excluded. In such case, this License incorporates the limitation as if written in the body of this License.

13. The Free Software Foundation may publish revised and/or new versions of the Library General Public License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns.

    Each version is given a distinguishing version number. If the Library specifies a version number of this License which applies to it and "any later version", you have the option of following the terms and conditions either of that version or of any later version published by the Free Software Foundation. If the Library does not specify a license version number, you may choose any version ever published by the Free Software Foundation.

14. If you wish to incorporate parts of the Library into other free programs whose distribution conditions are incompatible with these, write to the author to ask for permission. For software which is copyrighted by the Free Software Foundation, write to the Free Software Foundation; we sometimes make exceptions for this. Our decision will be guided by the two goals of preserving the free status of all derivatives of our free software and of promoting the sharing and reuse of software generally.

NO WARRANTY

11. BECAUSE THE LIBRARY IS LICENSED FREE OF CHARGE, THERE IS NO WARRANTY FOR THE LIBRARY, TO THE EXTENT PERMITTED BY APPLICABLE LAW. EXCEPT WHEN OTHERWISE STATED IN WRITING THE COPYRIGHT HOLDERS AND/OR OTHER PARTIES PROVIDE THE LIBRARY "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE ENTIRE RISK AS TO THE

QUALITY AND PERFORMANCE OF THE LIBRARY IS WITH YOU. SHOULD THE LIBRARY PROVE DEFECTIVE, YOU ASSUME THE COST OF ALL NECESSARY SERVICING, REPAIR OR CORRECTION.

12. IN NO EVENT UNLESS REQUIRED BY APPLICABLE LAW OR AGREED TO IN WRITING WILL ANY COPYRIGHT HOLDER, OR ANY OTHER PARTY WHO MAY MODIFY AND/OR REDISTRIBUTE THE LIBRARY AS PERMITTED ABOVE, BE LIABLE TO YOU FOR DAMAGES, INCLUDING ANY GENERAL, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OR INABILITY TO USE THE LIBRARY (INCLUDING BUT NOT LIMITED TO LOSS OF DATA OR DATA BEING RENDERED INACCURATE OR LOSSES SUSTAINED BY YOU OR THIRD PARTIES OR A FAILURE OF THE LIBRARY TO OPERATE WITH ANY OTHER SOFTWARE), EVEN IF SUCH HOLDER OR OTHER PARTY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

How to Apply These Terms to Your New Libraries

If you develop a new library, and you want it to be of the greatest possible use to the public, we recommend making it free software that everyone can redistribute and change. You can do so by permitting redistribution under these terms (or, alternatively, under the terms of the ordinary General Public License).

To apply these terms, attach the following notices to the library. It is safest to attach them to the start of each source file to most effectively convey the exclusion of warranty; and each file should have at least the "copyright" line and a pointer to where the full notice is found.

> *one line to give the library's name and a brief idea of what it does.*
> Copyright (C) *year name of author*
>
> This library is free software; you can redistribute it and/or modify it under the terms of the GNU Library General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.
>
> This library is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU Library General Public License for more details.
>
> You should have received a copy of the GNU Library General Public License along with this library; if not, write to the Free Software Foundation, Inc., 675 Mass Ave, Cambridge, MA 02139, USA.

Also add information on how to contact you by electronic and paper mail.

You should also get your employer (if you work as a programmer) or your school, if any, to sign a "copyright disclaimer" for the library, if necessary. Here is a sample; alter the names:

> Yoyodyne, Inc., hereby disclaims all copyright interest in the library 'Frob' (a library for tweaking knobs) written by James Random Hacker.
>
> *signature of Ty Coon*, 1 April 1990
> Ty Coon, President of Vice

That's all there is to it!

The license of CORE-MATH

The sources of CORE-MATH, in `3rd/math-core`, which GAOL compiles into its library, are distributed under the following license (MIT license):

```
Permission is hereby granted, free of charge, to any person
obtaining a copy of this software and associated documentation
files (the "Software"), to deal in the Software without
restriction, including without limitation the rights to use,
copy, modify, merge, publish, distribute, sublicense, and/or
sell copies of the Software, and to permit persons to whom the
Software is furnished to do so, subject to the following
conditions:

The above copyright notice and this permission notice shall be
included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND,
EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES
OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND
NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT
HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY,
WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING
FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR
OTHER DEALINGS IN THE SOFTWARE.
```

Bibliography

[1] Götz Alefeld and Jürgen Herzberger. *Introduction to Interval Computations*. Academic Press Inc., New York, USA, 1983. Traduit par Jon Rokne de l'original Allemand *'Einführung In Die Intervallrechnung'*.

[2] David Marc Goldberg. What every computer scientist should know about floating-point arithmetic. *ACM Computing Surveys*, 23(1):5–48, March 1991.

[3] Frédéric Goualard. Interval extensions of multivalued inverse functions. Technical Report hal-00288457, HAL, 2008.

[4] Frédéric Goualard. Fast and correct SIMD algorithms for interval arithmetic. In *PARA '08, Lecture Notes in Computer Science 6126–6127*. Springer, 2012.

[5] Frédéric Goualard. How do you compute the midpoint of an interval? *ACM Transactions on Mathematical Software*, 40(2), 2014.

[6] Eldon Robert Hansen. *Global Optimization Using Interval Analysis*. Pure and Applied Mathematics. Marcel Dekker Inc., 1992.

[7] Timothy J. Hickey, Qun Ju, and Maarten H. van Emden. Interval arithmetic: from principles to implementation. *Journal of the ACM*, 48(5):1038–1068, 2001.

[8] IEEE. IEEE standard for binary floating-point arithmetic. Technical Report IEEE Std 754-1985, Institute of Electrical and Electronics Engineers, 1985. Reaffirmed 1990.

[9] IEEE. *IEEE Standard for Interval Arithmetic*. Institute of Electrical and Electronics Engineers, 2015. IEEE Std 1788-2015, doi:10.1109/IEEESTD.2015.7140721.

[10] Ramon Edgar Moore. *Interval Analysis*. Prentice-Hall, Englewood Cliffs, N. J., 1966.

[11] Arnold Neumaier. *Interval methods for systems of equations*, volume 37 of *Encyclopedia of Mathematics and its Applications*. Cambridge University Press, 1990.

[12] Helmut Ratschek and Jon Rokne. Interval methods. In *Handbook of Global Optimization*, pages 751–828. Kluwer Academic, 1995.

[13] Alexei Sibidanov, Paul Zimmermann, and Stéphane Glondu. The CORE-MATH project. In *2022 IEEE 29th Symposium on Computer Arithmetic (ARITH)*, pages 26–34, 2022.

[14] Volker Stahl. *Interval Methods for Bounding the Range of Polynomials and Solving Systems of Nonlinear Equations.* Phd. thesis, Johannes Kepler Universität, Linz, September 1995.

Index

-Ofast, 21

↓↓, *see* downward rounding
↑↑, *see* upward rounding
abs, **76**
accuracy
    of the operations, 101
acos, **63**
acos_rel, **67**
acosh, **64**
acosh_rel, **67**
acospi, **64**
asin, **63**
asin_rel, **67**
asinh, **65**
asinh_rel, **68**
asinpi, **64**
atan, **63**
atan2, **63**
atan2_rel_x, **71**
atan2_rel_y, **71**
atan2pi, **64**
atan_rel, **67**
atanh, **65**
atanh_rel, **68**
atanpi, **64**
AVX-512, 8

bisect, **79**
bisection
    of an unbounded interval, 33
bison, 5
Boost.Interval, 97

c++
    standard, 4
C-XSC, 97
cancel_minus, **72**
cancel_plus, **72**
cancellative operations, 72
canonical interval, **30**
ceil, **77**
cell, vii
certainly_ge, **50**
certainly_geq, **50**
certainly_le, **50**
certainly_leq, **50**
certainly_negative, **51**
certainly_positive, **50**
certainly_strictly_negative, **51**
certainly_strictly_positive, **51**
chi, **76**
cleanup, **36**
CMake, 6
    using gaol from, 16
Codac, 97
compiler
    refused, 4
compiling
    a file using gaol, 14
compoundm1, **65**
configure, 9
containement property, 17
CORE-MATH, vii, 61
cos, **62**
cosh, **64**
cospi, **64**
crtfastmath.o, 21

debug_level, **117**
debugging
    option, 9
denormals-are-zero, 21
dependency problem, 14
div_rel, **69**
domain of a function, 31, 57
downward rounding (↓↓), 17

empty
    interval: constructing, 39
    interval: printing, 86
    interval: representation, 45
[empty], 86
empty set

in tests, 31
erf, **66**
erfc, **66**
even, **111**
exact_string, **86**
exceptions
floating-point, 24
exp, **61**
exp10, **61**
exp10m1, **61**
exp2, **61**
exp2m1, **61**
expm1, **61**

𝔽, 17
feven, **105**
flags
floating-point, 24
flags of interval arithmetic, 14
flex, 5
floating-point exceptions, 24
floor, **77**
floor_rel, **70**
flush-to-zero, 21
fma, **59**
format, **88**, **89**
function
undocumented, 2
fused multiply-add, 7

gaol
namespace, 13
pronunciation, 1
web page, 3
GAOL 4
moving from, 125
GAOL v5, 1
GAOL_DEBUG_VERBOSE, **117**
gaol_ERROR, **113**
gaol_error, **114**
GAOL_FILE_POS, **114**
gaol_warning, **115**

hausdorff, **75**
header
and namespace, *see* namespace
help
on configuration, 9
hull, **80**
hypot, **65**

IBEX, 97
IEEE 1788-2015, 1
names, 93
relations, 47
inflate, **74**
init, **35**
integer, **77**
intersect, **80**
interval
canonical, *see* canonical interval
computation site (web), 2
conversion to string, 86
midpoint and radius, 30
with infinite bounds, 42
zero, 31
interval::format
threads, 34
interval_format, **89**
invabs_rel, **69**
inverse, **58**
is_a_double, **55**
is_an_int, **55**
is_bisectable, **79**
is_canonical, **55**
is_common_interval, **56**
is_empty, **55**
is_entire, **55**
is_finite, **56**
is_signed, **106**
is_symmetric, **56**
is_zero, **56**

left, **76**
less, **51**
lexicographic_less, 53
libraries
names in other, 97
license
of CORE-MATH, vii
log, **61**
log10, **62**
log10p1, **62**
log1p, **62**
log2, **62**
log2p1, **62**

mag, **75**
max, **77**
max_rel, **70**
maximum, **106**
meson, 10
using gaol from, 16
mid, **75**
mid_rad, **73**
midpoint, **75**
midrad, **41**
mig, **74**

min, **77**
min_rel, **70**
minimum, **106**

namespace
  gaol, 13, 27
  gaol_core, 27
  gaol_ieee1788, 27, 93
nb_fp_numbers, **106**
NEON, 7, 21
Newton's method
  division, 33
next_float, **105**
nth_root, **60**
nth_root_rel, **68**
number
  reading, 83

odd, **111**
operator
  relational, 1
operator*, **58**
operator*=, **44**
operator+, **58**
operator+=, **43**
operator-, **58**
operator-=, **44**
operator/, **58**
operator/=, **44**
operator<, **52**
operator<=, **52**
operator<<, **86**
operator>, **53**
operator>=, **53**
operator>>, **81**
operator%, **58**, **66**
operator% , **66**
operator%=, **44**
operator%= , **44**
operator&, **79**
operator&=, **43**
operator|, **80**
operator|=, **43**
optimization
  configuration option, 9
  interprocedural, 8
outward rounding, 17

package, 8
pkg-config, 16
platforms, 3
pow
  in the two namespaces, 28
pow, **59**
pow_exponent_rel, **69**
pow_rel, **69**
power, 59
precision, **90**
previous_float, **105**
PROFIL/BIAS, 97

$\mathbb{R}$, 17
rad, **73**
references
  on interval arithmetic, 2
relation
  possibly (removed), 47
  symbols, 52
    pitfalls, 32
relational operator, 1
restore_rounding, **36**
reverse functions, *see* relational arithmetic
right, **76**
round_downward, **107**
round_nearest, **107**
round_ties_to_away, **78**
round_ties_to_even, **78**
round_upward, **107**
Rounding
  to nearest, **29**
rounding
  direction, 17
  downward, **17**
  effects on the program, 18
  left as found, 18
  outward, *see* outward rounding
  preserving, 7, 9, 18
  upward, **17**
rsqrt, **65**

self, 43
set_contains, **47**
set_disjoint, **48**
set_eq , **48**
set_ge, **49**
set_geq, **49**
set_le, **49**
set_leq, **49**
set_neq, **49**
set_strictly_contains, **48**
sign, **78**
sign_rel, **70**
sin, **62**
sinh, **65**
sinpi, **64**
smig, **74**
sorting intervals, 53
split, **78**

`split_left`, **78**
`split_right`, **78**
`sqr`, **59**
`sqrt` of a number, 30
`sqrt`, **59**
`sqrt_rel`, **68**
SSE2, 7
`std::set`, 53
`straddles_zero`, **54**
`strictly_less`, **52**
`strictly_straddles_zero`, **54**
`string`
    converting interval to, *see* interval
subnormal numbers, 21

`tan`, **62**
`tanh`, **65**
`tanpi`, **64**
tests, 11
`textToInterval`
    errors, 33
`textToInterval`, **40**, **41**
`this`, 43
threads
    output of intervals, 34
tightness, 101
tool
    mandatory ~ to compile gaol, 4
`trunc`, **78**
trust rounding mode, 17

undocumented
    use of ~ functions, 2
upward rounding (↑↑), 17

`version`, **109**
`version_major`, **109**
`version_micro`, **109**
`version_minor`, **109**

`width`, **73**
`width_enclosure`, **73**

zero
    sign of a bound, 31